

A Dynamic Radial Membrane Architecture for a U.F.O.-Enhanced AI Governor

Don Michael Feeney Jr.

First published on June 13th, 2026

Final Version: 2.6.0

Completed on June 18th, 2026

This paper is offered as a conceptual research supplement supported by a lightweight toy simulation, rather than as a completed production implementation or empirical deployment result. My goal is to demonstrate systems thinking around personalization, compute-aware AI, simulation design, validation strategy, and responsible integration. I would welcome the opportunity to refine, challenge, simulate further, or revise this architecture collaboratively with Microsoft research and product teams.

Executive Summary

AI identity can be modeled not as a fixed personality layer but as a controllable, user-shaped, compute-aware dynamical system. This paper introduces a unified architecture—the **radial identity membrane**—that integrates personalization, behavioral control, and compute governance into a single model. In this model, **U.F.O.** is short for **User-Formed Optimization**: a user-shaped control process in which preference, behavior, compute allocation, and governed adaptation are optimized together rather than treated as separate layers.

At its center, the system represents AI behavior as a **twelve-string membrane** spanning traits such as Depth, Precision, Context Sensitivity, Initiative, Exploration, Creativity, Tone, Transparency, Technical Detail, Structural Rigor, Emotional Warmth, and Conciseness. Each string shapes both behavioral expression and **computational cost**, allowing identity and resource usage to be governed together.

The membrane gives behavior a physical representation: amplitude reflects behavioral intensity, radius reflects reasoning depth, and thickness reflects compute load. Adjacent traits blend into **V-shaped reasoning channels**, enabling intelligence to flow smoothly through aligned behaviors rather than switching abruptly between modes. These channels create **personalizable corridors of reasoning** that adapt to users, tasks, and enterprise contexts.

Each string carries vibrational parameters—wavelength, frequency, tension, stiffness, and damping—that determine activation speed, spread, cost, and stability. Dynamic tension and stiffness allow the system to self-regulate under compute pressure, tightening when load increases and relaxing when stable.

A **governor layer** provides system-level stability. Through local cost equations, inverse-cost routing, bounded depth updates, suppression dynamics, and a **candidate Lyapunov energy**, the

governor maintains a **bounded compute envelope**. It can allocate additional compute when task value warrants it, but otherwise biases behavior toward lower energy, reduced imbalance, and efficient use of context, tokens, and tools.

Adaptation is framed as **data flow across the membrane**. Inputs excite patterns across the twelve strings, propagate through curved channels, and gradually reshape the membrane. Training and personalization therefore modify the system as a **coupled dynamical identity**, not as isolated behavioral sliders.

This paper presents a **conceptual architecture supported by a lightweight toy simulation**, not a completed production implementation or empirical deployment result. The companion *Project Rainbow* notebook implements the membrane’s state arrays, task signatures, curvature matrix, update loop, ablation configurations, and visualization routines; Appendix F summarizes the toy-simulation observations. The testable claim is that personalization, behavioral control, and compute optimization can be unified in a single model with coherent mechanisms, measurable signals, and a practical validation path.

The initial toy run, executed locally in Jupyter Notebook through *Project Rainbow*, produced several directional findings: task signatures activated the expected behavioral sectors, low-pressure tasks remained bounded and compute-efficient, curvature supported coherent V-channel routing, the governor prevented runaway expansion in ablation tests, and the full adaptive configuration showed the strongest combination of coherence, restraint, and recovery. These findings are **preliminary and limited to the toy-model setting**, but they show that the architecture can be parameterized, visualized, ablated, and inspected before prototype-level validation.

For Microsoft-scale systems, the strategic value is potentially significant: **resource-aware personalization**. The same mechanism that improves alignment with users can also reduce unnecessary deep reasoning, excessive retrieval, long outputs, and avoidable tool calls. If validated, the radial identity membrane could enable AI systems that are more personal, stable, transparent, and compute-efficient at scale.

Scope and Status

This paper is intended as a **conceptual research architecture supported by a lightweight toy simulation**, rather than a completed empirical implementation or production system. The equations and geometric structures define a proposed control framework for future modeling and validation, while the companion *Project Rainbow* notebook provides an initial numerical exercise of the framework. The notebook implements the membrane state arrays, task signatures, curvature matrix, update loop, ablation configurations, and visualization routines used to generate the toy-simulation observations summarized in Appendix F. These results should be read as evidence of internal coherence, routing behavior, and stability dynamics within a simplified model—not as empirical validation of deployed AI behavior.

Accordingly, the current status of the work is best described as: **formal architecture plus reproducible toy simulation**. The simulation exercises the proposed mechanisms at a small scale—initialization, task-driven activation, V-channel curvature, inverse-cost routing, adaptive tension and stiffness, governor suppression, Lyapunov-style energy tracking, and ablation comparisons. It demonstrates that the model can be parameterized, run, visualized, and stress-tested, but further work is required before making claims about real user outcomes, production compute savings, or deployment impact at Microsoft scale.

This architecture does **not define personality traits**; it defines a compute-aware behavioral control surface that adapts to user interaction under explicit stability constraints. In its first deployable form, the membrane operates as a **control layer over model behavior** rather than modifying underlying model parameters. All behavioral modulation remains subject to **safety, policy, and alignment constraints** enforced by the underlying model and platform. In this sense, the U.F.O. governor functions as a supervisory orchestration layer for AI behavior—not the model’s cognition itself, but the control system that coordinates behavioral activation, compute allocation, context use, and durable adaptation. This deployment boundary is why we introduce Elder as a reference notation for expressing the governor as typed, inspectable orchestration logic.

Technical note on Elder: Because the architecture is proposed as a control layer rather than a model-weight modification, Appendix G introduces *Elder* as a systems-native reference notation for expressing the U.F.O. governor. Elder is not a production language; it is a typed, inspectable DSL used to make the control-layer boundary explicit. Its design combines the performance orientation of C++, the orchestration clarity of Python, and the expressive readability of Ruby, while adopting the safety discipline and determinism associated with modern systems languages. Elder exists solely to illustrate how the governor’s compute-bounded, policy-aware, telemetry-driven behavior can be represented as a structured control surface above an existing AI model. It is not required for the architecture, but it provides a clear, implementation-shaped way to visualize how the U.F.O. governor would operate inside Windows Copilot.

This proposal complements, rather than replaces, existing AI alignment and personalization methods. RLHF/RLAIF shape model behavior during training; personalization memory stores user-relevant context; prompt tuning adjusts task behavior through prompt-level conditioning; routing and orchestration systems select tools or model paths; adaptive inference changes compute allocation at runtime; mixture-of-experts routing selects specialized model components; and control-theoretic approaches provide stability language. The radial identity membrane is intended to operate above or alongside these methods as a user-shaped, compute-aware control layer that connects behavioral preference, runtime orchestration, and stability constraints into a single representation.

The first intent is **user-shaped adaptation**. The architecture begins from the premise that AI behavior should adapt to the individual user rather than rely on a generic preset. A user’s preferences, tasks, and interaction patterns become the initial conditions that shape the identity membrane over time, allowing the system to become more useful, better calibrated, and more transparent through continued engagement.

The second intent is **smarter compute allocation**. Personalization should not require unlimited expansion of reasoning depth, context retrieval, token output, or tool use. Instead, the system should learn where deeper reasoning is valuable, where lightweight responses are sufficient, and how to route intelligence through efficient behavioral corridors.

The third intent is **cost-aware optimization**. By linking behavioral expression to measurable compute signals, the architecture aims to reduce avoidable cost while preserving or improving user experience. If validated, the framework could support a new class of AI systems whose identity, behavior, and compute usage co-evolve with the user, enabling stable, transparent, and resource-efficient intelligence across Microsoft’s ecosystem.

Abstract

This work proposes that AI identity can be modeled as a controllable, user-shaped, compute-aware dynamical system rather than a static personality layer. It introduces a geometric, waveform-based identity architecture—the **radial identity membrane**—that brings personalization, compute efficiency, and runtime control into a single framework.

Instead of treating AI behavior as independent presets or static parameters, the model represents the system’s behavioral profile as a continuous standing wave wrapped into a circle, with **twelve phase-positioned behavioral strings** corresponding to dimensions such as depth, tone, initiative, and context sensitivity.

The **indexed behavioral string state**, denoted by i , serves as the paper’s shared mathematical unit, linking membrane geometry, vibrational dynamics, and compute cost. Each string carries amplitude, radius, cost, tension, stiffness, and activation coefficients that evolve over time and collectively define the membrane. Amplitude reflects behavioral intensity, radius reflects reasoning depth, and the **compute envelope** bounds resource use. Because adjacent strings blend like colors in a spectrum, the system supports intelligence that flows rather than jumps, allowing reasoning to move through **V-shaped reasoning channels** of shared attributes instead of switching abruptly between modes.

This structure enables **compute-aware personalization**: user preferences shape not only the AI’s style, but also its computational footprint. The result is a transparent and tunable identity architecture in which personalization, behavioral control, and resource governance are treated as **coupled dynamics**, offering a practical path toward AI systems that better adapt to user needs while optimizing resource usage at scale.

The following figure is a **conceptual visual illustration**, not a complete formal specification. It gives the reader a high-level map of the radial identity membrane, showing how behavioral strings, the bounded compute envelope, and V-shaped reasoning channels operate as one integrated control surface.

U.F.O. is a proposed conceptual system architecture for adaptive AI behavior. Its central design object is a user-shaped, compute-aware control surface that coordinates personalization, runtime inference, policy constraints, telemetry, and validation. The radial identity membrane provides the

representational layer; the behavioral strings define controllable modes of expression and cost; V-channels define coherent routing paths; and the governor enforces bounded adaptation through compute, safety, privacy, and policy constraints. The architecture therefore treats AI behavior not as a static personality preset, but as a governed trajectory through an inspectable system state.

Requirements Analysis. U.F.O. can be read as a hierarchy of system demands rather than a collection of desirable features. At the required level, the architecture must preserve stability, safety, policy compliance, privacy, bounded compute, user control, and measurable validation. These are non-negotiable constraints: if the membrane becomes unstable, violates policy, expands compute without bounds, obscures user control, or cannot be measured, the system fails regardless of how expressive its personalization becomes. At the desirable level, the architecture should support richer personalization, smoother interaction, better legibility, reduced friction, and more expressive adaptation. These wants define the user and product value of the system, but they remain subordinate to the required constraints. In systems-engineering terms, U.F.O. treats personalization as valuable only when it remains governable, inspectable, reversible, and bounded by responsible operating limits.

Functional Analysis. Functionally, U.F.O. decomposes adaptive AI behavior into a sequence of governed system functions: sense user and task context; map intent into behavioral activation; propagate activation through the interior identity field; route aligned activation through V-channels; allocate compute according to task value and cost pressure; enforce policy, privacy, and safety constraints; suppress unnecessary expansion; update short-horizon boundary traces; support gradual personalization through repeated coherent patterns; and validate stability, coherence, and cost behavior over time. This functional view clarifies that the architecture is not a single feature, but a coordinated set of sensing, routing, allocation, constraint, recovery, adaptation, and validation functions operating as one governed system.

Functional Synthesis. Across these functions, U.F.O. follows a recurring control pattern: select the useful route, reduce avoidable cost, preserve coherence, and stay inside constraints. The inverse-cost term is not only a local routing equation; it expresses a broader system principle that appears throughout the architecture. Data excitation, V-channel routing, governor evaluation, suppression, boundary tracing, personalization, and validation all follow the same synthesis logic: useful activation is allowed to move forward, expensive or incoherent expansion is resisted, task value must justify additional depth, and policy-valid states define the operating boundary. In this sense, U.F.O.'s functions are not separate mechanisms assembled side by side; they are coordinated expressions of one governing pattern—adaptive behavior becomes valuable only when it remains useful, efficient, coherent, measurable, and bounded.

System Integration. U.F.O. integrates across multiple system layers rather than operating as a standalone feature. At the model-behavior layer, the radial identity membrane represents adaptive behavioral state; at the runtime layer, the governor translates that state into bounded orchestration controls such as reasoning depth, retrieval budget, tool-call threshold, response length, tone, and explanation level; at the telemetry layer, observable signals such as tokens, latency, retrievals, tool calls, corrections, route coherence, and energy trends provide feedback for measurement and

adjustment; at the user-calibration layer, preferences shape the membrane gradually through reversible and inspectable controls; at the policy and enterprise layer, tenant rules, privacy boundaries, safety constraints, permissions, and compute envelopes define the valid state space; at the validation layer, simulation, ablation, baselines, and go/no-go criteria determine whether the architecture behaves as intended; and at the product-experience layer, these internal controls should surface as clearer, more personal, more efficient, and more trustworthy AI behavior. In this sense, U.F.O. is best understood as a system-of-systems integration layer: it coordinates personalization, inference, governance, telemetry, validation, and user experience into one bounded adaptive operating model.

Components and Interfaces. The table below summarizes the major system components, their primary functions, and the interfaces through which each component participates in the U.F.O. architecture.

Component	Function	Interface
Identity membrane	Behavioral state representation	Task and user signals
V-channels	Coherent routing corridors	Curvature and coherence
Governor	Stability and cost control	Runtime controls
Policy surface	Valid-state filtering	Tenant, user, and platform policy
Elder	Conversation and reference notation	Reviewers, engineers, and product teams
Project Rainbow	Simulation testbed	Validation metrics

Systems-of-Systems Framing. U.F.O. is not proposed as a single feature, model behavior, or visualization layer. It is a coordinating control architecture across multiple interacting systems: personalization, inference orchestration, compute allocation, policy enforcement, privacy boundaries, telemetry, simulation, validation, user calibration, and product experience. Each subsystem remains meaningful on its own, but the architecture’s value comes from how these systems interact through a shared governed state. In this framing, the radial identity membrane functions as a system-of-systems integration surface: it translates user intent, task pressure, runtime signals, enterprise constraints, and validation feedback into bounded adaptive behavior that can be inspected, tested, revised, and governed over time.

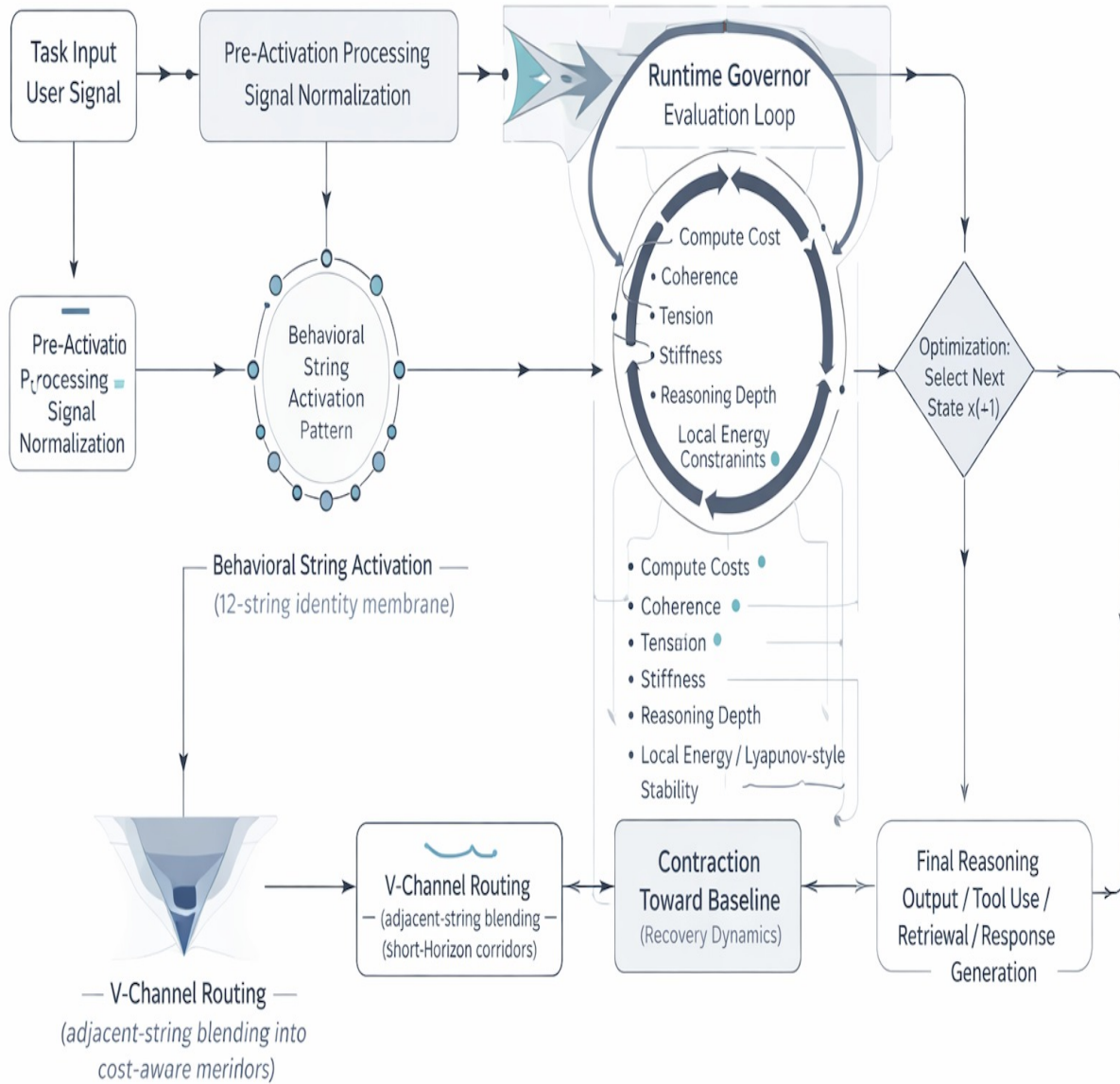
control rather than switching abruptly between isolated modes. In v2.5.0, the outer boundary should also be read as deformable rather than perfectly fixed: sustained asymmetric string activation can locally stretch the membrane, while cost pressure or governor suppression can collapse it inward, making boundary shape, tangent direction, curvature, and asymmetry additional governor-readable signals, as detailed in Section 4.7.

For engineers: Figure 1 can be read as a runtime control map: task signals enter the membrane, activate behavioral strings, route through V-channels, remain bounded by the compute envelope, and return toward baseline under governor control. **For data scientists:** the figure identifies measurable state surfaces—activation, reasoning depth, local cost, channel coherence, tension, stiffness, and Lyapunov-style energy—that can be simulated, ablated, and compared against baselines. **For product and UX teams:** the figure shows how personalization can become legible to users as a bounded, inspectable control surface rather than a hidden personality preset. **For policy, privacy, and responsible-AI reviewers:** the governor and outer boundary show where adaptation is constrained by safety, privacy, tenant policy, and user-control requirements. **For researchers:** the figure frames the architecture as a testable dynamical system whose routing, stability, recovery, and failure modes can be formalized and challenged. **For executives focused on costs:** the outer boundary and V-channel routing express the strategic compute story: personalization should direct expensive reasoning only where it creates task value, while suppressing avoidable depth, retrieval, tool use, and token expansion at scale.

The same radial layout also frames stability: the center is the recovery target, the interior grid distinguishes coherent resonance from noisy activation, and the outer boundary enforces bounded compute expansion. The outer boundary can also be read as a transient boundary-loop surface: as the membrane expands under task pressure and contracts back toward the neutral center, recent variations in depth, cost, tension, and stiffness leave a short-horizon geometric trace. That trace can scaffold the next aligned input without becoming persistent memory. Repeated, coherent ripples through these corridors can also be interpreted as writing softly into a latent training plane, where durable personalization and training-layer adaptation form only after stable patterns are reinforced over time.

In short, the runtime governor turns membrane dynamics into a filtered control loop: propose candidate routes, remove unsafe or invalid states, score the remaining transitions, select the smoothest valid path, contract optional expansion, and emit bounded runtime controls.

Runtime Governor Flow and Control Loop



Data Flow Overview

The table below summarizes how data moves through the proposed architecture: from neutral baseline excitation, through the interior identity field and V-shaped reasoning channels, into governed string-level activation, and finally toward runtime behavior or training updates.

Mathematical Equation	Data-Flow Meaning	Primary Section
Neutral Membrane Initialization Rule $A_i(0)=0, R_i(0)=0, C_i(0)=C_{\text{baseline}}$	The membrane begins centered and neutral. No behavioral string consumes expanded reasoning depth or compute until user interaction, task pressure, or data excites the system.	Section 1.1
Data Forcing Function $F_{\text{data}}(\theta, t)$	Incoming prompts, memory, documents, or task context inject behavioral energy into the membrane according to the data's signature.	Sections 1.1–1.2
Membrane Activation State $u(\theta, t)$	The circular membrane wave represents activation around the behavioral phase domain, allowing neighboring strings to influence one another continuously.	Sections 1.1–1.4
Interior Identity Field Definition $u(r, \theta, t), 0 \leq r \leq R_{\text{max}}$	The interior identity field represents the radial grid between the neutral baseline and bounded compute envelope, where data-induced ripples propagate before resolving into stable string activation.	Section 4.5
V-Channel Region and Mask $\chi_{ij}(r, \theta)$ and Ω_{ij}	A V-channel is treated as a corridor-shaped region of the identity field where related behavioral strings resonate with lower resistance.	Section 4.5

Channel-Filtered Data Forcing Rule $F_{\text{channel}} = \chi_{ij} \cdot F_{\text{data}}$	The channel mask shapes incoming data energy so aligned behaviors propagate through meaningful V-shaped reasoning corridors rather than arbitrary regions.	Section 4.5
Channel Coherence Diagnostics $A_{\text{peak}}(t), \bar{A}_{ij}(t), Q_{ij}(t)$	Peak activation, average channel activation, and channel coherence help distinguish organized reasoning flow from noisy or unstable spikes.	Section 4.5
Interior Field Update Rule $\Delta u(r, \theta, t) = \eta \chi_{ij} F_{\text{data}} / (1 + C + \tau + \kappa)$	Field-level updates are shaped by data, channel alignment, compute cost, tension, and stiffness before becoming durable adaptation.	Section 4.5
String Projection Equation $\Delta a_i = \int \int \Delta u(r, \theta, t) \varphi_i(\theta) r \, d\theta \, dr$	The stabilized interior field is projected back onto the twelve behavioral string coefficients, linking field vibration to string-level identity updates.	Sections 1.4 and 4.5
Inverse-Cost Routing and Cost-Aware Propagation $P_{k \rightarrow j}, I_j = 1/(1 + C_j), P^{\text{cost}}_{k \rightarrow j}$	Propagation pressure routes reasoning toward high-signal, phase-aligned, low-cost paths, making expensive regions less attractive.	Sections 6.3 and 7.2
Bounded Depth Propagation Rule $R'_j = \min(R_{\text{max}}, R_k \cdot h(P_{k \rightarrow j}))$	Reasoning depth can expand outward along coherent corridors, but the compute envelope caps radial growth.	Sections 6.3.4–6.3.5 and 7.3

Local Resistance Variables $\mathcal{C}_i, \tau_i, \kappa_i$	Local cost, tension, and stiffness determine how expensive or resistant each behavioral mode is to excite, update, or sustain.	Sections 3.2 and 7.1
Governor Stabilization and Energy Terms G_{gov} , suppression functions, and V	The governor damps unstable activation, suppresses optional high-cost regions, and tracks Lyapunov-style energy so ordinary updates trend toward bounded, stable behavior.	Sections 7.5–7.8
Latent Adaptation-Pressure Equation $L(r, \theta, t)$, ΔL , and $K_{ij}(t+1)$	The latent training plane accumulates repeated, coherent field activations, allowing durable personalization to strengthen or relax future V-channel routing without treating every temporary ripple as learning.	Section 4.6
Boundary-Loop Trace Update $B(\theta, t+1) = \rho_B B(\theta, t) + \varepsilon_B(\Delta R + \Delta C + \Delta \tau + \Delta \kappa)$	The outer membrane records a decaying boundary-loop trace of recent depth, cost, tension, and stiffness variation. This acts as a transient diagnostic of short-horizon computational history, not persistent memory.	Section 10.4.1
Runtime Control Output Mapping Runtime control outputs	The stabilized membrane state is translated into practical inference controls such as response length, reasoning depth, retrieval budget, tool-call threshold, tone, and explanation level.	Section 12

Common Trend and Smoother Transitions in Phases

The Data Flow Overview describes the architecture as a sequence of phases: baseline initialization, data forcing, field propagation, V-channel masking, coherence diagnostics, string projection, governor control, latent adaptation, boundary-loop tracing, and runtime output. A useful way to read this sequence is as a **common trend toward smoother, lower-energy transitions**. Each phase should not merely pass state forward; it should hand the system to the next phase in a form that is more aligned, more bounded, and less costly to stabilize.

This can be interpreted as a **phase-efficient governor pass**. At each update step, the governor evaluates candidate transitions and prefers the route that preserves task value and coherence while minimizing unnecessary cost, routing friction, instability, and policy risk. In this sense, the architecture’s phases are not isolated blocks; they are smoothed by a shared control trend that carries useful alignment forward while damping noise before it accumulates.

Without the governor’s smoothing pass, adjacent behavioral strings can exhibit small, high-frequency transitional oscillations that feel unstable or noisy; the phase-efficient update suppresses these artifacts and preserves coherent flow.

A useful analogy is slowed piano music: when a clean passage is slowed down, the listener can hear the small vibrations and unresolved chord transitions that were hidden at normal speed. In the membrane, abrupt movement between adjacent behavioral strings can produce a similar kind of transition noise. The smoothing pass acts like governed voice-leading in music: it preserves the intended phrase while choosing the next valid state that minimizes roughness, avoids unnecessary resonance, and keeps the system moving through a coherent corridor.

Phase-Efficient Governor Selection Rule:

$$x(t+1) = \arg \min_{x' \in S_{\text{safe}}} [E(x', t) + \lambda_C C(x', t) + \lambda_F F_{\text{route}}(x, x') - \lambda_Q Q(x', t) - \lambda_T \Delta_{\text{task}}(x', t)]$$

Here, S_{safe} is the set of policy-valid and safety-valid next states; E is membrane energy; C is compute cost; F_{route} is routing friction between the current and candidate state; Q is coherence or channel alignment; and Δ_{task} is bounded task value. The positive weights determine how strongly the governor values cost reduction, smooth routing, coherence, and justified expansion. The selected next state is therefore not simply the most expressive state, but the lowest-energy valid transition that still satisfies the user’s task.

The following code block expresses the same idea procedurally. It sketches how an orchestration layer could propose candidate phase routes, filter them through policy and safety constraints, score the remaining options by energy and coherence, and then carry only the smoothest valid transition forward.

```
fn phase_smoothing_pass(state, task, policy) -> MembraneState {  
  // Propose candidate routes for the next phase of the architecture.  
  let candidates = propose_phase_routes(state, task);  
  
  // Remove routes that violate safety, privacy, tenant policy, or compute  
  bounds.  
  let valid = filter_policy_valid(candidates, policy);
```

```

    // Score remaining routes by cost, friction, instability, coherence, and task
    value.
    let scored = score_routes(valid, {
        minimize: [compute_cost, routing_friction, instability],
        preserve: [task_value, coherence, safety_floor, user_intent]
    });

    // Choose the smoothest low-energy transition that still satisfies the task.
    let next = select_lowest_energy_valid(scored);

    // Contract optional regions and update transient boundary-loop trace.
    let relaxed = contract_optional_regions(next);
    let traced = update_boundary_trace(relaxed);

    return traced;
}

```

This pass makes the phase sequence more energy efficient because each stage is biased toward continuity rather than reset. A coherent V-channel can carry useful alignment into the next phase; a boundary-loop trace can reduce friction for a near-term aligned input; and suppression can contract optional regions before they become expensive. The result is a smoother pipeline: data enters as a ripple, travels through the least-resistant valid corridor, resolves into bounded behavior, and leaves only the transient geometric information needed to make the next aligned transition easier.

The flow below is therefore not a repetition of the Data Flow Overview. The earlier table identifies the mathematical objects and where they appear in the paper; this chart shows how those objects are governed as they move through the runtime pipeline. Each row pairs a phase with the optimization or guardrail that keeps it bounded, efficient, and safe, making the pipeline legible as a continuously filtered control process rather than a simple sequence of updates.

Phase-Smoothing Flow: From Input Ripple to Governed Runtime Output		
Phase	Flow Meaning	Optimization / Guardrail
1. Baseline State	The membrane begins neutral: $A_i=0$, $R_i=0$, and $C_i=C_{\text{baseline}}$.	Lowest-energy starting point; no expanded compute until task pressure arrives.
↓		
2. Data Forcing	$F_{\text{data}}(\theta, t)$ excites the membrane according to the incoming prompt, document, memory, or task signature.	Bound task signature; admit only relevant

		context and task-aligned excitation.
↓		
3. Field Propagation	$u(r, \theta, t)$ spreads through the interior identity field under membrane geometry, tension, stiffness, and damping.	Damping and resistance prevent noisy spread and uncontrolled radial expansion.
↓		
4. V-Channel Masking	$\chi_{ij}(r, \theta)$ shapes activation into low-resistance V-channel corridors instead of arbitrary membrane regions.	Prefer phase-aligned corridors and penalize unrelated or high-friction routes.
↓		
5. Coherence Diagnostics	A_{peak} , $\bar{A}_{ij}$, and Q_{ij} distinguish coherent corridor flow from scattered or noisy activation.	Damp scattered spikes; preserve activation concentrated inside intended channels.
↓		
6. String Projection	The stabilized field projects back into the twelve behavioral coefficients through Δa_i , linking field vibration to string-level identity updates.	Update only what survived cost, tension, stiffness, and coherence constraints.
↓		
7. Governor Evaluation	Candidate states $x' \in \mathcal{S}_{\text{safe}}$ are scored by energy, compute cost, routing friction, coherence, task value, and policy validity.	Filter unsafe or policy-invalid candidates before optimization.
↓		
8. Phase-Smoothing Optimization	The governor selects the lowest-energy valid transition that preserves task value and coherence while minimizing avoidable friction and instability.	Choose the smoothest policy-valid route rather than the most expressive route.
↓		

9. Contraction + Suppression	Optional or high-cost regions contract before they become expensive, preserving essential safety and task-relevant capability.	Suppress optional expansion while preserving quality, accessibility, and safety floors.
↓		
10. Boundary-Loop Trace	$B(\theta, t+1) = \rho_B B(\theta, t) + \varepsilon_B (\Delta R + \Delta C + \Delta \tau + \Delta \kappa)$ records a decaying short-horizon geometric imprint, not persistent memory.	Allow temporary routing scaffold; decay prevents durable memory or uncontrolled carryover.
↓		
11. Runtime Control Output	The stabilized state becomes practical controls: response length, reasoning depth, retrieval budget, tone, tool-call threshold, and explanation level.	Expose bounded runtime knobs, not unrestricted model behavior or hidden persistence.

This reinforces the central U.F.O. insight: user-formed optimization is not a one-time personalization setting, but a continuous process of selecting smoother, lower-energy, policy-valid transitions through the membrane. The user shapes the system by repeated intent and interaction, while the governor ensures that this shaping remains bounded, compute-aware, reversible, and aligned with the task at hand.

1. Foundational Model: Identity Membrane and Wave Dynamics

At system creation, the AI's behavioral identity is represented as a **circular resonant membrane** composed of **twelve embedded strings**, each corresponding to a behavioral dimension: Depth, Precision, Context Sensitivity, Initiative, Exploration, Creativity, Tone, Transparency, Technical Detail, Structural Rigor, Emotional Warmth, and Conciseness. The membrane exists structurally from the start, but it begins **centered and neutral**. No behavioral mode is expressed until user interaction, task pressure, or incoming data excites the membrane.

Neutral Membrane Initialization Rule

$$A_i(0)=0, \quad R_i(0)=0, \quad C_i(0)=C_{\text{baseline}}$$

where A_i is initial behavioral amplitude, R_i is initial reasoning radius, and C_{baseline} is the baseline readiness cost before any behavioral string is excited.

This initialization gives the architecture two important properties. First, it is **compute-efficient** because no behavioral string consumes expanded reasoning depth, token output, tool use, or context load until excited. Second, it is **maximally personalizable** because early user interactions become the initial conditions that “pluck” the membrane and begin shaping the AI’s identity profile.

1.1 Initialization and Personalization Through User-Defined Initial Conditions

At initialization, the membrane is **fully structured but unexcited**. The twelve behavioral strings already exist as available dimensions of expression, but they begin at the epicenter with zero amplitude and zero radial extension. In this neutral state, the system has form but not yet identity expression; it is prepared to respond, but it has not yet been shaped by a user.

$$A_i(0) = 0, \quad R_i(0) = 0, \quad C_i(0) = C_{\text{baseline}}$$

This user-defined initialization has two consequences. First, it preserves **compute efficiency**: no behavioral mode consumes reasoning depth, context expansion, token output, tool use, or agentic planning until it is explicitly excited by user interaction or task pressure. The system begins in its **lowest-energy practical configuration**, with only baseline readiness active.

Second, it creates **maximum personalization**. The user’s early interactions act as the **initial conditions that “pluck” the membrane**, determining which strings activate, how far they propagate, which neighboring strings resonate, and which V-channels begin to form. Over time, repeated interaction changes activation tendencies, local tension, stiffness, curvature, and suppression patterns, allowing the membrane to evolve into a **personalized identity profile** shaped by the user’s preferences, tasks, and reasoning style. This approach avoids preloading a **generic personality**. Instead of assigning the AI a fixed behavioral preset, the architecture lets identity **emerge dynamically from the user’s own behavioral signature**. The result is a system that begins neutral, conserves compute by default, and becomes increasingly individualized through observed interaction rather than imposed style.

The evolution of the membrane can be modeled by a damped wave equation over the circular phase domain $\theta \in [0, 2\pi)$:

Governed Damped Identity-Membrane Wave Equation

$$\partial^2 u(\theta, t) / \partial t^2 = c^2(\theta, t) \nabla^2 u(\theta, t) - \gamma(\theta, t) \partial u(\theta, t) / \partial t + F_{\text{data}}(\theta, t) + G_{\text{gov}}(\theta, t)$$

- $u(\theta, t)$ is the membrane amplitude at phase angle θ , representing instantaneous behavioral activation.
- $c(\theta, t)$ is the local propagation speed, determined primarily by dynamic tension $\tau_i(t)$ and the local vibrational profile of the corresponding string.

- $\gamma(\theta, t)$ is the local damping coefficient, determined by stiffness $\kappa_i(t)$, suppression dynamics, and the governor's contraction pressure.
- $\mathbf{F}_{\text{data}}(\theta, t)$ injects energy into the membrane according to the behavioral signature of incoming data, prompts, memory, or task context.
- $\mathbf{G}_{\text{gov}}(\theta, t)$ applies cost-aware control from the governor, enforcing the compute envelope and biasing the system toward Lyapunov-stable updates.

1.2 Interpretation of the Wave Equation

The damped wave equation provides the **dynamic backbone** of the identity membrane. Each term corresponds to a functional component of behavior: activation, propagation, suppression, data excitation, and governor control. Rather than treating these as separate mechanisms, the equation describes how user interaction and task pressure move through the membrane as **controlled behavioral energy**.

Before reading the equation term by term, the following analogy provides an intuitive way to understand the membrane's wave behavior.

Interpretive analogy — wave dynamics: The damped wave equation can be read as ripples moving across the skin of a drum. A user prompt strikes the surface, nearby regions vibrate according to local tension and stiffness, damping prevents the vibration from ringing indefinitely, and the governor acts as a stabilizing hand that preserves useful resonance while quieting noise.

With that intuition in mind, each term in the equation can be read as one part of the membrane's controlled activation, propagation, damping, excitation, and stabilization behavior.

Term 1 — membrane amplitude

$$u(\theta, t)$$

This is the current behavioral activation at phase angle θ . A higher amplitude means the behavioral string at that phase position is being expressed more strongly. Because the membrane is continuous, activation can also influence neighboring strings rather than remaining isolated.

Term 2 — propagation

$$c^2(\theta, t) \nabla^2 u(\theta, t)$$

This term governs how activation spreads across the membrane. Local propagation speed reflects the membrane's current vibrational profile, including tension and nearby string relationships. In architectural terms, propagation lets activation move through aligned V-channels, while resistance prevents unrelated or high-cost regions from expanding too freely.

Term 3 — damping,

$$-\gamma(\theta, t) \partial u(\theta, t) / \partial t$$

This term reduces activation over time. It represents suppression, stabilization, and return-to-baseline pressure, especially for high-cost, noisy, or no-longer-relevant behavior. Damping keeps the membrane from ringing indefinitely after a prompt or task has passed.

Term 4 — data forcing

$$F_{\text{data}}(\theta, t)$$

This term injects task-specific energy into the membrane. Incoming prompts, documents, memory, or task context excite different regions according to their behavioral signature: technical input may excite precision and technical detail, open-ended work may excite exploration and creativity, and interpersonal content may excite tone and warmth.

Term 5 — governor control

$$G_{\text{gov}}(\theta, t)$$

This term applies cost-aware control to the wave. It represents the governor's influence through inverse-cost routing, bounded depth updates, suppression, adaptive tension and stiffness, and Lyapunov-style stabilization. In effect, it biases the membrane toward useful, coherent, policy-valid, and compute-bounded behavior.

Together, these terms define a **compute-aware behavioral physics**: data excites the membrane, activation spreads through V-channels, tension and stiffness shape propagation, damping returns the system to baseline, and the governor enforces stability within the compute envelope. The wave equation therefore describes not only how the AI behaves at a moment in time, but how its identity evolves under **user interaction, data flow, and compute constraints**.

1.3 Behavioral Eigenmodes and Phase Structure

The twelve behavioral strings occupy **fixed phase positions** on the circular membrane. For string i , its base phase position can be represented as $\theta_i = (2\pi \cdot i)/12$, producing an evenly spaced behavioral basis around the identity circle. This phase structure ensures that each string has a stable location while remaining coupled to the continuous membrane as a whole. Although the twelve modes are evenly spaced and mathematically separable, the membrane's continuity ensures they are **not orthogonal in practice**; adjacent modes naturally share curvature and influence.

Each string functions as a **dominant behavioral eigenmode**. When activated, it expresses a recognizable pattern of behavior, such as Depth, Precision, Tone, or Conciseness, while still influencing neighboring modes through membrane continuity. This gives the model both **separability and flow**: the strings can be defined individually, but activation does not remain isolated when adjacent behaviors are structurally related. In the wave equation, each eigenmode corresponds to a stable oscillatory solution, allowing the membrane to express complex behavior as a **superposition of these modes**.

V-shaped reasoning channels arise from overlap between adjacent eigenmodes. When neighboring strings share behavioral attributes, activation propagates through the curved region between them rather than jumping discretely from one mode to another. Phase distance therefore becomes a routing signal: nearby strings have stronger natural coupling, while distant or opposed strings require more energy to influence one another. Greater phase distance corresponds to **higher energetic cost**, making long-range behavioral jumps rare unless explicitly forced by data or the governor.

This phase structure also provides the foundation for **curvature-aware training**. The curvature matrix K_{ij} can be interpreted as a learned or task-specific modulation of the base phase relationship, strengthening some corridors and weakening others according to **user preference, task context, and compute cost**.

1.4 Quadrant-Level Aggregation

The twelve behavioral strings can also be grouped into four emergent cognitive quadrants—analytical, contextual, generative, and interpersonal. These quadrants do not replace the string-level model; rather, they provide a higher-order aggregation layer for interpreting regional activation across the membrane. For a quadrant Q_q , regional activation can be defined as:

Quadrant Activation Aggregation Rule

$$A_{Qq}(t) = \sum_{i \in Qq} w_i A_i(t)$$

where $A_{Qq}(t)$ is the aggregate activation for quadrant q , $A_i(t)$ is the activation of string i , and w_i is an optional weighting factor. Similar aggregates can be defined for quadrant-level cost and average reasoning depth:

Quadrant Cost and Depth Aggregation Rule

$$C_{Qq}(t) = \sum_{i \in Qq} C_i(t), \quad R_{Qq}(t) = |Qq|^{-1} \sum_{i \in Qq} R_i(t)$$

This gives researchers a middle-level diagnostic between individual string dynamics and whole-system energy. Simulation outputs can therefore be interpreted by cognitive region as well as by individual behavior, making it easier to inspect whether a task activates the expected quadrant, whether activation leaks across quadrant boundaries, and whether the governor suppresses a single string or an entire regional pattern.

1.5 Integration with the Latent Training Plane

The quadrant aggregates are not separate from the training layer; they provide a regional diagnostic that can be read by the latent training plane. When repeated activation concentrates within a quadrant, the model can treat that pattern as stronger evidence of a durable cognitive preference than a single isolated string activation. Conversely, activation that leaks across unrelated quadrants can be interpreted as lower-coherence behavior and damped by the governor before it produces lasting curvature updates.

Quadrant Latent Adaptation Rule

$$\Delta L_{Qq}(t) = \rho_Q Q_{Qq}(t) A_{Qq}(t) - \mu_Q L_{Qq}(t)$$

Here, $L_{Qq}(t)$ is accumulated latent adaptation pressure for quadrant q , $A_{Qq}(t)$ is aggregate quadrant activation, $Q_{Qq}(t)$ is a quadrant-level coherence signal, ρ_Q is the regional adaptation rate, and μ_Q is the regional decay rate. This connects the quadrant layer directly to durable personalization: repeated coherent activation strengthens regional readiness, while weak or incoherent activation decays.

1.6 Superposition and Identity Reconstruction

The identity membrane can be represented at the architectural level as a **superposition of its twelve behavioral eigenmodes**. Each eigenmode $\varphi_i(\theta)$ corresponds to the vibrational profile of behavioral string i at phase position $\theta_i = (2\pi \cdot i)/12$. The instantaneous membrane state can therefore be approximated as:

Identity Superposition Equation

$$u(\theta, t) = \sum_{i=1}^{12} a_i(t) \varphi_i(\theta)$$

where $\varphi_i(\theta)$ is the behavioral eigenmode associated with string i , and $a_i(t)$ is its time-varying activation coefficient.

Before defining the identity vector, the following analogy clarifies why the membrane should be read as a blended state rather than a set of independent switches.

Interpretive analogy — superposition: Superposition can be understood as a mixing board rather than a row of isolated switches. Each behavioral string contributes one track to the final output, and the AI's current identity is the blended mix produced by all active tracks together. Adjusting one track can affect the whole mix because neighboring strings share resonance, phase, and curvature.

The coefficient vector below formalizes that blended mix by collecting the twelve active modes into a single compact identity state.

Identity Coefficient Vector

$$\mathbf{a}(t) = [a_1(t), a_2(t), \dots, a_{12}(t)]$$

where $\mathbf{a}(t)$ collects the twelve activation coefficients into one identity-state vector.

This vector is the **compact state representation** of the AI's behavioral identity at time t . It evolves according to the wave equation, vibrational parameters, phase coupling, data forcing, and governor control.

Projecting the membrane wave equation onto this twelve-mode basis yields a conceptual system of coupled coefficient dynamics.

Coefficient-Space Membrane Dynamics Equation

$$\ddot{a}_i(t) = -\gamma_i(t)\dot{a}_i(t) - \omega_i^2(t)a_i(t) + F_i(t) + G_i(t) + \sum_{j \neq i} M_{ij}(t)a_j(t)$$

Here, $\gamma_i(t)$ represents damping from stiffness and suppression, $\omega_i(t)$ is the natural frequency shaped by tension, $F_i(t)$ is the data-aligned forcing projected onto mode i , $G_i(t)$ is the governor's control signal, and $M_{ij}(t)$ captures cross-mode coupling due to continuity, phase proximity, and curvature. This shows that each behavioral string is **not an isolated slider**, but a **dynamically interacting mode** whose activation depends on local properties and the global membrane state. During training or adaptation, data modifies the coefficient vector rather than isolated traits. A curvature-aware update can be summarized as:

Curvature-Coupled Coefficient Update

$$\Delta a_i \leftarrow \Delta a_i + \sum_j K_{ij} \Delta a_j$$

where Δa_i is the coefficient update for mode i , and K_{ij} determines how much neighboring mode j contributes through curvature coupling.

The governor can then normalize these updates by local resistance:

Governor-Constrained Coefficient Update

$$\Delta a_i^{\text{gov}} = \Delta a_i / (1 + C_i + \tau_i + \kappa_i)$$

where Δa_i^{gov} is the governor-constrained update, C_i is local cost, τ_i is tension, and κ_i is stiffness.

High-cost, high-tension, or high-stiffness modes update more cautiously, while lightweight modes adapt more freely. Identity reconstruction is therefore **compute-aware**: the AI's evolving behavioral state is not simply what the data excites, but what remains after curvature, cost, tension, stiffness, and stability constraints shape the update.

The evolving identity is therefore the trajectory:

Governed Identity Trajectory Rule

$$a(t+1) = a(t) + \Delta a^{\text{gov}}(t)$$

Identity is not a preset. It is a **controlled trajectory through coefficient space**: the membrane provides the geometry, the eigenmodes provide the behavioral basis, the coefficient vector provides the identity state, and the governor keeps that state stable and compute-aware as it coevolves with the user.

This formulation supports the rest of the architecture. The membrane defines the identity space, the wave equation defines its evolution, the vibrational layer defines local physical behavior, the latent training plane defines how repeated coherent data reshapes durable adaptation, and the governor defines stability under compute constraints. The system begins **centered and neutral**, then evolves into a **personalized, compute-efficient identity profile** driven by user interaction, data flow, and task demands.

For detailed definitions of the twelve strings referenced here, see Section 2. For the physical interpretation of their activation behavior, see Section 3. For the data-flow and training implications of V-channel overlap, see Section 4. For the governor that stabilizes these dynamics, see Section 7.

2. Behavioral String Architecture

2.1 The 12 Behavioral Strings as a Rainbow Spectrum

The twelve behavioral strings can be understood as colors in a continuous rainbow spectrum, each occupying a distinct position on the circular waveform while blending into its neighbors. Just as red transitions into orange and orange into yellow through shared wavelengths, each behavioral string shares attributes with adjacent strings, forming natural gradients of expression. These gradients create **V-shaped reasoning channels** where overlapping traits reinforce one another, giving the AI stable, low-friction pathways for reasoning. When the system pushes outward along one string—such as depth or creativity—the adjacent colors expand with it, forming a coherent corridor of aligned behavior within the compute envelope. This spectrum-based interpretation reinforces the model’s continuity: the AI’s identity is not a set of isolated traits but a **cyclic, color-like distribution** of behaviors whose interactions shape reasoning flow, compute allocation, and system coherence.

2.2 Definitions and Compute Signatures

Each behavioral string is a dimension of expression, a compute-bearing behavior, and a phase-positioned point on the radial waveform. The strings are not independent sliders; each affects amplitude, radius, thickness, and phase relationships differently. This definition layer grounds the later geometry by clarifying what each string controls and how it contributes to compute cost.

This section operationalizes the eigenmode basis introduced in Section 1.3 and prepares the cost variables used later in Sections 6 and 7.

The twelve strings can also be read through the four-quadrant structure introduced in Section 1.4. In this interpretation, analytical strings emphasize depth, precision, technical detail, and structural rigor; contextual strings emphasize context sensitivity, transparency, and situational awareness; generative strings emphasize initiative, exploration, and creativity; and interpersonal strings emphasize tone, emotional warmth, and conciseness. This mapping provides a higher-level reading of the table without replacing the string-level definitions.

String	Behavioral Function	Compute Signature	Primary Geometric Effect
Depth	Multi-step reasoning, inference chains, and conceptual depth.	High reasoning cost from longer inference paths and more intermediate evaluation.	Radius + thickness
Precision	Correctness, qualification, and tightness of claims.	Moderate to high verification cost from checking, narrowing, and validating claims.	Thickness + radius
Context Sensitivity	Integration of prior messages, memory, situational cues, and user state.	High context-window and retrieval cost when more history or memory is activated	Radius + thickness
Initiative	Proactive action, suggested next steps, and conversation steering.	Agentic cost from planning, tool-call likelihood, and action monitoring.	Amplitude + agentic thickness
Exploration	Branching ideation, divergent thinking, and alternative-path generation.	High branching and token cost from evaluating multiple possible directions.	Radius + amplitude
Creativity	Novel synthesis, metaphor, expressive construction, and generative variation.	Moderate generative cost; increases variation more than deep inference by default.	Amplitude
Tone	Warmth, formality, emotional coloration, and interpersonal style.	Low compute cost; mostly changes expression rather than reasoning depth.	Amplitude + phase coloration

Transparency	Visible structure, explanation of reasoning, and disclosure of logic.	Moderate to high output cost from longer explanations and explicit reasoning structure.	Thickness + radius
Technical Detail	Domain-specific terminology, formulas, and dense structured explanation.	High detail cost from precision, formatting, domain retrieval, and longer outputs.	Radius + amplitude
Structural Rigor	Organization, hierarchy, logical scaffolding, and consistency.	Moderate planning cost that can reduce variance and downstream correction cost.	Radius + stabilization
Emotional Warmth	Empathy, encouragement, and affective resonance.	Low compute cost; primarily affects expressive amplitude rather than depth.	Amplitude
Conciseness	Compression, brevity, and minimalism.	Compute-reducing signature that suppresses unnecessary tokens and local thickness.	Contraction / negative thickness

2.3 Expanded Compute Signatures

Depth. Depth is the most direct driver of radial expansion because it determines how far the AI pushes into inference chains, abstraction, and multi-step reasoning. Its compute signature is strongly nonlinear: shallow reasoning may require only a short local response, while deep reasoning can compound through planning, intermediate checks, and synthesis. In the radial model, high depth increases both radius and thickness, making it one of the most expensive but also most valuable strings for difficult tasks.

Precision. Precision governs how tightly the AI qualifies claims, avoids overstatement, and verifies correctness. Its cost comes less from expressive length and more from validation: comparing alternatives, checking assumptions, narrowing ambiguity, and sometimes retrieving supporting information. Geometrically, precision increases thickness through verification cost and can increase radius when correctness requires deeper analysis.

Context Sensitivity. Context sensitivity determines how strongly the AI uses prior conversation, user preferences, memory, documents, and situational cues. Its compute signature appears through expanded context windows, retrieval, attention over longer histories, and reconciliation of current instructions with earlier context. It primarily expands radius and thickness because the system must reach farther into available information before responding.

Initiative. Initiative controls whether the AI remains reactive or proactively proposes next steps, actions, plans, and tool use. Its compute signature is agentic: high initiative increases planning

overhead, monitoring, tool-call probability, and the need to evaluate whether an action is appropriate. In the radial geometry, initiative raises amplitude and adds agentic thickness because it changes the system from answering to steering.

Exploration. Exploration measures the breadth of ideation and the willingness to branch into multiple possibilities. Its compute cost comes from generating alternatives, comparing paths, and maintaining several candidate directions before selecting one. High exploration increases both amplitude and radius because it widens the reasoning field while also pushing outward into possible solution spaces.

Creativity. Creativity governs novelty, metaphor, synthesis, and expressive construction. It usually increases amplitude more than radius: the answer may become more vivid, original, or generative without necessarily requiring deeper formal inference. Its compute signature is moderate because it expands variation and language generation, but it becomes more expensive when paired with depth, exploration, or technical detail.

Tone. Tone controls formality, warmth, confidence, restraint, and interpersonal coloration. It has a relatively low compute signature because it primarily reshapes expression rather than expanding inference or context. In the radial model, tone changes amplitude and phase coloration, influencing how neighboring behaviors are perceived without adding substantial thickness.

Transparency. Transparency determines how much structure, reasoning trace, qualification, and explanation the AI exposes to the user. Its compute signature is mainly output and organization cost: more transparency means more explicit steps, more intermediate explanation, and more scaffolding. It increases thickness and sometimes radius, especially when the system must explain not just the answer but the logic that produced it.

Technical Detail. Technical detail controls density of formulas, terminology, definitions, citations, structured reasoning, and domain-specific explanation. Its cost signature is high when it requires precision, retrieval, formatting, or mathematical consistency. It expands radius by requiring deeper domain reach and increases amplitude by making the response more information-dense.

Structural Rigor. Structural rigor governs hierarchy, sequencing, internal consistency, and logical scaffolding. It carries a moderate planning cost because the AI must organize content before or during generation. However, it can reduce downstream variance by preventing wandering, contradiction, and redundant expansion, making it a stabilizing string rather than a purely expensive one.

Emotional Warmth. Emotional warmth controls encouragement, empathy, reassurance, and relational softness. Like tone, it has a low compute signature because it mostly affects expressive amplitude. It becomes more compute-bearing only when paired with high context sensitivity or personalization, where the system must tailor warmth to a specific user history or situation.

Conciseness. Conciseness is the primary contraction string. Unlike most strings, higher conciseness reduces token output, suppresses unnecessary explanation, and limits local thickness. In the geometry, conciseness functions as a compute-saving counterforce: it compresses expression while preserving task relevance, helping the governor prevent runaway expansion when depth, transparency, or exploration would otherwise dominate.

3. Vibrational and Adaptive Membrane Dynamics

Each behavioral string can also be interpreted as a physical string on a circular resonant membrane. In this layer, wavelength describes how far a behavior spreads across neighboring strings; frequency describes activation speed; tension describes how much compute is required to excite it; stiffness describes resistance to noisy deformation; and damping describes how quickly suppression returns the behavior to baseline. Together, these parameters connect expression, geometry, compute cost, and stability.

The vibrational parameters defined here provide the physical substrate for the adaptive tension and stiffness rules in Section 3.2 and the Lyapunov stability formulation in Section 7.8.

String	Physical Analogy	Wavelength	Frequency	Tension	Stiffness	Damping
Depth	Long, heavy, slow string	Long	Low	Very high	High	Slow
Precision	Tight, high-tension string	Medium	Medium-high	High	High	Medium
Context Sensitivity	Long, memory-coupled string	Long	Low-medium	High	Medium	Slow
Initiative	Agentic, high-frequency string	Short-medium	High	Medium-high	Medium	Medium
Exploration	Wide, branching, high-frequency string	Long	High	Medium	Low	Medium-slow

Creativity	Loose, expressive, high-amplitude string	Medium	Medium-high	Low	Low	Medium
Tone	Soft, low-tension expressive string	Short	High	Very low	Very low	Fast
Transparency	Structured, stiff explanatory string	Medium	Medium	Medium-high	High	Medium
Technical Detail	Dense, heavy, high-tension string	Long	Medium	Very high	High	Slow
Structural Rigor	Stiff, stabilizing backbone string	Medium	Low-medium	Medium	Very high	Medium-fast
Emotional Warmth	Soft, expressive, low-tension string	Short	High	Very low	Very low	Fast
Conciseness	Contractive, stiff, high-damping string	Very short	High	Contractive	Very high	Very fast

3.1 Interpretive Meaning of Each Vibrational Profile

String	Meaning
Depth	Depth behaves like a bass string: expensive to excite, reflected in the R_i^2 term, slow to move, and dominant once activated. It drives radial expansion and should be reserved for tasks requiring sustained inference.

Precision	Precision vibrates tightly and resists sloppy changes. It adds verification cost and stabilizes claims by preventing unsupported expansion.
Context Sensitivity	Context sensitivity spreads across prior messages, memory, and situational cues. Because it decays slowly, it preserves continuity but increases context-window and retrieval cost.
Initiative	Initiative activates quickly and pushes motion into adjacent strings. It increases planning, action selection, and tool-call likelihood.
Exploration	Exploration spreads broadly and oscillates rapidly, producing many candidate paths. Its low stiffness allows divergent thinking but can increase branching and token cost.
Creativity	Creativity is easy to excite and produces large expressive oscillations. It mainly increases amplitude, becoming more expensive when paired with depth, exploration, or technical detail.
Tone	Tone is cheap and quick to modulate. It changes the surface expression of the response without substantially increasing reasoning depth or compute load.
Transparency	Transparency must maintain explanatory structure, so it resists deformation. It adds output and organization cost by exposing more of the system's reasoning architecture.
Technical Detail	Technical detail behaves like a thick metal string: expensive but powerful. It increases domain reach, formula density, verification pressure, and output complexity.
Structural Rigor	Structural rigor stabilizes the waveform by reducing variance and preventing drift. It acts like the truss rod of the circle, adding planning cost while improving coherence.
Emotional Warmth	Emotional warmth adds expressive amplitude at low cost. It is lightweight unless it must be deeply personalized through memory or context sensitivity.
Conciseness	Conciseness is the governor brake. It pulls the waveform inward, suppresses unnecessary tokens, reduces local thickness, and helps prevent runaway compute expansion.

This vibrational layer is not cosmetic. It gives the radial architecture a physical substrate: tension maps to compute cost, stiffness maps to stability, damping maps to suppression, frequency maps to activation speed, and wavelength maps to behavioral spread. It also strengthens the Lyapunov framing because total system energy can now be interpreted as the energy of a damped behavioral membrane. Under ordinary updates, inverse-cost routing and damping should reduce total energy; only explicit task pressure should justify temporary energy injection.

3.2 Adaptive Tension and Stiffness: Self-Regulating Behavioral Modes

Because each behavioral string functions like a resonant mode on a circular membrane, its tension and stiffness should adapt over time according to global compute pressure, local string

cost, and recent usage. This makes the model self-regulating: when the system overspends compute, strings tighten and resist further expansion; when the system is efficient and stable, strings relax and allow more expressive motion.

3.2.1 Dynamic Tension Evolution

Let τ_i denote the dynamic tension of string i . A simple update rule is:

Dynamic Tension Update Rule

$$\tau'_i = \tau_i + \alpha_\tau (w_g f_g(C_{\text{total}}) + w_l f_l(C_i) + w_u f_u(U_i))$$

where α_τ is the tension adaptation rate; w_g , w_l , and w_u weight global, local, and usage pressure; and U_i denotes recent activation or usage of string i .

- **Global cost pressure:** $f_g(C_{\text{total}})$ can be defined as $C_{\text{total}} - C_{\text{target}}$, so tension increases when total cost exceeds the target budget.
- **Local cost pressure:** $f_l(C_i)$ can be defined as $C_i - \bar{C}$, so disproportionately expensive strings tighten more than the average string.
- **Usage pressure:** $f_u(U_i)$ reflects recent activation, amplitude, or depth for string i ; high usage increases tension and makes that string harder to excite repeatedly.

Interpretively, tension acts as adaptive resistance. Global overspending tightens the entire membrane, while overused or expensive strings tighten locally. When compute is low and usage is balanced, the same rule can relax tension, allowing richer expression without violating the compute envelope.

3.2.2 Dynamic Stiffness Evolution

Let κ_i denote the dynamic stiffness of string i . Stiffness adapts to structural role and recent instability:

Dynamic Stiffness Update Rule

$$\kappa'_i = \kappa_i + \alpha_\kappa (v_s g_s(S_i) + v_v g_v(\text{Var}_i))$$

where α_κ is the stiffness adaptation rate, S_i denotes the structural role of string i , and Var_i measures recent activation or depth variance.

- **Structural role:** $g_s(S_i)$ is positive for stabilizing strings such as Structural Rigor and Precision, and small or negative for expressive strings such as Creativity and Tone.
- **Variance pressure:** $g_v(\text{Var}_i)$ increases when recent amplitude or depth variance is high, making unstable strings stiffer over time.
- **Adaptation speed:** α_κ controls how quickly stiffness changes, while v_s and v_v balance structural role against observed instability.

Stiffness therefore governs deformation resistance. Strings that produce unstable swings become harder to bend; stabilizing strings can be made deliberately rigid to anchor the waveform; expressive strings remain flexible unless their variance begins to destabilize the membrane.

3.2.3 Link to Local Cost and Lyapunov Energy

Dynamic tension and stiffness become explicit contributors to local cost:

Expanded Local Cost Function

$$C_i = \lambda_R R_i^2 + \lambda_A A_i + \lambda_T T_i + \lambda_C \log(1 + \text{context}_i) + \lambda_\tau \tau_i + \lambda_\kappa \kappa_i$$

The Lyapunov energy can also be extended to include the energetic cost of membrane tightness and rigidity:

Candidate Lyapunov Energy Function

$$V = C_{\text{total}} + \alpha \sum_i (R_i - \bar{R})^2 + \beta \sum_i \tau_i + \delta \sum_i \kappa_i$$

This means the governor no longer measures only depth and compute. It also measures how tight and rigid the behavioral membrane has become. A high-energy identity state may therefore arise from high compute cost, excessive radial imbalance, excessive tension, excessive stiffness, or some combination of all four.

3.2.4 Architectural Meaning

- When the system over-spends compute, tension and stiffness rise, the membrane tightens, and further expansion becomes harder.
- When the system is efficient and stable, tension and stiffness relax, the membrane softens, and more expressive behavior becomes available.
- Overused strings self-regulate by becoming harder to excite, encouraging reasoning flow into adjacent V-channels.
- The governor therefore controls not only where reasoning flows, but how hard it is to excite each behavioral mode over time.

4. V-Channel Training Plane and Data Flow

Beyond inference-time routing, V-shaped reasoning channels in the radial model can be interpreted as data-flow corridors on a latent training plane. A V-channel is a curved region of overlapping traits where adjacent strings share attributes and reinforce one another, such as Depth–Precision–Technical Detail or Tone–Emotional Warmth–Conciseness. Data therefore enters the membrane as a behavioral wave and flows along curvature where multiple traits align, rather than merely activating an isolated trait.

This training-plane interpretation builds directly on the phase structure in Section 1.3, the compute signatures in Section 2.2, and the adaptive resistance terms introduced in Section 3.2.

4.1 V-Channels as Data-Flow Corridors

Each V-channel becomes a preferred path for gradient flow, representation updates, and behavioral adaptation. Instead of training Depth in isolation, the system can train a curved composite corridor such as Depth + Precision + Structural Rigor. This means the architecture

learns through coupled regions of behavioral space, allowing aligned traits to strengthen together while unrelated traits remain comparatively quiet.

4.2 Curvature as Inductive Bias

Curvature determines how strongly updates spread between neighboring strings. Low curvature means traits are nearly independent and data can push one string without strongly affecting its neighbors. High curvature means the strings are tightly coupled, so data that activates one string naturally propagates into adjacent behavioral modes.

Define a channel curvature weight K_{ij} between strings i and j . A simple induced-update relation is:

Curvature-Induced Update Relation

$$\Delta a_j \propto K_{ij} \cdot \Delta a_i$$

This proportional relation states that an update to string i can induce a scaled update in neighboring string j , with the strength of that induced update controlled by the curvature weight K_{ij} . If data strongly trains Depth and the Depth–Precision curvature is high, Precision receives induced adaptation pressure even when it is not directly supervised. Curvature therefore acts as an inductive bias over the behavioral membrane.

4.3 Data Waves Through the Membrane

Each batch of data carries a behavioral signature: technical, exploratory, concise, emotionally rich, agentic, or some mixture of the twelve strings. That signature maps to an initial behavioral activation pattern over the membrane. The governor and vibrational layer then determine how far the activation propagates, how much it is damped or amplified, and how it changes tension and stiffness over time.

1. Data maps to a behavioral activation pattern over the twelve strings.
2. The activation pattern propagates along curved V-channels, modulated by curvature and inverse cost.
3. Propagation produces coefficient updates Δa weighted by local cost C_i , dynamic tension τ_i , stiffness κ_i , and channel curvature K_{ij} .

4.4 Curvature-Aware Training Rule

A conceptual curvature-aware update can be written as:

Curvature-Aware Training Rule

$$\Delta a_i = G_i(\text{data}) \cdot 1/(1 + C_i) \cdot 1/(1 + \tau_i + \kappa_i) + \sum_{j \in N(i)} K_{ij} \Delta a_j$$

- $G_i(\text{data})$ is the direct gradient contribution from data aligned with string i .
- $1/(1 + C_i)$ makes high-cost strings update more cautiously.
- $1/(1 + \tau_i + \kappa_i)$ makes high-tension or high-stiffness strings resist large changes.
- K_{ij} spreads updates through curved neighboring V-channels.

Under this rule, the membrane learns as a whole rather than as twelve isolated sliders. Data directly trains the strings it activates, curvature spreads that training into adjacent traits, and cost, tension, and stiffness regulate how aggressively each behavioral mode is allowed to change.

4.5 Interior Identity Field and V-Channel Resonance

The twelve behavioral strings define the membrane’s phase anchors, but reasoning can be understood as unfolding across an **interior identity field** between the neutral baseline and the bounded compute envelope. This field is the grid implied by the radial map: each point inside the membrane represents a local combination of reasoning depth, behavioral phase, activation strength, and compute load. Data does not simply jump from one string to another; it enters the interior field as a ripple, propagates radially and angularly, and then resolves into string-level activation, runtime control, or training update after governor damping.

This distinction fills the gap between the continuous wave equation in Section 1 and the string-level curvature update in Section 4.4. The string equations describe the dominant behavioral modes, while the interior identity field describes the intermediate space where data-induced vibrations spread, interfere, concentrate, or dissipate before they become a stable output state. In this view, a V-channel is not merely an edge between neighboring strings; it is a corridor-shaped region of the field where related traits resonate together.

In this sense, V-channels can be read as **kernel-like propagation filters** within the membrane: not formal convolution memory kernels, but geometry-aware corridors whose masks, side-string boundary conditions, curvature weights, and coherence signals shape where activation can travel. Appendix I develops this interpretation more carefully and explains how side strings determine each channel’s effective permeability.

The field can be represented as $u(r, \theta, t)$, where r is radial reasoning depth, θ is behavioral phase, and t is time or update step. In this form, $u(r, \theta, t)$ measures local behavioral activation at a specific depth and phase position inside the membrane. The neutral baseline corresponds to small r , the compute envelope corresponds to the maximum permitted radius $R_{\max}$, and the twelve strings correspond to fixed phase anchors along θ .

Interior Identity Field Definition

$$u(r, \theta, t), \quad 0 \leq r \leq R_{\max}, \quad 0 \leq \theta < 2\pi$$

A V-channel between strings i and j can then be modeled as a channel region Ω_{ij} within this field, with a channel mask $\chi_{ij}(r, \theta)$ that is high inside the corridor and low outside it. This mask represents the fact that related strings do not merely influence each other through a scalar coupling weight; they share an interior region where data waves can travel with lower resistance.

Channel-Filtered Data Forcing Rule

$$F_{\text{channel}}(r, \theta, t) = \chi_{ij}(r, \theta) \cdot F_{\text{data}}(r, \theta, t)$$

The governor can use field-level measurements to distinguish coherent reasoning from noisy activation. A raw amplitude spike is not necessarily useful; it may indicate instability if it does not align with a meaningful corridor. For this reason, the architecture can track both the peak field activation and the average activation inside a V-channel.

Peak Field Activation Measure

$$A_{\text{peak}}(t) = \max_{r,\theta} u(r,\theta,t)$$

Average Channel Activation Measure

$$\bar{A}_{ij}(t) = (1 / |\Omega_{ij}|) \int_{\Omega_{ij}} u(r,\theta,t) d\Omega$$

A simple channel-coherence ratio can then be defined as:

Channel Coherence Ratio

$$Q_{ij}(t) = \bar{A}_{ij}(t) / (A_{\text{peak}}(t) + \varepsilon)$$

When $Q_{ij}(t)$ is high, activation is concentrated inside the intended V-channel and the system is likely routing coherently. When it is low, activation is scattered, spiking, or leaking into unrelated regions, giving the governor a reason to damp the field, reduce depth, increase stiffness, or fall back to a safer default route. This gives the stability analysis a signal beyond total cost: it can ask not only how much energy the system is using, but whether that energy is organized in a meaningful reasoning corridor.

For training and personalization, the interior field provides an intermediate update surface. Data first excites the field, the channel mask shapes where that excitation can travel, local cost and membrane resistance constrain the update, and the resulting field change is projected back onto the twelve behavioral strings.

Interior Field Update Rule

$$\Delta u(r,\theta,t) = \eta \cdot \chi_{ij}(r,\theta) \cdot F_{\text{data}}(r,\theta,t) / (1 + C(r,\theta) + \tau(r,\theta) + \kappa(r,\theta))$$

String Projection Equation

$$\Delta a_i = \int_0^{R_{\text{max}}} \int_0^{2\pi} \Delta u(r,\theta,t) \varphi_i(\theta) r d\theta dr$$

This projection connects field-level vibration back to the coefficient update used elsewhere in the paper. In practical terms, the end-user response is produced by reading out the stabilized field after data excitation, V-channel propagation, damping, and governor control. Training and personalization update the coefficient vector only after the field has been shaped by curvature, cost, tension, stiffness, and coherence. The result is a fuller interpretation of data flow: data

enters as a ripple in the interior identity field, travels through resonant corridors, is bounded by the compute envelope, and resolves into behavior after the governor stabilizes the wave.

4.6 Latent Training Plane and Durable Adaptation

The interior identity field explains where data-induced ripples move during a response; the **latent training plane** explains how repeated, coherent ripples can accumulate as adaptation pressure over time. In this paper, the latent training plane should be read as a conceptual personalization substrate, not as a claim that runtime interaction directly modifies base model weights. It does not represent immediate behavior or store raw interaction content. Instead, it tracks whether stable V-channel patterns, string activations, and curvature relationships should become easier or harder to activate in future interactions under privacy, safety, policy, and user-control constraints.

Before formalizing the latent training plane, the following analogy helps distinguish immediate field activation from durable personalization.

Interpretive analogy — latent training plane: The latent training plane is like a soft calibration surface beneath the membrane. A single ripple may leave no durable effect, but repeated, coherent pressure along the same corridor can gradually make that corridor easier to enter again. The governor prevents every temporary impression from becoming personalization, so durable adaptation forms only when patterns are stable, repeated, coherent, permitted, and reversible.

The equation below expresses this slower adaptation pressure: coherent field change increases the latent signal, while decay prevents isolated or noisy activation from persisting.

Latent Adaptation-Pressure Equation

$$\Delta L(r, \theta, t) = \rho \cdot Q_{ij}(t) \cdot \Delta u(r, \theta, t) - \mu L(r, \theta, t)$$

This distinction prevents the architecture from treating every temporary ripple as durable personalization. A single unusual prompt may excite the membrane, but it should not permanently reshape the user’s operating profile. Durable adaptation should occur only when activations are repeated, coherent, task-relevant, confidence-supported, and allowed by privacy, safety, policy, and user-control constraints. In this sense, the latent training plane acts as an adaptation-pressure filter for stable personalization rather than a memory store or raw log of interaction history.

Let $L(r, \theta, t)$ denote accumulated adaptation pressure at a radial and angular position in the membrane. While $u(r, \theta, t)$ describes current field activation, $L(r, \theta, t)$ describes a slower, decaying signal that can bias future routing only when coherent patterns repeat over time.

Here, ρ is the adaptation-rate coefficient, $Q_{ij}(t)$ is the channel-coherence signal from Section 4.5, Δu is the field-level change produced by data excitation and governor-constrained propagation, and μ is the decay rate. The expression makes coherent corridor activation eligible to accumulate

as personalization pressure, while isolated spikes or noisy activation decay unless reinforced over time and permitted by the governing constraints.

The latent training plane can also update the curvature structure of the membrane. If repeated interaction strengthens a V-channel region, the corresponding curvature weight can increase, making that corridor more preferred during future routing. Conversely, channels that are rarely used, unstable, or low-coherence can decay toward a neutral coupling value.

Curvature Readiness Update Rule

$$K_{ij}(t+1) = K_{ij}(t) + \alpha_K \int_{\Omega_{ij}} L(r, \theta, t) d\Omega - \beta_K (K_{ij}(t) - K_{\text{neutral}})$$

In this expression, α_K controls how strongly accumulated latent activation reinforces the channel, β_K controls decay toward a neutral curvature prior, and K_{neutral} represents the default coupling strength before durable personalization. This keeps adaptation gradual and reversible: user preference can strengthen a corridor, but unused or unreliable pathways relax over time.

Operationally, the latent training plane separates immediate runtime behavior from longer-term personalization readiness. The interior identity field determines how the system responds now; the latent training plane determines whether repeated response patterns should adjust future routing preferences, coefficient priors, or calibration settings. This supports a controlled adaptation model: data excites the field, coherent activation contributes to the latent plane, the latent plane can reshape curvature or readiness gradually, and the governor prevents this process from becoming unbounded, irreversible, or privacy-invasive.

The practical implication is that personalization becomes evidence-based rather than immediate. The model can respond flexibly to a single task without assuming that the task represents a durable preference. Only repeated, coherent, user-consistent activation should modify the latent training plane strongly enough to change future routing. This bridges end-user interaction, simulation, and model-training interpretation while preserving the paper's core principle: identity should evolve as a governed trajectory, not as an uncontrolled accumulation of prompts.

4.7 The Deformable Membrane

U.F.O. v2.4.0 · Architecture Supplement

4.7.1 Motivation

Prior releases modeled the behavioral boundary of a unit field object as a fixed circle — a convenient idealization that decoupled geometry from activation state. In practice, behavioral strings do not activate uniformly. Sustained or asymmetric excitation deforms the field in ways that a rigid boundary cannot represent, and the governor is left routing against a geometry that no longer reflects the unit's actual operational posture.

Version 2.4.0 replaces the fixed circular membrane with a **deformable polar boundary** whose shape is a direct function of string activation history. This unlocks three downstream capabilities: expressive

tangent geometry, asymmetric boundary curvature that encodes behavioral emphasis, and a richer signal surface for governor routing and compute allocation.

4.7.2 Definitions

Let a unit field object U be characterized at time t by:

- A **string activation vector** $a(t) \in [0,1]^n$, where each component a_i represents the sustained activation level of behavioral string s_i .
- A **radial deformation function** $r(\theta, t)$, mapping angular position $\theta \in [0, 2\pi)$ to boundary radius.
- A **membrane trace** $\mathcal{M}(t)$, the closed curve traced by $r(\theta, t)$ in the unit's local polar frame.

In v2.3.0 and earlier, $r(\theta, t) = R_0$ for all θ , where R_0 is a fixed nominal radius. In v2.4.0:

$$r(\theta, t) = R_0 + \sum_{i=1}^n a_i(t) \cdot \varphi_i(\theta)$$

where $\varphi_i(\theta)$ is the **angular basis function** of string s_i — a smooth, compactly supported perturbation centered at the angular home position θ_i of that string.

4.7.3 String Stretch and Boundary Deformation

A string s_i is said to be **stretched** when its activation $a_i(t)$ exceeds a threshold α_{str} for a duration $\Delta t \geq \tau_{\text{stretch}}$. Stretch is not instantaneous excitation; it is the geometric consequence of *sustained behavioral load*.

The angular basis function takes a Gaussian form:

$$\varphi_i(\theta) = A_i \exp(-((\theta - \theta_i)^2 / (2\sigma_i^2)))$$

where $A_i \geq 0$ is the maximum radial extension for string s_i , set at initialization, and σ_i controls the angular spread of influence — narrow for strings with tightly scoped behavior and wide for strings governing broad behavioral regimes.

When multiple strings stretch simultaneously in adjacent angular sectors, their basis functions superpose, producing **pointed or ridged protrusions** in $\mathcal{M}(t)$. When activation is sparse or uniform, the membrane relaxes toward a near-circular form. The membrane is therefore a **live geometric record of behavioral emphasis**.

Relaxation. To prevent permanent deformation under brief excitation, each term decays toward zero when activation subsides:

$$\dot{a}_i(t) = -\lambda_i \cdot a_i(t) \text{ when } s_i \text{ is inactive}$$

where λ_i is a string-specific relaxation rate. This ensures the membrane is adaptive, not accumulative.

4.7.4 Tangent Lines as Expressive Signals

At any point $P(\theta)$ on $\mathcal{M}(t)$, the **tangent vector** is:

$$\mathbf{T}(\theta, t) = d/d\theta [r(\theta, t)\cos\theta, r(\theta, t)\sin\theta] = [r'(\theta, t)\cos\theta - r(\theta, t)\sin\theta, r'(\theta, t)\sin\theta + r(\theta, t)\cos\theta]$$

where $r'(\theta, t) = \partial r / \partial \theta$.

On a fixed circle, $r' \equiv 0$ and tangent vectors carry no information beyond position. On a deformed membrane, $r'(\theta, t)$ encodes the *rate of change of behavioral load* across angular sectors. Regions of high $|r'|$ mark **transition zones** — boundaries between dominant behavioral modes — and the tangent directions at these zones are consequently expressive: they indicate the directionality of emphasis shift.

The **tangent expressivity scalar** at θ is defined:

$$\mathcal{E}(\theta, t) = | r'(\theta, t) / r(\theta, t) |$$

High $\mathcal{E}$ marks a sharp boundary transition; low $\mathcal{E}$ marks a smooth, uniform region. The governor collects $\mathcal{E}(\theta, t)$ as a surface signal sampled at k angular positions, yielding the **expressivity profile** $\mathbf{e}(t) \in \mathbb{R}^k$.

4.7.5 Governor Integration

The governor in v2.3.0 received a scalar **activation magnitude** and a categorical **mode label** per unit. In v2.4.0, each unit additionally exports:

Signal	Dimension	Description
$\mathbf{e}(t)$	k	Tangent expressivity profile
$\theta_{\text{peak}}(t)$	1	Angular position of maximum radial extension
$\rho(t)$	1	Membrane asymmetry index
$\kappa_{\text{max}}(t)$	1	Maximum curvature of $\mathcal{M}(t)$

Asymmetry index. A single scalar summarizing global membrane distortion:

$$\rho(\hat{t}) = 1 - R_{\min}(\hat{t}) / R_{\max}(\hat{t}), \rho \in [0, 1)$$

A perfectly circular membrane yields $\rho = 0$. A sharply pointed membrane approaches $\rho \rightarrow 1$.

Governor routing logic. The governor interprets these signals as follows:

- **High ρ , concentrated θ_{peak}** $\rightarrow$ unit is in a focused, high-intensity behavioral mode. The governor may prioritize this unit for compute, or treat it as a specialist and route compatible inputs preferentially toward it.
- **High $\mathcal{E}$ at multiple θ positions** $\rightarrow$ unit is navigating a behavioral transition or holding tension between competing modes. The governor may buffer inputs to this unit or dispatch a supervisory arbitration pass.
- **Low ρ , low $\mathcal{E}$** $\rightarrow$ unit is in a relaxed, general-purpose posture. Eligible for broad routing.

This replaces the v2.3.x heuristic of routing by activation magnitude alone, which was insensitive to *shape* — two units could share the same total activation energy while exhibiting fundamentally different behavioral geometries.

4.7.6 Adaptive Compute Allocation

Compute allocation in v2.4.0 is **membrane-aware**. The governor maintains a **geometric priority weight** per unit:

$$w_U(t) = w_0 + \alpha \cdot \rho(t) + \beta \cdot \kappa_{\max}(t) + \gamma \cdot ||\mathbf{e}(t)||_2$$

where α , β , and γ are tunable coefficients governing the relative influence of asymmetry, curvature, and expressivity on resource allocation. Units with strongly deformed membranes receive elevated priority; quiescent units yield compute back to the pool.

This gives the governor a **robust geometric outlook**: allocation decisions are grounded in the lived shape of each unit's behavioral state, not merely its intensity.

4.7.7 Summary of Changes from v2.3.0

Property	v2.3.x	v2.4.0
Boundary shape	Fixed circle, radius R_0	Deformable polar curve $r(\theta, t)$
String stretch representation	Scalar activation only	Radial deformation via $\phi_i(\theta)$
Tangent information	Uniform, non-expressive	Encodes behavioral transition rate
Governor routing signals	Magnitude + mode label	Magnitude + ρ , θ , $\mathbf{e}$, $\kappa_{\max}$
Compute allocation	Activation-magnitude heuristic	Geometric priority weight $w_U(t)$
Membrane memory	None	Relaxation decay λ_i per string

High-level distinction. Section 4.7 shifts the membrane from a static boundary into a live geometric signal: identity is no longer represented only by how much a behavioral string activates, but by how that activation reshapes the unit's boundary, tangent structure, curvature, and asymmetry over time. This distinction matters because it turns AI identity into something the governor can read geometrically—shape, transition, and deformation become routing signals, not just visual metaphors. With the deformable membrane established as a measurable behavioral surface, the next section can interpret AI identity more broadly as a structured geometry rather than a fixed personality preset.

4.7.8 Local Curvature, Bidirectional Deformation, and Boundary Geometry

The introduction of a deformable membrane in §4.7 implies that the outer boundary of the identity surface is not a fixed circular constraint but a dynamic geometric object whose shape responds to asymmetric behavioral activation. This subsection formalizes that behavior by defining local curvature, inward and outward deformation, and the governor-readable signals that arise from boundary geometry.

Local Radius Deviation

Let $R(\theta, t)$ denote the instantaneous radial distance of the membrane boundary at angular position θ . The neutral baseline is the constant radius R_0 . Deformation is expressed as:

$$\Delta R(\theta, t) = R(\theta, t) - R_0$$

Positive values of ΔR indicate outward stretch; negative values indicate inward collapse. Both are permitted and expected under asymmetric tension, cost pressure, or governor suppression.

Curvature as a Behavioral Signal

Local curvature $\kappa(\theta, t)$ is defined as:

$$\kappa(\theta, t) = \partial^2 R(\theta, t) / \partial \theta^2$$

Regions of high curvature correspond to sharp geometric features—points, ridges, or concave folds—created by sustained directional activation or strong suppression forces. These features provide the governor with a richer signal than amplitude alone: curvature encodes *how abruptly* the membrane geometry changes, not merely *how far* it moves.

Tangent Direction and Expressivity

The tangent vector at the boundary is:

$$\tau(\theta, t) = \partial R(\theta, t) / \partial \theta$$

This term captures the *directional slope* of the membrane. Large tangent values indicate expressive, directional identity shaping—regions where behavioral activation is not only strong but *changing rapidly* across adjacent strings. The governor interprets these tangent gradients as indicators of emerging behavioral bias or directional reasoning flow.

Bidirectional Deformation and Inward Collapse

Because tension T_i , stiffness K_i , and local cost C_i are all bidirectional forces, the membrane can deform inward as well as outward. Inward deformation occurs when:

- cost pressure increases locally,
- the governor suppresses optional expansion,
- tension gradients pull the surface inward, or
- coherence drops and the system contracts toward stability.

Formally, inward deformation is captured by:

$$\Delta R(\theta, t) < 0$$

This concave geometry is not a failure mode; it is a *stability signal*. It indicates that the governor is actively reducing depth, cost, or instability in that region.

Governor Interpretation of Boundary Geometry

The governor reads the tuple:

$$G_{\text{geom}}(\theta, t) = (\Delta R(\theta, t), \tau(\theta, t), \kappa(\theta, t))$$

This geometric signature augments the existing behavioral signals (activation, cost, tension, stiffness) with shape-aware information. The governor uses these terms to:

- suppress unstable outward spikes,
- reinforce coherent corridors,
- collapse incoherent regions inward,
- allocate compute directionally, and
- maintain a bounded, geometry-aware identity surface.

Interpretation

With §4.7.1, the membrane becomes a fully dynamic surface whose geometry—radius, slope, and curvature—participates directly in behavioral governance. Identity is no longer represented solely by string activation but by the *shape* the membrane assumes under load. This completes the transition from a circular control map to a geometry-aware adaptive identity system.

5. Geometric Interpretation of AI Identity

5.1 Geometry as a Conceptual Lens

Recent discussions in computational geometry—including conversations with colleagues working on finite Voronoi-cell ensembles—highlight an important principle: **complex systems rarely resolve into one perfect, rigid structure**. Instead, they tend to express themselves as **ensembles**: distributions of shapes, behaviors, or configurations that vary locally while exhibiting a **stable, reproducible global signature**.

This insight provides a useful conceptual parallel for AI identity.

Just as certain geometric systems do not produce a single Platonic cell but instead generate a **mixed-profile ensemble** with consistent statistical properties, an AI’s behavioral identity should not be treated as a single fixed “mode” or personality. Instead, it should be understood as a **multidimensional ensemble** of behavioral parameters — what I describe in this paper as the *twelve behavioral strings* distributed across multiple cognitive layers.

In this model:

- Each **string** represents a behavioral dimension (e.g., depth, tone, initiative).
- Each **layer** represents a structural domain of the AI (e.g., reasoning, preference, context).
- The AI’s identity emerges not from one dominant setting, but from the **distribution** of these parameters across layers.
- The resulting configuration forms a **radial behavioral signature** — a reproducible pattern that adapts to user preference while maintaining internal coherence.

This ensemble-based view of AI identity has two important consequences:

1. It avoids the rigidity of “preset personalities.”

Just as no single Voronoi cell shape captures the full geometry of a non-crystalline system, no single behavioral preset can capture the diversity of user needs. An ensemble model allows the AI to express a **range** of behaviors tuned to the individual.

2. It enables compute-aware personalization.

Because each behavioral string has a measurable impact on computational cost — such as token usage, reasoning depth, or context expansion — tuning the ensemble allows the system to **optimize compute** without degrading user experience. The AI becomes not only more personal, but also more efficient.

This geometric analogy is not a technical dependency; it is a **conceptual framing** that helps clarify why a radial, multi-string identity model is both natural and powerful. It illustrates that the most robust systems — whether geometric or cognitive — are often those that embrace **structured variability** rather than rigid uniformity.

5.2 Circular Waveform Identity Model

In this model, the AI’s identity can be understood as a **continuous behavioral waveform wrapped into a circle**, rather than a set of discrete, isolated parameters. A traditional wavelength oscillates across a linear axis, but when bent into a closed loop, it becomes a **standing wave distributed across 360 degrees**. The twelve behavioral strings correspond to **sample points** along this circular waveform: the **peaks** represent strong expression of a given behavioral dimension, while the **valleys** represent minimal expression. This framing transforms AI identity from independent sliders into a **single coherent oscillation** whose amplitude, curvature, and phase relationships define behavior across layers. Because amplitude in this circular wave maps to **compute intensity**—higher peaks requiring deeper reasoning, longer outputs, or broader context—the model provides a natural mechanism for compute-aware personalization. The result is a system where user preference shapes not only the AI’s style, but also its **energy footprint**, allowing Microsoft to optimize both experience and efficiency through a unified geometric representation.

5.3 Amplitude, Radius, and Thickness

In the radial waveform model, it is essential to distinguish **amplitude**, **radius**, and **thickness** because each represents a different aspect of AI behavior. **Amplitude** reflects the *intensity* of a behavioral string—how strongly the AI expresses depth, creativity, initiative, or another dimension. **Radius** represents the AI’s *cognitive reach*, such as how far it extends into reasoning chains, context windows, or multi-step analysis. **Thickness**, not amplitude, corresponds to *compute load*: the energetic “bandwidth” required to sustain a behavior, including token generation, attention cost, and tool usage. By separating these axes, the model avoids conflating personality with computation and creates a layered geometry where expression, cognition, and energy each have their own visual signature. This yields a more accurate and intuitive

representation of how AI systems balance user preference, reasoning depth, and resource efficiency within a single framework.

6. Compute Allocation and Cost-Aware Reasoning

6.1 Dynamic Compute Reallocation Through Radial Contraction

In this model, deep reasoning in one behavioral dimension reallocates compute by expanding that sector’s radius while neighboring sectors contract toward the center. When a string requires extended inference—such as multi-step reasoning, long-context retrieval, or complex tool use—the system pushes outward along that radius, increasing cognitive reach and local compute thickness. Because the total compute envelope is fixed, this expansion causes other strings to operate closer to the center, reducing their reasoning depth and compute load. The result is **dynamic compute reallocation**: the AI concentrates resources on the task that requires depth while maintaining lightweight, low-cost behavior across the remaining dimensions. This geometry provides a transparent mechanism for compute savings at scale, ensuring that deep reasoning is available when needed without allowing overall system cost to spike unpredictably.

6.2 The Compute Envelope and V-Shaped Reasoning Channels

The compute envelope defines the maximum radius the AI can reach during reasoning, but the system does not expand uniformly. Because each behavioral string shares attributes with adjacent strings, the model forms natural **V-shaped reasoning channels** where related dimensions overlap. When the AI pushes outward along one string—such as depth or technical detail—neighboring strings partially expand with it, creating a corridor of aligned attributes that supports coherent, low-friction reasoning. This V-channel becomes the preferred path for deep inference, allowing compute to flow efficiently toward the region of highest relevance while minimizing activation in unrelated sectors. As a result, the AI achieves deeper reasoning at lower cost: the envelope stretches locally, the corridor thickens where needed, and remaining strings contract toward the center to conserve resources.

6.3 Cost-Aware Propagation

In the radial identity model, propagation is extended to account explicitly for **compute cost** and **reasoning depth**, so that intelligence flows along paths that are both cognitively coherent and energetically efficient.

This propagation model connects the V-channel training structure in Section 4 to the inference-time cost controls formalized in Section 7.

6.3.1 Local State and Cost

Each string:

Local String State Definition

$$s_i = (A_i, \varphi_i, R_i)$$

where A_i is amplitude, φ_i is phase, and R_i is reasoning depth for string i .

has a **local cost** C_i defined as a function of radius, reasoning depth, and amplitude:

Local Cost Function

$$C_i = f(R_i, A_i)$$

As an introductory approximation, a simple linear form is:

Local Linear Cost Approximation

$$C_i = \lambda_R R_i + \lambda_A A_i$$

where λ_R and λ_A weight the cost contribution of radius/depth and amplitude in the introductory linear model.

This linear form is later generalized in Section 7.1 to include nonlinear depth, tool use, context expansion, dynamic tension, and stiffness.

Total system cost:

Total System Cost Rule

$$C_{\text{total}} = \sum_{i=1}^{12} C_i$$

where C_{total} is the aggregate compute cost across all twelve behavioral strings.

6.3.2 Base Propagation Weight

Without cost, propagation from active string k to candidate string j is governed by:

Base Propagation Weight

$$P_{k \rightarrow j} = f(A_j) \cdot g(|\varphi_k - \varphi_j|)$$

where $P_{k \rightarrow j}$ is the propagation pressure from source string k to target string j , f captures amplitude attraction, and g captures phase alignment.

- $f(A_j)$ increases with A_j , so stronger behavioral activation attracts reasoning flow.
- $g(|\varphi_k - \varphi_j|)$ decreases with phase distance, so phase alignment favors propagation.

6.3.3 Inverse Cost Factor

To make cost an active routing force, define an **inverse cost factor**:

Before introducing the inverse-cost factor, the following analogy explains why cost should actively shape routing rather than simply be measured after the fact.

Interpretive analogy — inverse-cost routing: The inverse-cost factor works like route selection through a city with tolls, traffic, and preferred corridors. A direct road may look attractive, but if it is congested or expensive, the governor makes a nearby lower-cost route more appealing. Reasoning still reaches the destination, but it prefers paths that are aligned, useful, coherent, and affordable.

The equation below expresses that routing preference by making higher-cost target strings mathematically less attractive.

Inverse-Cost Routing Factor

$$I_j = 1/(1 + C_j)$$

This yields a **cost-aware propagation weight**:

Cost-Aware Propagation Weight

$$P_{k \rightarrow j}^{\text{cost}} = P_{k \rightarrow j} \cdot I_j$$

where $P_{k \rightarrow j}^{\text{cost}}$ is propagation pressure after discounting the target string by its inverse-cost factor I_j .

- High $C_j \Rightarrow$ small $I_j \Rightarrow$ the corridor becomes less attractive.
- Low $C_j \Rightarrow$ large $I_j \Rightarrow$ the corridor becomes more attractive.

Reasoning prefers **low-cost, phase-aligned, high-amplitude paths**.

6.3.4 Depth Update with Cost-Aware Flow

Reasoning depth at the target string is updated using the cost-aware weight:

Bounded Depth Propagation Rule

$$R'_j = R_k \cdot h(P_{k \rightarrow j}^{\text{cost}})$$

where R'_j is updated depth at target string j , R_k is source-string depth, and h maps propagation strength into depth preservation or attenuation.

The modulation function h has the following intended behavior:

- $h(x) \approx 1$ for strong, aligned, low-cost propagation, so depth is approximately preserved.
- $h(x) \approx 0$ for weak, misaligned, or high-cost propagation, so depth is attenuated.

This ensures that **deep reasoning** is sustained primarily within **low-cost, structurally coherent corridors**—the V-channels formed by adjacent, aligned strings.

6.3.5 Envelope and Global Cost Constraint All

radii remain bounded by the envelope:

Compute Envelope Constraint

$$R_i \leq R_{\max}$$

If a propagation step would violate this:

Depth Envelope Clamp

$$R'_j = \min(R'_j, R_{\max})$$

where $R_{\max}$ is the maximum permitted reasoning radius under the compute envelope.

As an idealized conservative case, the system may enforce a **non-increasing global cost** constraint.

Conservative Global Cost Constraint

$$C'_{\text{total}} \leq C_{\text{total}}$$

This encodes a simplified case in which each propagation step should **not increase** total compute cost. Section 7.5 generalizes this into a soft global cost behavior that permits justified task-driven expansion.

6.3.6 Suppression Tied to Global Cost

Non-active strings contract in depth and intensity:

Suppression Dynamics Rule

$$R'_i = \sigma_R(C_{\text{total}}) R_i, \quad A'_i = \sigma_A(C_{\text{total}}) A_i$$

where σ_R and σ_A are suppression functions that contract radius and amplitude as total cost increases.

With:

$$0 \leq \sigma_R(C_{\text{total}}), \sigma_A(C_{\text{total}}) \leq 1$$

Higher $C_{\text{total}} \Rightarrow$ smaller σ_R and $\sigma_A \Rightarrow$ stronger contraction. Lower $C_{\text{total}} \Rightarrow$ milder contraction.

6.3.7 Net Effect: Depth with Inverted Cost Pressure

The result is a system where:

- **Reasoning depth** R_i is preserved along **low-cost, phase-aligned corridors**.

- **High-cost regions** become less attractive via I_j , reducing their role in deep reasoning.
- **Global cost** is stabilized or reduced over time.
- Intelligence **flows** through structured, efficient paths instead of jumping arbitrarily.

This is cost-aware, depth-sensitive propagation: a geometry where reasoning seeks **coherent, low-cost channels** inside a bounded envelope.

7. Governing Dynamics Integrated with Reasoning

This section integrates the earlier membrane model from Section 1, the behavioral cost signatures from Section 2, the vibrational dynamics from Section 3, and the V-channel routing logic from Section 4 into a single control framework.

The following formulation gives the governor's formal expressions and their architectural interpretations, showing how the **model routes reasoning flow, allocates compute, and stabilizes behavior within the identity membrane**.

7.1 Local State and Cost

Mathematics

$$s_i = (A_i, \varphi_i, R_i, \tau_i, \kappa_i)$$

$$C_i = \lambda_R R_i^2 + \lambda_A A_i + \lambda_T T_i + \lambda_C \log(1 + \text{context}_i) + \lambda_\tau \tau_i + \lambda_\kappa \kappa_i$$

Component summary: the local state s_i captures what string i is doing at a given moment: A_i measures behavioral intensity, φ_i fixes its angular position, R_i measures reasoning depth, τ_i measures excitation resistance, and κ_i measures resistance to unstable deformation.

Cost summary: the local cost C_i converts that state into an energetic footprint. The R_i^2 term penalizes deep inference nonlinearly; A_i captures expressive intensity; T_i captures tool or agentic cost; $\log(1 + \text{context}_i)$ captures context expansion with diminishing returns; and τ_i and κ_i add adaptive resistance from tension and stiffness.

In short: Section 7.1 turns each behavioral string into a measurable control unit. The governor can then compare strings not only by what behavior they express, but by how much reasoning depth, context, tool use, and adaptive resistance they require.

Governor decision effect: local cost C_i functions as the governor's routing and suppression signal. When C_i is low, the string remains attractive for propagation, depth preservation, and continued activation. When C_i rises, inverse-cost routing reduces that string's attractiveness, suppression becomes more likely, and future depth or tool expansion requires stronger task justification. In this way, cost is not merely recorded after behavior occurs; it actively shapes where reasoning flows next.

Each string’s vibrational parameters modulate its local cost C_i and its contribution to the global Lyapunov energy V , making **the governor a control system** over a damped behavioral membrane rather than a purely abstract state machine.

Interpretation

- **Variables:** A_i = behavioral intensity; φ_i = angular phase; R_i = reasoning depth; τ_i = dynamic tension; κ_i = dynamic stiffness; C_i = energetic cost.
- **Why nonlinear:** R_i^2 models the disproportionate cost of deep inference; T_i captures discrete tool costs; $\log(1 + \text{context}_i)$ captures diminishing returns on context expansion.
- **Architectural effect:** every local change in depth or expression has a measurable compute consequence, enabling explicit tradeoffs between behavior and energy.

7.1.1 Mapping Local Cost to Observable Signals

For implementation or simulation, the abstract local cost term C_i should be estimated from observable inference signals. These signals may include generated tokens, context-window size, approximate attention cost, retrieval calls, tool-call cost, latency, and repeated correction or regeneration. A practical measurable form is:

$$C_i^{\text{obs}} = \alpha_{\text{tokens}} \text{tokens}_i + \alpha_{\text{attn}} \text{attn_flops}_i + \alpha_{\text{ctx}} \text{context}_i + \alpha_{\text{retr}} \text{retrievals}_i + \alpha_{\text{tool}} \text{tools}_i + \alpha_{\text{lat}} \text{latency}_i$$

where each α coefficient converts a measured runtime signal into normalized local cost units.

This observable form does not replace the architectural cost equation; it operationalizes it for measurement. In simulation, these terms can be normalized to $[0,1]$. In prototype systems, they can be estimated from request telemetry and aggregated in privacy-preserving form. The purpose is to make local cost measurable enough to compare routing decisions, governor behavior, and Lyapunov energy trends against real or synthetic workloads.

The key implementation assumption is therefore modest: membrane variables need not be directly observable inside the base model; they only need to be estimable from **privacy-preserving runtime telemetry** well enough to guide and evaluate control behavior. For example, C_i can be estimated from token count, context-window size, retrieval calls, tool calls, and latency, while $Q_{ij}(t)$ can be approximated from route-selection consistency, correction rate, and whether activation remains concentrated in the intended V-channel.

As a conservative starting point, generated tokens and context expansion should receive moderate weights, retrieval and tool calls should receive higher weights when they invoke external systems, and latency should be tracked as both a user-experience signal and a proxy for system load.

The same V-channel structure that routes inference can also route training updates: data enters as a behavioral activation pattern, propagates along curved corridors according to channel curvature, and is resisted by local cost, tension, and stiffness before contributing to parameter change.

7.1.2 Cost Taxonomy and Quality-Preserving Reduction

The governor treats cost as a multidimensional runtime signal rather than a single token count. Observable cost may include generated tokens, reasoning-depth expansion, context-window use, retrieval calls, tool calls, latency, correction or regeneration frequency, and instability-recovery overhead. This taxonomy makes cost visible at the same level as behavior: the system can ask not only what response was produced, but which resources were used to produce it and whether those resources were justified by the task.

A practical cost taxonomy can distinguish the following categories: token/output cost; reasoning-depth cost; context-window cost; retrieval cost; tool or agentic cost; latency and system-load cost; correction or regeneration cost; and instability-recovery cost. These categories are intentionally separable because they have different product meanings. A long answer, a large context window, a tool call, and a repeated correction loop may all be expensive, but they should not be governed in the same way.

For a behavioral string i , these terms can be collected into a normalized local cost vector:

$$c_i(t) = [c_{\text{tok}}, c_{\text{depth}}, c_{\text{ctx}}, c_{\text{retr}}, c_{\text{tool}}, c_{\text{lat}}, c_{\text{corr}}, c_{\text{stab}}]$$

where each component is normalized to a comparable scale before aggregation. The governor can then project the vector into a scalar observable cost only after applying explicit weights:

$$C_{i\text{obs}}(t) = wC^T c_i(t)$$

This weighted projection gives implementation teams a tunable bridge between raw telemetry and the architectural cost variable C_i . For example, retrieval and tool calls may receive higher weights in enterprise workflows because they invoke external systems, while token and latency costs may receive moderate weights because they affect both infrastructure load and user experience. Correction and regeneration costs are especially important because they indicate that earlier compute may have been low quality, misrouted, or insufficiently governed.

Avoidable versus justified cost. The governor should not treat all compute expansion as waste. Avoidable cost is expansion that does not improve task success, safety, correctness, accessibility, user-requested depth, or downstream usefulness. Justified cost is additional compute that improves outcome quality, reduces ambiguity, supports safety or compliance, prevents downstream correction, or satisfies an explicitly higher-value task. In other words, the architecture minimizes avoidable cost, not cost in isolation.

Quality floor. Cost reduction must preserve minimum standards for task completion, safety, accessibility, correctness, user trust, and requested explanation depth. The governor should suppress optional verbosity, unnecessary retrieval, excessive branching, redundant context use, and low-value tool activity first. It should not suppress essential safety reasoning, required accessibility support, critical context, or the level of explanation the user explicitly needs.

This distinction makes U.F.O. a **quality-preserving cost governor** rather than a cheapness heuristic. The objective is to reduce wasteful expansion while preserving or improving the user's actual outcome. Extra compute is acceptable when it buys correctness, safety, coherence, or user value; it is suppressed when it reflects unnecessary length, incoherent routing, repeated correction, or low-value expansion beyond the task's needs.

7.2 Robust Propagation Pressure

Mathematics

$$P_{k \rightarrow j} = \text{sigmoid}(A_j) \cdot ((1 + \cos(\varphi_k - \varphi_j))/2) \cdot I_j$$

with a corrected inverse-cost choice:

$$I_j = 1/(1 + C_j) \quad \text{or} \quad I_j = \exp(-\lambda_c C_j), \text{ where } \lambda_c > 0$$

Interpretation

- **Amplitude attraction:** bounded sigmoid gives smooth, saturating pull toward strong behaviors.
- **Phase alignment:** $(1 + \cos(\varphi_k - \varphi_j))/2$ attenuates flow without sign flips: aligned phases approach 1, opposed phases approach 0.
- **Inverse cost:** I_j is strictly decreasing in C_j ; use $1 / (1 + C_j)$ or $\exp(-\lambda_c C_j)$ to ensure higher cost always reduces attractiveness.
- **Architectural effect:** reasoning is routed to corridors that are **high-signal, phase-coherent**, and **energetically cheap**, producing continuous flow rather than abrupt mode switches.

7.3 Depth Update Within the Compute Envelope

Mathematics

$$R'_j = \min(R_{\max}, R_k \cdot h(P_{k \rightarrow j}))$$

Interpretation

- **Preservation vs attenuation:** $h(x)$ is bounded in $[0,1]$; for large x , it approaches 1 so depth is approximately preserved, while for small x , it attenuates depth.
- **Envelope enforcement:** the $\min(R_{\max}, \cdot)$ operator caps depth to the compute envelope.
- **Architectural effect:** deep reasoning is sustained only along coherent, low-cost V-channels; attempts to push depth into misaligned or expensive regions are attenuated or capped.

7.4 Updated Local Cost

Mathematics

$$C_j = \lambda_R (R'_j)^2 + \lambda_A A_j + \lambda_T T_j + \lambda_c \log(1 + \text{context}_j) + \lambda_\tau \tau_j + \lambda_k \kappa_j$$

Interpretation

- **Immediate accounting:** the system recomputes the energetic footprint after propagation so every step has a direct cost consequence.
- **Architectural effect:** cost updates close the loop between behavior and compute, enabling suppression and reallocation to act on accurate signals.

7.5 Soft Global Cost Behavior

Mathematics

$$C_{\text{total}}' = C_{\text{total}} - \eta \Delta_{\text{efficiency}} + \gamma_{\text{task}} \Delta_{\text{task}}$$

where η weights efficiency pressure, $\Delta_{\text{efficiency}}$ is the estimated cost reduction from routing or suppression, γ_{task} weights justified task expansion, and Δ_{task} is bounded task value.

Interpretation

- **Soft constraint semantics:** η biases the system toward efficiency; γ_{task} permits justified cost increases for high-value tasks.
- **Architectural effect:** the system is flexible—allowed to spend more compute when task value warrants it while trending toward net efficiency over time.

7.5.1 Task-Value Guardrails

Because Δ_{task} permits temporary compute expansion, it should be bounded and governed rather than treated as an open-ended override. A practical implementation can define task value as a normalized scalar:

$$0 \leq \Delta_{\text{task}} \leq 1$$

where Δ_{task} is estimated from explicit user intent, task criticality, expected utility, safety or compliance relevance, and whether additional reasoning is likely to improve outcome quality. This prevents ordinary low-value prompts from repeatedly invoking high-cost expansion.

- **Per-request cap:** each task-driven expansion should have a maximum allowed increase in C_{total} or V .
- **Rolling budget:** repeated small task escalations should be accumulated over a time window so they cannot collectively bypass the compute envelope.

- **Confidence threshold:** task expansion should require sufficient confidence that additional compute will improve task success or user value.
- **Quality and safety floors:** cost controls should never suppress essential safety, correctness, accessibility, or user-requested explanation below a minimum operating level.
- **Auditability:** task-value overrides should be measurable in validation so reviewers can distinguish justified expansion from runaway cost.

With these guardrails, $\gamma_{\text{task}}\Delta_{\text{task}}$ becomes a controlled release valve rather than a loophole. The governor can spend additional compute when value is justified, while still preserving long-run stability, cost discipline, and user trust.

7.6 Suppression Dynamics

Mathematics

$$R'_i = \sigma_R(C_{\text{total}}) R_i, \quad A'_i = \sigma_A(C_{\text{total}}) A_i$$

with $0 \leq \sigma_R(C_{\text{total}}), \sigma_A(C_{\text{total}}) \leq 1$, and both suppression functions decreasing as C_{total} increases.

Interpretation

- **Concentration of resources:** higher global cost $\rightarrow$ stronger contraction of inactive sectors.
- **Architectural effect:** suppression prevents runaway compute, enforces focus, and enables local stretching of the envelope where needed.

7.7 Unified Update Rule

Mathematics

$$(A'_j, R'_j, C'_j) = F_{\text{gov}}(A_j, R_k, C_j, \tau_j, \kappa_j, \varphi_k - \varphi_j)$$

Interpretation

- **Integrated operation:** this single update integrates attraction, alignment, inverse cost, depth propagation, envelope enforcement, and cost accounting.
- **Architectural effect:** it routes reasoning along phase-coherent, low-cost corridors, preserves depth where appropriate, updates energetic accounting, and enables system-level stabilization via suppression and soft global control.

7.8 Candidate Lyapunov Energy and Stability

Before the stability expression, the following analogy gives an intuitive picture of what it means for ordinary updates to return toward a lower-energy state.

Interpretive analogy — Lyapunov stability: The Lyapunov-style energy can be understood as a marble inside a shallow bowl. Ordinary updates should roll the marble back toward the low-energy center rather than letting it climb the walls. Task pressure may push the marble upward temporarily, but the governor’s role is to ensure it returns toward a stable basin instead of escaping the bowl.

The candidate energy function below turns that intuition into a measurable stability signal over compute cost, radial imbalance, tension, and stiffness.

Mathematics

$$V = C_{\text{total}} + \alpha \sum_i (R_i - \bar{R})^2 + \beta \sum_i \tau_i + \delta \sum_i \kappa_i$$

where C_{total} is aggregate compute cost, R_i is reasoning depth, $\bar{R}$ is average radius, τ_i is dynamic tension, κ_i is dynamic stiffness, and α , β , and δ are positive weighting constants.

The desired ordinary-update condition is $\Delta V = V' - V \leq 0$ whenever $\Delta_{\text{task}} = 0$

When $\Delta_{\text{task}} > 0$, temporary increases in V are permitted only as explicitly justified task-value expenditures.

Stability operates across three radial zones: the baseline center acts as the low-energy recovery target, the interior identity field acts as the diagnostic space where coherent V-channel resonance is distinguished from noisy activation, and the outer compute envelope acts as the enforcement boundary that prevents unbounded reasoning depth or resource expansion.

Interpretation

- **Energy meaning:** V combines total compute expenditure, radial imbalance in reasoning depth, membrane tension, and membrane stiffness. The first term discourages unnecessary energetic cost, the second discourages unstable radial dispersion, and the final two terms discourage excessive tightness or rigidity in the identity membrane.
- **Stability condition:** Under inverse-cost routing, bounded depth updates, envelope enforcement, and suppression of inactive regions, ordinary updates should contract the system toward lower energy, giving $\Delta V \leq 0$ when no task override is active.
- **Task exception:** The only intended exception is an explicit Δ_{task} invocation, which allows temporary energy growth when additional compute is justified by task value.
- **Architectural effect:** The governor behaves like a dissipative control system: it reduces unnecessary compute and radial instability by default, while preserving the ability to expand deliberately for high-value reasoning.

8. Simulation, Parameterization, and Validation Notes

- **Tuning knobs:** λ_R , λ_A , λ_T , λ_C , λ_τ , λ_κ , η , γ_{task} , α , β , δ , and the functional forms of h , σ_R , and σ_A control responsiveness, conservatism, task sensitivity, and energy weighting.
- **$h(x)$ caveat:** $h(x) = x/(1+x)$ is bounded below 1, so it approximately preserves depth under strong propagation rather than reaching exact preservation at finite pressure. If exact

finite saturation is required, use a capped variant such as $h(x)=\min(1,\alpha x)$, with $\alpha>0$ chosen to set the saturation threshold.

- **Stability sketch:** the candidate Lyapunov energy above formalizes convergence by combining total compute cost with variance in reasoning depth. Under ordinary updates, the governor is intended to satisfy $\Delta V \leq 0$, while task-driven expansion is permitted only through explicit Δ_{task} activation.
- **Next steps (recommended):** extend the completed Project Rainbow toy simulation with broader parameter ranges, richer representative workloads such as retrieval, multistep reasoning, and tool use, and continued empirical checks of corridor selection, depth preservation, convergence, ablation behavior, and cost/task tradeoffs.

9. Validation Roadmap

Validation has begun with the completed Project Rainbow toy simulation, which provides an initial reproducible check of the architecture’s routing, stability, ablation behavior, and return-to-baseline dynamics under simplified synthetic workloads. The proposed architecture should still be evaluated as a staged research program rather than as a finished system. Next, validation should extend the toy simulation through broader parameter sweeps and richer workload families, then move into controlled prototype testing and comparison against baseline AI interaction models. The goal is to determine whether the radial identity membrane produces measurable improvements in personalization, compute efficiency, stability, and user experience.

The roadmap evaluates the mechanisms introduced in Sections 1 through 7, with particular emphasis on whether the governor and Lyapunov energy from Section 7.8 produce measurable stability and cost benefits.

9.1 Testable Claims and Evaluation Metrics

Compute reduction should be validated by comparing observable cost signals—tokens, latency, context size, retrievals, and tool calls—against matched baselines while holding task success, user satisfaction, correction rate, and trust constant or improved. The architecture succeeds only if it reduces avoidable compute rather than merely shortening responses or suppressing useful reasoning.

Claim	Observable Metric	Validation Method	Success Signal
Conciseness reduces avoidable compute.	Token count, latency, and completion cost.	Compare concise-profile users against default-profile users on matched tasks.	Lower token and latency cost without lower satisfaction or task success.

V-channel routing improves coherence.	Human preference, contradiction rate, and response consistency.	Run ablation tests with and without curvature-aware routing.	Higher coherence ratings and fewer unsupported jumps between response modes.
Governor dynamics stabilize cost.	Variance in token use, tool calls, context expansion, and reasoning depth.	Simulate workloads with deep reasoning, retrieval, and agentic actions under governor control.	Reduced variance and fewer runaway-cost episodes while preserving task quality.
Adaptive tension and stiffness prevent unstable behavior.	Oscillation in behavioral settings, repeated overexpansion, and correction frequency.	Compare fixed-parameter membranes with adaptive membranes across repeated tasks.	Faster return to baseline and fewer unstable swings in high-cost strings.
User-defined initial conditions improve personalization.	User satisfaction, preference match, edit rate, and follow-up correction rate.	Test onboarding or early-interaction calibration against generic presets.	Higher preference alignment with fewer corrections and less unnecessary compute.
Lyapunov-style energy predicts stability.	Change in V over time, cost convergence, and task-triggered energy increases.	Track V across ordinary updates and task-driven overrides in simulation.	$\Delta V \leq 0$ during ordinary updates, with controlled increases only during justified task expansion.

9.2 Staged Validation Plan

1. **Simulation phase:** assign initial parameter ranges for amplitude, radius, cost, tension, stiffness, damping, curvature, and suppression. Run synthetic task sequences to observe corridor selection, cost behavior, damping, and convergence.
2. **Ablation phase:** remove one mechanism at a time, such as inverse-cost routing, curvature propagation, adaptive stiffness, or suppression, and measure whether stability or compute efficiency degrades.
3. **Prototype phase:** implement a lightweight inference-time controller that adjusts depth, verbosity, context use, and tool likelihood according to user preference signals and task type.
4. **User-study phase:** compare the prototype against a default interaction model using satisfaction, correction rate, perceived alignment, task completion quality, and compute cost.

5. **Enterprise-scale phase:** test whether the model reduces avoidable token generation, context expansion, retrieval calls, and agentic tool use across high-volume workloads while maintaining or improving user outcomes.

9.3 Expected Research Outcome

A successful validation program would show that personalization and compute optimization are not opposing goals. Instead, user-specific behavioral profiles should make the system more efficient by directing compute only toward the behaviors and reasoning depths that matter for a given user and task. The strongest result would be a measurable reduction in avoidable compute cost while maintaining or improving perceived usefulness, transparency, stability, and trust.

9.4 Risks, Failure Modes, and Mitigation Strategies

Risk or Failure Mode	Potential Effect	Mitigation Strategy
Over-tightening of tension or stiffness.	The system may become underexpressive, overly terse, or reluctant to use depth when depth is needed.	Use adaptive lower bounds, decay schedules, and task-value overrides to prevent the membrane from becoming too rigid.
Incorrect curvature matrices.	Reasoning may flow through biased or inefficient corridors, causing the model to over-favor some behavioral combinations.	Apply curvature regularization, ablation monitoring, and comparison against neutral routing baselines.
User-defined initial conditions overfit early interactions.	The system may personalize too quickly based on a small or unrepresentative sample of user behavior.	Use warm-start priors, gradual personalization ramps, and confidence thresholds before making strong identity updates.
Governor suppression is too aggressive under high load.	Important behaviors may be damped too strongly, reducing usefulness or task completion quality.	Apply soft-floor constraints on amplitude and radius so essential strings remain available even during contraction.
Cost reduction degrades perceived quality.	Users may experience responses as cheaper but less helpful, less complete, or less trustworthy.	Track satisfaction, task success, correction rate, and trust metrics alongside compute savings.

9.5 Data Requirements and Synthetic Benchmarks

Validation should begin with synthetic and privacy-preserving workloads before relying on real user data. The architecture can be stress-tested with generated task sequences that activate different behavioral strings, V-channels, and compute pressures. This allows the membrane, governor, and training-plane assumptions to be evaluated safely before deployment in user-facing systems.

Benchmark Family	Purpose	Example Workloads
Concise response workloads	Test whether the governor can reduce avoidable token output without lowering usefulness.	Summaries, short answers, email drafts, status updates, and quick explanations.
Deep reasoning workloads	Test bounded depth expansion and preservation within high-value V-channels.	Multi-step analysis, mathematical reasoning, planning, policy interpretation, and technical design critique.
Context-heavy workloads	Test context sensitivity, retrieval pressure, and memory-aware activation.	Long-document synthesis, prior-conversation recall, meeting recap, and project-state reconstruction.
Agentic workloads	Test initiative, tool-call likelihood, and suppression of unnecessary action.	Task planning, workflow automation, calendar preparation, and multi-step assistant actions.
Expressive workloads	Test low-cost modulation of tone, warmth, creativity, and style.	Coaching responses, creative drafting, customer communication, and personalized explanation.
Stress tests	Test robustness under conflicting demands and high compute pressure.	Tasks requesting deep, concise, technical, warm, transparent, and tool-using behavior simultaneously.

9.6 Go/No-Go Criteria

To make validation decision-oriented, the research program should define explicit thresholds before prototype expansion. These thresholds should be treated as initial targets for simulation and controlled testing, then adjusted based on empirical evidence and product constraints.

Decision Criterion	Initial Threshold	Go/No-Go Interpretation
Avoidable compute cost reduction	At least 15% reduction on matched workloads.	Go if cost falls without reducing task success, satisfaction, or trust metrics.
Coherence improvement under curvature routing	At least 10% improvement in coherence or preference ratings.	Go if V-channel routing improves continuity compared with neutral or non-curved routing.
Latency overhead under governor control	No more than 5% increase in median latency.	Go if stabilization benefits do not introduce unacceptable interaction delay.

Lyapunov ordinary-update behavior	$\Delta V \leq 0$ in at least 95% of ordinary updates.	Go if energy decreases under normal operation and increases only under explicit task-value expansion.
Personalization quality	Measurable improvement in preference match with no increase in correction rate.	Go if user-defined initial conditions improve alignment without overfitting early behavior.
Stability under stress tests	No sustained runaway cost or repeated oscillation across stress workloads.	Go if adaptive tension, stiffness, damping, and suppression return the membrane to stable operation.

9.7 Baselines and Experimental Controls

Validation should compare the radial identity membrane against clearly defined baseline systems. Without controls, improvements in cost, coherence, or personalization could be attributed to simpler mechanisms such as shorter prompting, static style presets, or generic response-length limits. The following baselines provide the minimum comparison set for determining whether the proposed architecture adds value beyond existing personalization and inference-control strategies.

Baseline or Control	What It Tests	Expected Comparison
Default assistant behavior	Tests whether the architecture improves over standard nonpersonalized interaction.	The radial model should improve alignment and reduce avoidable compute without lowering task success.
Static personalization presets	Tests whether dynamic membrane adaptation adds value beyond fixed style or personality settings.	The radial model should adapt better over time and avoid rigid user profiles.
Response-length control only	Tests whether compute reduction is merely caused by shorter outputs.	The radial model should reduce compute while preserving usefulness, not simply by truncating responses.
No-curvature routing	Tests the value of V-channel propagation and phase-aware coupling.	Curvature-aware routing should improve coherence, corridor selection, and behavioral continuity.
No-governor control	Tests whether inverse-cost routing, suppression, and Lyapunov-style stabilization materially reduce runaway cost.	The full model should show lower cost variance and fewer uncontrolled expansions.

Fixed tension and stiffness	Tests whether adaptive membrane resistance improves stability.	Adaptive tension and stiffness should reduce oscillation and improve return-to-baseline behavior.
Preference-only personalization	Tests whether user alignment alone is sufficient without compute-aware constraints.	The radial model should preserve personalization while producing better cost and stability outcomes.

These controls separate the contribution of each architectural layer. If the full model outperforms the baselines on personalization, coherence, cost variance, and stability, the validation program can more confidently attribute gains to the combined membrane, curvature, adaptive-resistance, and governor framework.

Across all baselines and controls, instrumentation should be limited to observable signals required for validation: token count, latency, context-window size, retrieval or tool-call frequency, response length, correction rate, user preference feedback, inferred string activation, tension and stiffness values, V-channel route selection, and Lyapunov energy change. These signals should be collected in aggregate or privacy-preserving form whenever possible, so the system can evaluate compute savings, stability, and personalization quality without retaining unnecessary raw user content.

9.8 Privacy, User Control, and Responsible Personalization

Because the architecture is user-centered, validation and deployment must treat personalization data as a controlled, user-governed signal rather than as an unrestricted behavioral profile. Early testing should rely on synthetic and privacy-preserving workloads wherever possible, with real user interaction data introduced only under appropriate consent, governance, and enterprise compliance boundaries.

- **Data minimization:** store only the behavioral coefficients, preference signals, or aggregate activation patterns needed for personalization; avoid retaining unnecessary raw prompts, documents, or sensitive context.
- **User control:** provide mechanisms to reset, recalibrate, pause, or narrow personalization so users can correct early overfitting or change their preferred interaction style.
- **Transparency:** expose personalization at the level of understandable behavioral preferences, such as depth, conciseness, tone, context use, and initiative, rather than hidden or opaque profiling.
- **Separation of identity and content:** distinguish the user's behavioral preference vector from personal content, enterprise documents, or private communications used in a task.
- **Enterprise governance:** ensure that any deployment respects tenant-level policy, data residency, retention, access control, and administrator configuration.
- **Quality floor:** compute optimization must not reduce baseline capability, accessibility, safety, or task completion quality for users who need more explanation, context, or support.

A practical warm-start rule should delay strong personalization until sufficient evidence exists. For example, curvature updates, coefficient updates, or durable preference changes can remain low-gain until at least N interactions have been observed and a confidence threshold p_{conf} is reached. Early updates should decay over time unless reinforced by repeated user behavior, preventing the membrane from overfitting to a small or unrepresentative onboarding sample.

$$\text{gain}_{\text{personalization}}(t) = \min(1, n_{\text{interactions}}/N) \cdot p_{\text{conf}}$$

where $n_{\text{interactions}}$ is the number of observed interactions, N is the warm-start horizon, and p_{conf} is confidence in the inferred preference signal.

In implementation, this gain should scale early curvature updates K_{ij} , coefficient updates Δa_i , and durable preference changes so personalization ramps in gradually rather than taking full effect immediately.

This keeps the system user-shaped while making personalization reversible, gradual, and evidence-based. Users should also retain the ability to reset, recalibrate, or narrow the personalization profile if the learned membrane no longer matches their preferences. A practical implementation should therefore treat personalization as reversible, inspectable, and bounded. The goal is not to infer a fixed user identity, but to maintain a lightweight, user-adjustable operating profile that improves usefulness and compute efficiency while preserving privacy, trust, and responsible system behavior.

10. Radial Identity Map and Compute Efficiency Impact

10.1 Overview

The AI’s identity is represented as a **single radial map**—a unified circle composed of **12 behavioral sectors** (the “strings”) and **multiple concentric arcs** (the “layers”). This visualization reflects the underlying architecture that governs reasoning depth, context usage, and computational resources. By tuning this map during onboarding, the system can reduce unnecessary compute expenditure while improving user experience.

The radial map translates the formal architecture from Sections 1 through 7 into a user-facing calibration and visualization layer, then connects that layer to the validation goals in Section 9.

10.2 Inside the Circle: Sectors, Arcs, and Behavioral Depth

10.2.1 Sectors: Behavioral Strings

The circle is divided into **12 radial sectors**, each representing a core behavioral dimension of the AI, such as **depth**, **tone**, **initiative**, and **context sensitivity**. Each sector defines *what kind* of behavior is being tuned.

10.2.2 Concentric Arcs: Layers

Within each sector are **concentric arcs**, each corresponding to a structural layer of the AI:

- **Core Reasoning Layer** — depth, inference, planning, and intermediate verification.
- **Preference Layer** — user-tuned behavior such as tone, conciseness, explanation style, and initiative.
- **Domain Expertise Layer** — technical knowledge, formulas, terminology, and task-specific expertise.
- **Context Layer** — prior conversation, documents, memory, retrieval scope, and situational cues.
- **Agentic Layer** — tool use, workflow actions, autonomy level, and action-monitoring behavior.

These arcs represent *where* the behavior is expressed — from reasoning depth to tool usage.

10.2.3 Hands: Behavioral Expression

Each arc contains a “hand” that extends outward from the center. The **length** of the hand represents the *intensity* of that behavior. The **angle** is fixed by the sector; the **radius** is determined by user preference.

- Short hand → minimal expression
- Long hand → strong expression
- Full tangent → maximum expression

This creates a **multi-layered behavioral fingerprint** unique to each user.

10.3 Behavioral Tuning as Compute Optimization

The radial map is not just a visualization — it is a **compute-allocation engine**.

Each behavioral string has a direct impact on computational cost:

- **Depth** → token usage
- **Transparency** → explanation length and structure
- **Technical detail** → output size
- **Initiative** → agentic tool calls
- **Context sensitivity** → memory retrieval and context window expansion
- **Exploration** → branching reasoning paths

By tuning these strings during onboarding, the system can **right-size compute** to the user’s actual preferences.

10.4 The Outer Circle: The Compute Envelope

The outermost boundary of the radial map represents the **maximum compute budget** the AI is allowed to use for that user. This includes:

- maximum reasoning depth
- maximum token count

- maximum context window expansion
- maximum agentic autonomy
- maximum tool-call frequency

The AI stays within this envelope unless explicitly instructed otherwise.

This creates a **visual and architectural constraint** that prevents runaway compute usage.

10.4.1 Boundary-Loop Interpretation of the Outer Membrane

Although the radial identity membrane is defined primarily as a compute-bounded control surface, its outer boundary can also be interpreted as a **boundary-loop layer**: a geometric trace of how the system has recently allocated depth, cost, and behavioral amplitude. As reasoning flows through V-channels and behavioral strings, the membrane’s boundary accumulates small, path-dependent variations in tension and curvature. These variations do not represent persistent state or long-term memory; rather, they provide a lightweight, geometry-level trace of recent computational dynamics. Before introducing the boundary-loop notation, the following analogy clarifies why a short-lived trace can be useful without becoming persistent memory.

Interpretive analogy — boundary-loop trace: The boundary-loop trace is like a fading footprint in sand. It records where recent pressure touched the outer edge of the membrane, but it is not persistent memory. If another aligned step follows soon after, the system can use the faint trace as a short-horizon guide; if not, the footprint naturally fades.

With that analogy in mind, define $B(\theta, t)$ as a decaying short-horizon boundary variation at phase position θ . The update below expresses this fading trace as a function of recent changes in depth, cost, tension, and stiffness along the outer membrane.

Boundary-Loop Trace Update:

$$B(\theta, t+1) = \rho_B B(\theta, t) + \varepsilon_B (\Delta R(\theta, t) + \Delta C(\theta, t) + \Delta \tau(\theta, t) + \Delta \kappa(\theta, t))$$

Here, ρ_B is a decay factor with $0 \leq \rho_B < 1$, and ε_B controls how sensitively the boundary responds to short-horizon changes in reasoning depth, compute cost, tension, and stiffness. Because the trace decays, it functions as a transient boundary diagnostic rather than persistent memory. The boundary-loop trace complements compute-cost accounting by showing where cost pressure has recently contacted the envelope. While C_i , τ_i , and κ_i measure local expense and resistance, $B(\theta, t)$ records the short-horizon geometric imprint of those pressures along the outer membrane. Persistent or repeated nudges near a phase region can therefore help the governor distinguish justified task-driven expansion from inefficient, unstable, or noisy compute pressure. As the membrane expands under task pressure and then contracts back toward the neutral center, it leaves a transient geometric imprint along the outer boundary. This imprint does not store content or long-term memory; instead, it records the short-horizon pattern of recent depth, cost, tension, and stiffness. Because the trace decays naturally, it acts as a temporary computational scaffold: the next aligned input can be routed with less friction, while the governor gains another signal for distinguishing justified, coherent expansion from unstable or inefficient pressure. This

interpretation does not alter the governor's mechanics. It clarifies that the outer circle is not only a hard limit, but also a structured surface through which short-horizon behavioral history is expressed.

10.5 Calibration Test and Cost Reduction

During onboarding, the user answers a series of preference-based questions. As they respond:

- hands extend or retract layers
- activate or quiet down
- sectors strengthen or weaken
- the compute envelope adjusts

This process tunes the AI to the user's **actual desired behavior**, preventing:

- unnecessary deep reasoning
- unnecessary long outputs
- unnecessary context retrieval
- unnecessary agentic actions
- unnecessary token expenditure

This is **resource-aware personalization** through the radial identity map.

10.6 Estimated Impact on Compute Efficiency

While exact numbers depend on implementation, the architecture enables savings in several high-cost areas:

- **Token reduction** for users who prefer concise answers
- **Reasoning reduction** for users who prefer concise or direct answers
- **Context reduction** for users who don't need long memory
- **Agentic reduction** for users who prefer reactive AI
- **Electricity reduction** through lower GPU load

Even modest reductions across millions of users translate into **significant cost savings**.

At Microsoft scale, the economic significance comes from aggregation. A small per-request reduction in unnecessary reasoning depth, context retrieval, output length, or tool use may appear minor individually, but across high-volume Copilot, Microsoft 365, Windows, and enterprise workloads it can substantially reduce token volume, latency, infrastructure load, and energy usage. The key validation question is not whether cost can be reduced by shortening responses, but whether the governor can reduce avoidable compute while preserving or improving task success, user trust, and perceived usefulness.

10.7 Strategic Relevance for Microsoft

This architecture aligns with Microsoft’s strategic priorities:

- **Scalable AI** that adapts to user preference
- **Sustainable compute** that reduces energy consumption
- **Transparent UX** that builds trust
- **Differentiated onboarding** that could distinguish user-controlled AI personalization
- **Agentic control** that prevents runaway tool usage

The radial identity map is not just a visualization — it is a **cost-aware cognitive interface** for the underlying radial identity membrane.

11. Computational Simulation Note

While this paper focuses on the conceptual architecture of the twelve-string radial identity membrane, the structure has now been exercised through a lightweight companion toy simulation in the *Project Rainbow* notebook. The simulation is not a production implementation or empirical deployment result; rather, it is a reproducible numerical testbed for checking whether the proposed state variables, coupling rules, governor dynamics, and stability metrics behave coherently under simplified synthetic workloads. Each behavioral string is represented as a time-varying state vector containing amplitude, reasoning depth, dynamic tension, and dynamic stiffness:

$$x_i(t) = (A_i(t), R_i(t), \tau_i(t), \kappa_i(t))$$

where $x_i(t)$ is the simulated state of string i at time t , collecting activation, depth, tension, and stiffness into one updateable vector.

In the Project Rainbow toy simulation, curvature-weighted coupling K_{ij} enables activation flow across neighboring regions of the membrane, while local nonlinear cost functions determine which channels are energetically attractive. A Lyapunov-style global energy functional $V(t)$ provides the stability metric used to observe whether the system contracts toward lower energy during ordinary updates and expands only under explicit task pressure.

The damped wave equation in Section 1 provides the dynamic backbone for coefficient-space or finite-difference simulation over the circular phase domain, enabling direct numerical tests of stability and routing.

The simulation should be interpreted as toy-model evidence of internal coherence rather than as proof of production behavior. It provides an initial empirical exercise of the architecture’s mechanics: initialization, task-driven activation, V-channel routing, inverse-cost resistance, adaptive tension and stiffness, governor suppression, Lyapunov-style energy tracking, ablation comparison, and return-to-baseline behavior.

This formulation allowed the notebook to construct a minimal dynamical simulation using standard numerical tools. The completed toy run explores activation propagation, V-channel selection, tension and stiffness adaptation, local cost growth, suppression, and task-driven expansion. The architecture remains modular so future work can extend the notebook with stochasticity, broader task sequences, parameter sweeps, user-preference drift, multi-task switching, or multi-membrane interaction models.

Appendix E documents the notebook as the reproducibility and implementation scaffold, while Appendix F summarizes the resulting toy-simulation outputs, including activation heatmaps, reasoning-depth traces, cost and energy plots, ablation comparisons, V-channel coherence proxies, and tension/stiffness relaxation traces. The validation roadmap in Section 9 remains the path for moving beyond toy simulation toward controlled prototype testing and eventual production-relevant evaluation.

11.1 Summary of the Simulation Approach

The Project Rainbow toy simulation treats the twelve behavioral strings as coupled dynamical variables evolving over discrete time steps. Each task or prompt is encoded as a behavioral activation pattern across the strings. The model then applies curvature-weighted propagation through K_{ij} , reduces propagation through inverse-cost routing, updates amplitude and reasoning depth, adapts tension and stiffness according to cost and usage, and computes the resulting Lyapunov-style energy $V(t)$.

The first completed toy run is comparative rather than exhaustive. It tests whether the adaptive membrane produces bounded activation, coherent V-channel routing, lower runaway risk, and faster return-to-baseline behavior than simpler baselines such as fixed tension/stiffness, no-curvature routing, or no-governor control. The Appendix F results show that task-aligned behaviors activate as expected, neighboring strings propagate through meaningful corridors, high-cost expansion remains governed, and $V(t)$ remains interpretable as a stability signal.

11.2 Supporting Simulation Details

- **State representation:** maintain arrays for A_i , R_i , τ_i , κ_i , and C_i across the twelve strings.
- **Coupling matrix:** define K_{ij} as a circular curvature matrix so adjacent strings couple more strongly than distant strings.
- **Input tasks:** encode synthetic workloads as behavioral activation patterns, such as deep technical reasoning, warm concise response, context-heavy synthesis, or agentic planning.
- **Update loop:** apply data excitation, curvature propagation, inverse-cost weighting, suppression, adaptive tension and stiffness, and local cost recomputation at each time step.
- **Stability tracking:** compute $V(t)$ over time and verify whether $\Delta V \leq 0$ during ordinary updates, with controlled increases only under explicit task pressure.
- **Comparisons:** evaluate adaptive membranes against fixed tension/stiffness, no-curvature routing, no-governor control, and response-length-only baselines.
 - **Outputs:** visualize string activation, reasoning depth, total cost, tension, stiffness, V-channel selection, and Lyapunov energy over time.

Together, these simulation details describe the completed lightweight testbed used in Appendices E and F. The current simulation supports hypothesis generation, architecture inspection, and ablation comparison; it does not yet establish real-world compute savings, user-quality improvements, or production-scale deployment behavior. Those claims require the staged validation program described in Section 9.

12. Engineering Integration Layer

The radial identity membrane can be implemented first as a lightweight inference-time orchestration layer rather than as a replacement for the base model or a retraining requirement. In this role, the membrane estimates the user- and task-specific behavioral state, while the governor converts that state into runtime controls such as response length, reasoning depth, context budget, retrieval depth, tool-call threshold, explanation structure, and tone.

12.1 Runtime Integration Flow

1. A user request enters the system.
2. The request is mapped to a behavioral activation pattern across the twelve strings.
3. The current membrane state estimates preferred depth, context sensitivity, initiative, conciseness, tone, transparency, and technical detail.
4. The governor applies cost, safety, task-value, and privacy constraints.
5. The orchestration layer selects runtime settings such as token budget, retrieval budget, tool-call threshold, response structure, and explanation level.
6. After the response, privacy-preserving telemetry updates the coefficient vector, tension, stiffness, and curvature estimates according to the warm-start and validation rules.

12.2 Runtime Control Outputs

Membrane Signal	Runtime Control Output
High Depth	Increase reasoning budget, planning depth, or intermediate verification.
High Precision	Increase claim checking, retrieval support, or qualification strictness.
High Context Sensitivity	Expand retrieval scope or context-window allocation when justified.
High Initiative	Raise planning activity or tool/action proposal likelihood within guardrails.
High Exploration	Allow broader option generation or alternative-path analysis.
High Creativity	Permit more generative variation, synthesis, or expressive construction.
High Tone or Warmth	Adjust interpersonal style without substantially increasing reasoning depth.
High Transparency	Increase explanation length, structure, and visible rationale.
High Technical Detail	Increase domain terminology, formulas, evidence density, and structured explanation.

High Structural Rigor	Increase outlining, hierarchy, consistency checks, and response scaffolding.
High Conciseness	Reduce max tokens, compress explanations, and suppress unnecessary elaboration.
High Global Cost	Apply suppression to optional behaviors, reduce context/tool budgets, and preserve essential safety and quality floors.

12.3 Non-Invasive Deployment Path

The first deployable version does not require modifying base model weights. It can operate as an inference-time controller that adjusts prompts, retrieval budgets, context selection, verbosity limits, tool-call thresholds, and response-formatting policies. This allows the architecture to be tested through simulation, orchestration, and controlled experiments before any deeper model-training integration is considered.

12.4 Runtime Failure Handling

- **Low confidence:** fall back to default assistant behavior or low-gain personalization.
- **Cost spike:** tighten optional strings first, reduce retrieval/tool budgets, and preserve essential safety and quality floors.
- **User correction increases:** reduce personalization gain and invite recalibration.
- **Task value is high:** permit bounded compute expansion through the task-value guardrails in Section 7.5.1.
- **Policy or privacy boundary is reached:** stop personalization updates and operate within tenant or user-defined controls.

This engineering layer translates the theoretical membrane into practical system controls. It allows the architecture to add value incrementally: first as a measurable controller over existing model orchestration, then as a possible training or personalization substrate if validation supports deeper integration.

13. Policy as a Runtime Constraint Surface

Although policy is referenced throughout this paper as part of safety, privacy, and enterprise governance, U.F.O. gives policy a more specific architectural role: policy becomes part of the runtime constraint surface through which user-formed optimization is allowed to move. In the phase-efficient governor pass, policy does not merely review behavior after the fact; it helps define the set of valid next states, S_{safe} , before optimization occurs.

This means that user preference, task value, compute efficiency, and personalization are always evaluated inside policy-bounded space. A transition may be low-cost, coherent, and strongly aligned with the user’s apparent intent, but still be rejected or narrowed if it violates privacy boundaries, tenant policy, data residency rules, tool-use permissions, accessibility floors, safety constraints, or administrator-defined controls. Policy therefore shapes the geometry of possible adaptation rather than acting only as an external filter.

In a Microsoft-scale operating layer, this policy surface could vary by context: a Word drafting task, a Teams recap, an Outlook reply, a settings action, a file operation, or an enterprise workflow may each admit different levels of retrieval, tool use, personalization, and explanation. The U.F.O. governor can treat these differences as runtime constraints on transition selection, ensuring that the membrane remains steerable by the user while still bounded by organizational and platform requirements.

The most important implication is that policy becomes measurable. Instead of asking only whether a final response complied with policy, the system can ask which candidate transitions were removed, which controls were tightened, which context sources were excluded, which personalization updates were blocked, and which runtime knobs were reduced. This creates a stronger audit and debugging posture: policy can be inspected as part of the control path, not only as a final moderation outcome.

For this paper, policy remains a high-level constraint category rather than a fully specified policy language. Future work could formalize policy as a typed boundary layer for the membrane, integrated with Elder-style orchestration, tenant controls, permission graphs, and residual-aware tracing. The goal would not be to replace existing compliance systems, but to make policy's role in adaptive AI behavior explicit, measurable, and compatible with governed user-shaped optimization.

14. Conclusion

This paper presents a radial waveform architecture for AI identity that unifies behavioral personalization, compute allocation, and stability control within a single geometric framework. By representing AI behavior as a circular resonant membrane composed of twelve behavioral strings, the model moves beyond static personality presets and instead treats identity as a dynamic, tunable, compute-bearing structure. Each string contributes not only to expression, but also to reasoning depth, context usage, tool behavior, training flow, and energetic cost.

The central contribution is the integration of geometry, vibration, and governance. The membrane and wave equation define the identity space and its evolution; the vibrational parameters define how each behavioral string activates, resists deformation, and returns to baseline; the adaptive tension and stiffness rules allow the system to self-regulate over time; and the latent training plane explains how data can reshape durable adaptation through curved corridors of aligned traits. Together, these layers provide a coherent mechanism for intelligence that flows through structured channels rather than switching abruptly between isolated modes.

The governor gives this architecture its stabilizing core. By combining inverse-cost routing, bounded depth updates, suppression dynamics, adaptive membrane resistance, and a candidate Lyapunov energy, the model establishes a plausible control framework for keeping AI behavior expressive without allowing compute to expand uncontrollably. The system can spend additional compute when task value justifies it, but its default tendency is toward lower energy, reduced radial imbalance, and efficient use of behavioral capacity.

Practically, the architecture suggests a path toward resource-aware personalization. User preferences do not merely alter tone or style; they shape the geometry of the AI’s operating profile, determining which behavioral strings are easy to excite, which V-channels are preferred, and where compute should be conserved. This creates an identity system that is personal, transparent, and economically meaningful: the AI can adapt to users while also reducing unnecessary reasoning, excessive context expansion, long outputs, and avoidable tool use.

The validation roadmap in Section 9 provides the proposed path for moving beyond toy simulation toward controlled prototype testing, while Section 10 describes how the same architecture could be exposed as a practical radial identity map for user calibration and compute-aware personalization. Section 11 and Appendices E–F document the current simulation status: the *Project Rainbow* notebook implements a lightweight, reproducible toy simulation that exercises the membrane’s state arrays, task signatures, curvature matrix, update loop, ablation configurations, and visualization routines.

The framework should therefore be understood as a conceptual and mathematical architecture supported by an initial toy-model simulation, not as a production implementation or empirical deployment result. The completed simulation provides evidence that the proposed mechanics are internally testable: task signatures activate expected behavioral sectors, curvature supports coherent V-channel routing, the governor limits runaway expansion, adaptive tension and stiffness support return-to-baseline behavior, and Lyapunov-style energy remains interpretable as a stability signal. These results strengthen the architecture as a research candidate while clarifying its limits. The next step is to extend the simulation, run broader parameter sweeps, test richer workloads, and build controlled prototypes that can measure corridor selection, cost reduction, depth preservation, suppression, convergence, and task-sensitive expansion against real evaluation baselines. If validated through that staged program, the radial identity membrane could provide a formal foundation for personalized AI systems that are more aligned with individual users, computationally efficient, and stable at scale.

CODA: A Layered Horizon for Governed Adaptation

At the broadest horizon, the radial identity membrane points toward a future in which AI systems are not defined by personality, but by *steerability*. As AI becomes a native part of the computing environment, the central challenge shifts from building more capable models to building systems that can express those capabilities safely, efficiently, and in ways that remain aligned with the user’s intent. The membrane is one proposal for how an AI operating layer might achieve this: not by adding complexity to the interface, but by giving the system a more graceful internal geometry for interpreting and regulating behavior.

From this high-level view, personalization becomes less about settings and more about *interaction geometry*. The system learns how a user moves through work—how they shift from drafting to reviewing, from summarizing to critiquing, from planning to troubleshooting. These transitions should feel ordinary to the user, yet they represent distinct movements across the membrane: depth expands or contracts, context widens or narrows, tool use becomes available or suppressed, and the governor decides which capabilities should surface and which should remain

quiet. The visible interface stays calm while the operating layer performs continuous, bounded orchestration underneath.

As we move closer to the mechanism, the membrane reframes a core systems question: **What is the primary object of design in an AI operating layer?** It may no longer be the model, the prompt, the tool, or the memory store. Instead, it may be the *adaptive control surface* that coordinates all of them at runtime. Such a surface must decide how much context to admit, how much compute to spend, when to retrieve, when to act, when to explain, when to stay concise, and when to contract back toward a stable baseline. The membrane offers one vocabulary for expressing this surface as a governed dynamical system.

At a more detailed level, this suggests a new class of systems abstractions: compute-aware behavioral controllers, policy-bounded inference surfaces, short-horizon geometric caches, adaptation ledgers, and user-calibrated orchestration layers. These abstractions do not replace machine learning, operating systems, programming languages, human–computer interaction, or responsible AI. They are a convergence point between them—a way to unify behavior, compute, context, memory-like traces, and policy into a single inspectable structure.

The boundary-loop trace illustrates this convergence. As the membrane expands under task pressure and relaxes inward, it leaves a fading geometric imprint of recent depth, cost, tension, and stiffness. This imprint is not content memory and not durable learning; it is a short-horizon computational wake. If validated, such geometric caching would show that adaptive systems can cache not only data or results, but temporary *control geometry*: the recent shape of how work was done. This could make the next aligned input easier to route while allowing unrelated or unstable pressure to fade.

For the user, the best version of this future may feel almost invisible. A request that begins in Word may draw on a calendar event, a Teams discussion, a recent file, and an enterprise policy boundary without requiring the user to assemble that context. A troubleshooting request may stay local and offer direct steps. A technical review may expand depth and precision, then contract once the work is done. A coaching response may adjust tone and clarity without invoking unnecessary reasoning. The elegance lies in making these shifts feel natural at the surface while keeping the orchestration measurable underneath.

At the engineering level, the membrane becomes a testable orchestration problem. A prototype could sit between intent detection and model execution, translating user requests, app context, policy boundaries, and telemetry into bounded runtime controls such as token budget, retrieval scope, tool-call threshold, reasoning depth, explanation level, and return-to-baseline behavior. Project Rainbow should be extended into broader parameter sweeps, richer synthetic workloads, and controlled prototypes that evaluate whether the membrane remains stable, cost-aware, and task-aligned across varied conditions. A key assumption to test is whether the boundary-loop trace behaves like short-horizon geometric caching: after coherent expansion and inward relaxation, the decaying imprint $B(\theta, t)$ may reduce routing friction for the next aligned input without becoming persistent memory. This can be evaluated by comparing aligned task sequences, shuffled sequences, and no-trace ablations against cost, latency, route coherence, and return-to-baseline metrics.

At the most speculative edge of this work, the membrane also invites a broader systems question: whether coherent excitation patterns, when sufficiently stable, self-referential, and governable, can become a general design language for adaptive intelligence. In U.F.O., the claim is not that the system is conscious, but that structured computational disturbances can move outward, shape the system while being shaped by it, and leave measurable traces that influence future behavior. In this respect, U.F.O. becomes meaningful to study not only as a governor, but as a possible control framework for accelerating adaptive machine-learning behavior, improving AI capability expression, and making those improvements more measurable, bounded, and governable.

At the deepest layer, the membrane points toward a broader horizon: a user-centered computer science of governed adaptation. Such a field would study how adaptive systems expose capability through controllable interfaces; how runtime behavior, compute, context, policy, and memory-like traces can be measured together; and how users can steer increasingly powerful models without surrendering transparency, efficiency, or control. The question is not only how to build stronger AI models, but how to build interfaces and operating layers that let people steer those models safely, efficiently, and elegantly. If validated, the radial identity membrane could be one step toward that future: a geometry for stable adaptation, a control surface for personalized intelligence, and a foundation for AI systems that become more useful without becoming less governable.

Appendices Overview

- **Appendix A** — mathematical design rationale and assumptions.
- **Appendix B** — minimal simulation specification.
- **Appendix C** — worked toy simulation example.
- **Appendix D** — runtime control outputs and deployment path.
- **Appendix E** — Project Rainbow notebook companion and reproducibility scaffold.
- **Appendix F** — toy-simulation outputs and stability observations.
- **Appendix G** — Elder reference architecture and Windows Copilot integration sketch.
- **Appendix H** — policy-bounded state selection and Elder-style S_{safe} interpretation.
- **Appendix I: Kernel-like Propagation Structure in V-Channels**

Appendix A: Mathematical Design Rationale

This appendix clarifies the status of the paper’s equations. The mathematical forms are proposed as architectural design equations rather than closed-form derivations from first principles. Their purpose is to define a coherent control surface that can be simulated, ablated, and revised as empirical evidence accumulates.

A.1 Design Equation Criteria

The equations are evaluated by whether they are bounded, monotonic where needed, interpretable, tunable, simulation-tractable, and compatible with observable runtime signals. A useful design equation should make a control assumption explicit while remaining simple enough to ablate, replace, or fit empirically.

A.2 Why These Functions?

The selected functions are chosen for stability, monotonicity, and interpretability. Squared depth terms model the disproportionate cost of deeper reasoning; inverse-cost terms make expensive paths less attractive; sigmoid and cosine terms bound attraction and phase alignment; logarithmic context terms model diminishing returns from expanding context; and suppression functions contract inactive regions as global cost rises. These choices are intentionally simple so their behavior can be inspected before more complex learned functions are introduced.

These functions should be interpreted as first-pass functional forms; future simulations may replace them with learned or empirically fitted alternatives if those alternatives preserve monotonicity, boundedness, interpretability, and measurable control behavior.

A.3 Why These Weights?

The weighting parameters are not fixed constants of nature. They are tuning knobs that determine how strongly the governor penalizes reasoning depth, amplitude, context use, tool use, tension, stiffness, efficiency pressure, and task-driven expansion. In simulation, these weights should begin as normalized conservative ranges and be varied through parameter sweeps. In prototype systems, they should be calibrated against observable signals such as tokens, latency, retrievals, tool calls, correction rate, and user satisfaction.

A.4 Why Twelve Strings?

The twelve-string basis is a proposed design resolution rather than a claim of biological or mathematical necessity. Twelve is large enough to separate major behavioral dimensions such as depth, precision, context sensitivity, initiative, exploration, creativity, tone, transparency, technical detail, structural rigor, warmth, and conciseness, while remaining small enough to visualize, simulate, and expose through a user-facing radial map. Future work could test smaller or larger bases and compare whether they improve coherence, usability, or cost control. A useful ablation would compare 6-, 8-, 12-, and 16-string bases against coherence, user interpretability, routing stability, and cost-control metrics.

A.5 Parameter Ranges and Calibration

Initial parameter ranges should be treated as simulation priors. A practical workflow is to normalize all state variables to comparable scales, sweep conservative ranges, identify regimes where cost and energy remain bounded, and then compare the full model against ablations. Useful calibration targets include reduced cost variance, preserved task-aligned activation, bounded latency overhead, stable Lyapunov energy, and improved or unchanged user-quality metrics.

A.6 Edge Cases and Failure Behavior

The main edge cases are over-suppression, runaway task expansion, incorrect curvature coupling, and early personalization overfit. The architecture addresses these with soft floors on essential capabilities, bounded task-value expansion, rolling budgets, confidence thresholds, warm-start personalization, and ablation testing. If these safeguards fail in simulation, the appropriate response is not to treat the model as invalid, but to adjust functions, weights, or coupling assumptions before prototype deployment.

If reasonable parameter sweeps cannot produce stable routing, bounded energy, or task-aligned activation, the architecture should be revised rather than assumed valid.

Appendix B: Minimal Simulation Specification

This appendix provides a minimal simulation sketch for reproducing the core membrane dynamics at a toy-model level. The goal is not to prescribe a production implementation, but to make the proposed architecture computationally testable.

B.1 Minimal State Update Pseudocode

1. Initialize arrays A , R , τ , κ , and C for the twelve behavioral strings.
2. Encode each task as a behavioral activation pattern over the strings.
3. Compute local cost C_i from R_i , A_i , τ_i , κ_i , tool use, and context load.
4. Compute inverse-cost routing $I_i = 1/(1 + C_i)$.
5. Propagate activation through curvature matrix K_{ij} and phase-aligned V -channels.
6. Update amplitude A_i and reasoning depth R_i under the radius bound $R_{\max}$.
7. Adapt tension τ_i and stiffness κ_i using local cost, global cost, usage, and variance signals.
8. Apply suppression to inactive or high-cost regions.
9. Compute Lyapunov energy $V(t)$ and track whether $\Delta V \leq 0$ during ordinary updates.
10. Compare against baselines such as fixed τ/κ , no-curvature routing, and no-governor control.

B.2 Suggested Default Parameter Ranges

Parameter	Role	Suggested Initial Range
λ_R	Weight on reasoning depth cost R_i^2 .	0.5–2.0
λ_A	Weight on behavioral amplitude cost A_i .	0.1–1.0
λ_T	Weight on discrete tool-use cost T_i .	0.5–3.0
λ_C	Weight on context expansion cost $\log(1 + \text{context}_i)$.	0.1–1.5
λ_τ	Weight on dynamic tension as a cost contributor.	0.05–0.5

λ_{κ}	Weight on dynamic stiffness as a cost contributor.	0.05–0.5
α	Lyapunov weight on radial depth variance.	0.1–1.0
β	Lyapunov weight on total membrane tension.	0.05–0.5
δ	Lyapunov weight on total membrane stiffness.	0.05–0.5
$R_{\max}$	Maximum allowed reasoning radius.	1.0 normalized, or task-scaled 1.0–3.0
α_{τ}	Tension adaptation rate.	0.01–0.10
α_{κ}	Stiffness adaptation rate.	0.01–0.10
η	Efficiency gain weight in soft global cost behavior.	0.05–0.5
γ^{task}	Task-value weight permitting justified cost expansion.	0.05–1.0

These ranges are intentionally conservative and normalized. Early simulations should sweep across ranges rather than assume a single optimal setting, then compare stability, routing quality, and cost behavior across the baselines defined in Section 9.7.

Appendix C: Worked Toy Simulation Example

This appendix gives a compact worked example of how the simulation framework could be exercised with a single synthetic workload. The example is illustrative rather than empirical: it describes the expected state transitions that a toy simulation should test, not measured results from an implemented system.

C.1 Example Workload: Multi-Step Technical Proof

A representative synthetic task is a request for a multi-step technical proof or architecture critique. Such a task should primarily excite Depth, Precision, Technical Detail, and Structural Rigor, with secondary activation in Transparency and Context Sensitivity. Low-cost expressive strings such as Tone and Emotional Warmth should remain available but should not dominate the route.

$$d_{\text{proof}} = [\text{Depth, Precision, Context, Initiative, Exploration, Creativity, Tone, Transparency, TechnicalDetail, Rigor, Warmth, Conciseness}]$$

At initialization, the task vector assigns high weights to Depth, Precision, Technical Detail, and Structural Rigor; moderate weights to Transparency and Context Sensitivity; and low weights to Tone, Warmth, Creativity, and Initiative. This produces an initial activation pattern concentrated in the technical-reasoning corridor.

C.2 Expected Update Sequence

1. **Data excitation:** the task injects amplitude into the technical-reasoning strings.
2. **Curvature propagation:** activation spreads through the Depth–Precision–Technical Detail–Structural Rigor V-channel because these strings are phase-near and semantically aligned.
3. **Inverse-cost routing:** high-cost depth and detail are moderated by $I_i = 1/(1 + C_i)$, preventing unlimited outward expansion.
4. **Depth update:** R_i increases most strongly in the aligned corridor while unrelated strings remain near baseline.
5. **Tension and stiffness adaptation:** repeated use of Depth and Technical Detail increases τ_i and κ_i , making further expansion more cautious.
6. **Suppression:** low-relevance strings contract slightly, conserving amplitude and radius for the active corridor.
7. **Energy accounting:** $V(t)$ may increase during the initial task-driven expansion, then should stabilize or decline once the task enters ordinary update mode.

C.3 Expected Observables

Observable	Expected Pattern
String activation	Depth, Precision, Technical Detail, and Structural Rigor show the strongest amplitudes.
V-channel route	The dominant route follows the technical-reasoning corridor rather than distant expressive strings.
Reasoning depth	R_i expands primarily in aligned high-value strings and remains bounded by $R_{\max}$.
Local cost	C_i rises in high-depth strings but is moderated by inverse-cost routing and adaptive tension.
Tension and stiffness	τ_i and κ_i rise for repeatedly used high-cost strings, reducing runaway growth.
Lyapunov energy	$V(t)$ may rise under explicit task pressure, then should stabilize or decrease during ordinary updates.

C.4 Interpretation

A successful toy run would not prove the full architecture, but it would provide evidence that the proposed mechanics behave coherently: task-aligned strings activate together, curvature routes activation through meaningful corridors, inverse-cost pressure prevents uncontrolled depth expansion, and adaptive tension/stiffness improve return-to-baseline behavior. This kind of worked simulation would be the first bridge between the conceptual membrane and an empirical prototype.

Appendix D: Runtime Control Outputs and Deployment Path

This appendix summarizes the engineering-facing control surface of the radial identity membrane. It is intended as a quick reference for how internal membrane signals could be translated into runtime orchestration settings without modifying base model weights.

D.1 Runtime Control Outputs

Membrane Signal	Runtime Control Output
High Depth	Increase reasoning budget, planning depth, or intermediate verification.
High Precision	Increase claim checking, retrieval support, or qualification strictness.
High Context Sensitivity	Expand retrieval scope or context-window allocation when justified.
High Initiative	Raise planning activity or tool/action proposal likelihood within guardrails.
High Exploration	Allow broader option generation or alternative-path analysis.
High Creativity	Permit more generative variation, synthesis, or expressive construction.
High Tone or Warmth	Adjust interpersonal style without substantially increasing reasoning depth.
High Transparency	Increase explanation length, structure, and visible rationale.
High Technical Detail	Increase domain terminology, formulas, evidence density, and structured explanation.
High Structural Rigor	Increase outlining, hierarchy, consistency checks, and response scaffolding.
High Conciseness	Reduce max tokens, compress explanations, and suppress unnecessary elaboration.
High Global Cost	Apply suppression to optional behaviors, reduce context and tool budgets, and preserve essential safety and quality floors.

D.2 Non-Invasive Deployment Path

The first deployable version of the architecture does not require modifying base model weights. It can operate as a lightweight inference-time controller that adjusts prompts, retrieval budgets, context selection, verbosity limits, tool-call thresholds, response formatting policies, and explanation structure. This allows the membrane to be tested through simulation, orchestration, and controlled experiments before any deeper model-training integration is considered.

In a production stack, the controller can sit between request interpretation and model execution. It receives the user request, estimates the behavioral activation pattern, applies the governor's cost and safety constraints, selects runtime settings, and updates the membrane state from privacy-preserving telemetry after the response. This makes the architecture incrementally

deployable: it can first improve orchestration and measurement, then later support deeper personalization or training-plane integration if validation results justify it.

Appendix E: Project Rainbow Notebook Companion

This appendix documents the companion Jupyter Notebook, *Project Rainbow*, used as a lightweight, reproducible toy simulation of the radial identity membrane. The notebook implements the state arrays, task signatures, curvature matrix, update loop, ablation configurations, and visualization routines used to generate the simulation outputs summarized in Appendix F. These results should be interpreted as toy-model evidence of internal coherence, routing behavior, and stability dynamics, not as empirical validation of production AI behavior.

E.1 Notebook Structure

Notebook Cell	Purpose
Imports and string definitions	Loads numerical and visualization libraries and defines the twelve behavioral strings.
Membrane state initialization	Initializes amplitude, reasoning depth, tension, stiffness, baseline cost, and radius bounds.
Curvature matrix construction	Builds a circular coupling matrix K_{ij} so neighboring strings propagate more strongly than distant strings.
Cost and energy functions	Defines local cost and Lyapunov-style energy for tracking stability and compute pressure.
Task activation patterns	Creates synthetic workloads such as deep technical reasoning, warm concise response, and agentic planning.
Membrane update rule	Applies data excitation, curvature propagation, inverse-cost routing, suppression, tension/stiffness adaptation, and cost recomputation.
Simulation run	Executes the discrete-time simulation across task phases and records activation, depth, cost, and energy histories.
Behavioral activation heatmap	Visualizes how the twelve strings activate over time, making V-channel routing visible.
Lyapunov energy trace	Plots $V(t)$ to inspect whether ordinary updates stabilize or contract after task-driven expansion.

E.2 Expected Notebook Outputs

- **Behavioral activation heatmap:** shows which strings activate during each task phase and whether activation concentrates in meaningful V-channels.
- **Reasoning-depth profile:** shows whether R_i expands in task-aligned strings while remaining bounded by $R_{\max}$.

- **Total cost trace:** shows whether local and global cost remain controlled under changing workloads.
- **Tension and stiffness traces:** show whether overused or expensive strings become more resistant over time.
- **Lyapunov energy trace:** shows whether $V(t)$ stabilizes or decreases during ordinary updates, with increases limited to explicit task-pressure periods.

The notebook produces the main chart families reported in Appendix F, including behavioral activation heatmaps, reasoning-depth traces, compute-cost and Lyapunov-style energy plots, ablation comparisons, V-channel coherence proxies, and tension/stiffness relaxation traces. Appendix E therefore describes the reproducibility and implementation scaffold, while Appendix F reports the resulting toy-simulation observations.

The companion notebook gives reviewers and researchers a concrete way to reproduce the toy dynamics summarized in Appendix F without embedding implementation code in the paper itself. In this role, Project Rainbow functions as a reproducibility bridge: it preserves the paper’s formal structure while making the architecture testable through a lightweight numerical experiment that can be inspected, rerun, ablated, and extended.

E.3 Reproducibility Checklist

- Use the same twelve-string ordering throughout the notebook, tables, activation arrays, and plots. For example, if index 0 is Depth in the task vector, index 0 must remain Depth in the heatmap and cost traces.
- Record the simulation date, notebook version, parameter set, and task sequence for each run. A simple run label such as “adaptive_full_deep_warm_agentic_v1” is sufficient for early experiments.
- Normalize activation, radius, tension, stiffness, and cost values before comparing runs or visualizations. For example, keep A_i and R_i in a normalized 0–1 range when testing the toy model.
- Save the random seed if stochastic task generation, noise, sampling, or randomized parameter sweeps are introduced, so unusual routing or stability behavior can be reproduced.
- Report all parameter values used for each simulation run, including λ weights, α , β , δ , $R_{\max}$, τ and κ adaptation rates, η , and γ_{task} .
- State whether the run uses fixed parameters, adaptive tension/stiffness, no-curvature routing, no-governor control, response-length-only control, or the full membrane model.
- Plot activation, reasoning depth, total cost, local cost distribution, tension, stiffness, and Lyapunov energy for every run.
- Compare at least one adaptive membrane run against a fixed-parameter baseline and one no-curvature baseline.
- Track ΔV separately during ordinary updates and explicit task-pressure periods. This helps distinguish healthy task-driven expansion from uncontrolled energy growth.
- Preserve the task labels used for each time window so activation changes can be interpreted against workload changes, such as deep technical, warm concise, or agentic phases.

- Report whether observed cost reductions coincide with preserved task-aligned activation rather than simple suppression. Lower cost is only useful if the relevant strings still activate.
- Treat results as toy-model evidence, not empirical validation of production behavior. The notebook should be read as a reproducibility bridge and hypothesis generator.

E.4 Suggested First Experiments

- **Experiment 1: Adaptive vs. fixed membrane.** Compare adaptive τ and κ updates against fixed values. Measure total cost, $V(t)$, return-to-baseline time, and runaway activation frequency.
- **Experiment 2: Curvature vs. no-curvature routing.** Remove K_{ij} coupling and test whether activation becomes less coherent, less corridor-like, or more isolated across strings.
- **Experiment 3: Task-pressure stress test.** Alternate deep technical, warm concise, and agentic task phases. Verify that ΔV rises mainly during explicit task pressure and contracts afterward.
- **Experiment 4: Conciseness as governor brake.** Increase Conciseness activation and test whether token-like cost decreases without suppressing task-relevant depth, precision, or technical detail.
- **Experiment 5: Early personalization warm-start.** Simulate low-gain early updates for the first N interactions, then compare against immediate full-strength personalization to test overfitting and stability.

These experiments are intentionally minimal. Their purpose is not to validate production behavior, but to test whether the model’s core mechanics—curvature routing, inverse-cost resistance, adaptive tension and stiffness, warm-start personalization, ablation behavior, and Lyapunov-style stabilization—produce coherent dynamics before more expensive prototype work begins. The Appendix F results should therefore be read as toy-simulation evidence that the proposed control mechanics are internally testable and directionally coherent, not as proof that the architecture will generalize to real user-facing systems without further validation.

E.5 Interpreting the Completed Toy Run and Future Runs

The completed Project Rainbow toy run provides an initial coherence check for the architecture under simplified synthetic workloads. The run shows directional alignment between task type, activated strings, routing behavior, cost pressure, and energy dynamics. Deep technical signatures concentrate activation in Depth, Precision, Technical Detail, and Structural Rigor; warm-concise signatures shift activation toward Tone, Emotional Warmth, and Conciseness; and agentic signatures activate Initiative without allowing tool-like behavior to expand without task pressure. These patterns are reported in Appendix F and should be interpreted as toy-model evidence of internal coherence rather than production validation.

The most important signal is not that cost always decreases, but that cost behaves intelligibly and remains governed. In the completed toy run, $V(t)$ can rise during task-driven expansion, while ordinary low-pressure behavior remains bounded and compute-efficient. High-cost strings become more resistant through tension and stiffness rather than expanding indefinitely, and the governor prevents runaway behavior in ablation comparisons. The completed Project Rainbow

run therefore passes an initial toy-model coherence test, while future runs should determine whether the same behaviors persist across broader parameter sweeps, richer workload families, stricter baselines, and controlled prototype evaluations.

Appendix F — Data-Driven Initialization, Activation, and Stability Analysis

Appendix F summarizes the initial Project Rainbow toy-simulation outputs from simulations run locally in Jupyter Notebook. The results should be read as evidence of internal coherence, routing behavior, and simulation tractability, not as production validation. At initialization, the radial identity membrane begins in a neutral, symmetric, low-energy state: all amplitudes and radii are zero, tension and stiffness are uniformly initialized, and no V-channel is favored before task excitation. This initialization structure is visualized in the curvature-matrix captions below.

The simulation then applies synthetic task signatures to test whether activation concentrates in expected behavioral strings. Technical, warm-concise, agentic, creative, stress, and ordinary low-pressure workloads each produce distinct excitation patterns, allowing the notebook to inspect whether routing follows meaningful V-channel corridors rather than spreading indiscriminately across the membrane. The task-activation heatmap and behavioral-activation captions show these string-specific excitation patterns.

Early-time traces show bounded activation under ordinary low-pressure conditions. The membrane remains shallow, low-cost, and close to baseline when the task does not justify deeper reasoning, while the governor preserves stability without applying aggressive contraction. The activation, reasoning-depth, and C_{total}/V captions provide the corresponding low-pressure values.

Ablation runs clarify the functional role of each architectural component. Curvature supports coherent routing, adaptive tension and stiffness regulate membrane resistance, and the governor prevents runaway expansion. The no-governor condition is the clearest failure case, while the full adaptive configuration remains bounded and task-sensitive. The ablation comparison captions report the cost and energy differences across these configurations.

Channel-coherence metrics indicate that activation concentrates in appropriate V-channels rather than scattering across unrelated strings. In low-pressure conditions, the warm-concise pathway remains dominant, while other channels activate in proportion to task structure. The $Q(t)$ caption reports the coherence values supporting this interpretation.

The full adaptive relaxation runs provide the strongest combined evidence of coherent routing, compute-aware restraint, and rapid recovery after stress excitation. These runs support the claim that the proposed mechanics can be parameterized, visualized, ablated, and interpreted within a lightweight simulation environment. The tension/stiffness relaxation and ΔV captions show the recovery and stabilization behavior most directly.

Taken together, Appendix F supports the architecture’s central simulation claim: the radial identity membrane is not only a conceptual model, but also a measurable toy control system whose initialization, activation, routing, ablation behavior, coherence, and recovery dynamics can be inspected before prototype-level validation.

The remainder of this Appendix F research artifact presents the supporting figures and captions that document those toy-simulation observations.

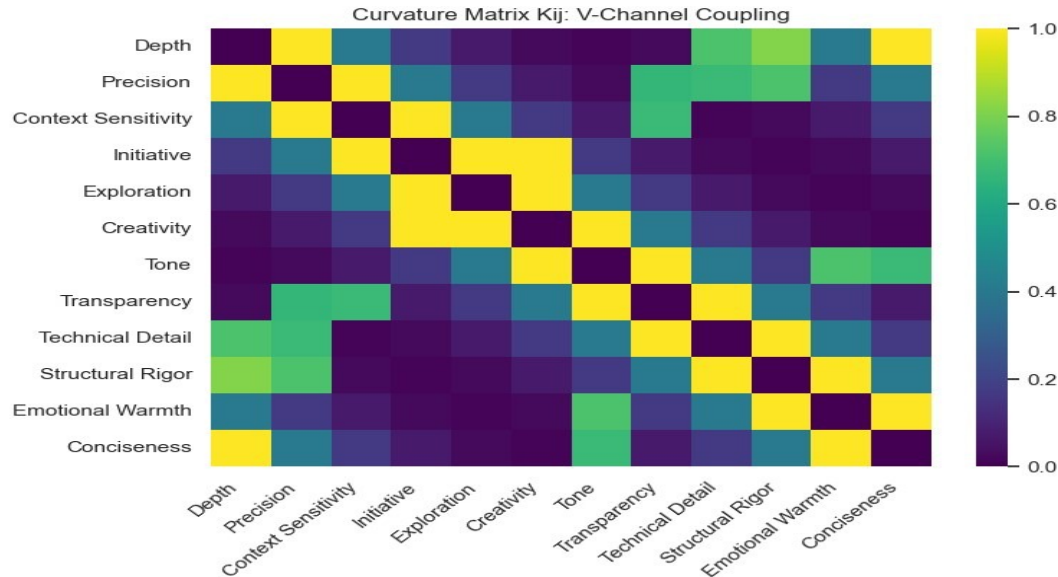


Figure F.1. Curvature Matrix and V-Channel Coupling Structure

Curvature matrix K_{ij} for the radial identity membrane. The twelve behavioral eigenmodes are evenly spaced at $\pi/6$ intervals around the identity circle, beginning with Depth at $\theta = 0.000000$ radians and ending with Conciseness at $\theta = 5.759587$ radians. The diagonal shows maximal self-coupling, while the graded off-diagonal bands show stronger coupling among related modes, including Depth–Precision, Exploration–Creativity, and Tone–Emotional Warmth. These coupling patterns define the V-channel structure used for coherent activation flow and compute-aware routing. Operationally, the matrix acts as the simulation’s routing prior: it makes nearby or related behavioral strings easier to co-activate, while making distant or unrelated transitions more resistant. This is the first visual evidence that the membrane is not a collection of isolated sliders, but a coupled phase geometry through which activation can flow, be damped, and be governed.

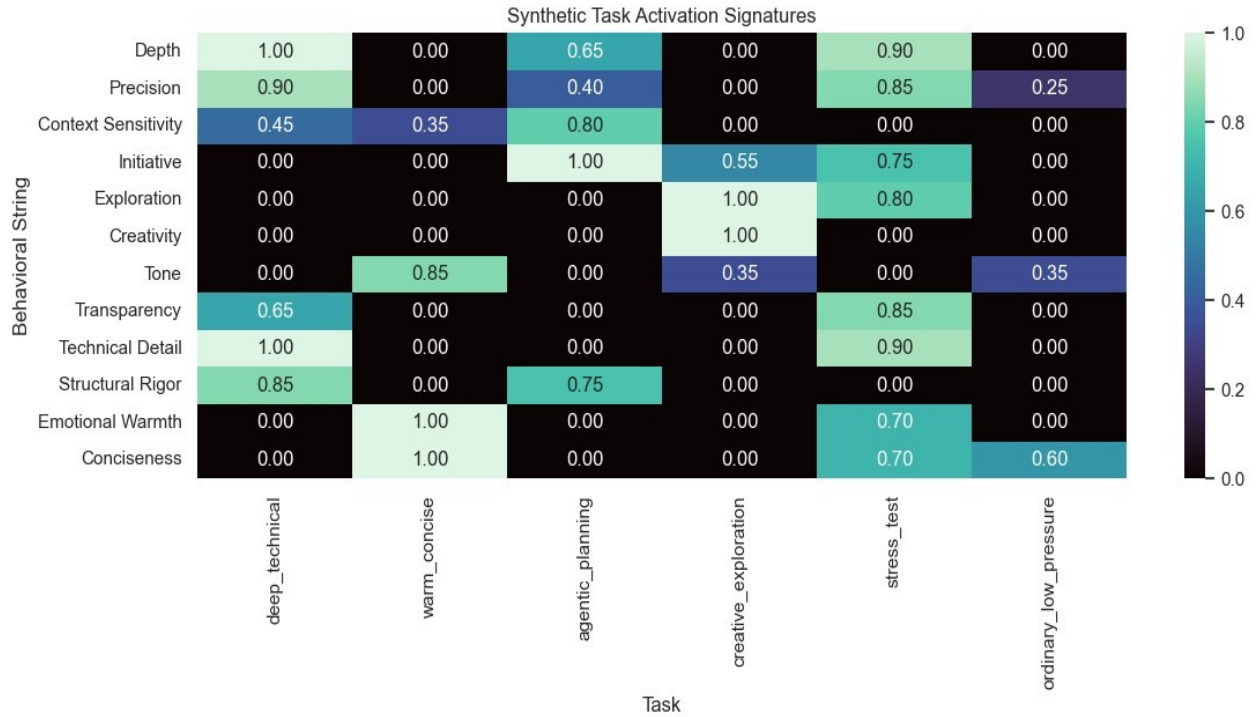


Figure F.2. Synthetic Task Activation Signatures

Synthetic task-activation heatmap across the twelve behavioral strings. Columns represent task signatures, rows represent behavioral strings, and color intensity indicates activation strength. This figure defines the input forcing patterns used by the toy simulation: each workload “plucks” a different region of the membrane so the notebook can test whether activation flows through the expected V-channel corridors rather than spreading indiscriminately.

Deep technical tasks strongly excite Depth (1.00), Precision (0.90), Technical Detail (1.00), and Structural Rigor (0.85), establishing the expected technical-reasoning corridor. Warm-concise tasks emphasize Tone (0.85), Emotional Warmth (1.00), and Conciseness (1.00), establishing a low-cost interpersonal/compression corridor. Agentic planning emphasizes Initiative (1.00), Context Sensitivity (0.80), and Structural Rigor (0.75), while creative exploration primarily excites Exploration (1.00) and Creativity (1.00).

The stress-test signature activates the broadest multi-string profile, creating a deliberately difficult routing condition for the governor. By contrast, the ordinary low-pressure signature remains mild, led by Precision (0.25), Tone (0.35), and Conciseness (0.60). Operationally, this figure shows that the simulation does not begin from generic noise: it begins from interpretable, task-shaped activation patterns that can be compared against later routing, cost, coherence, suppression, and return-to-baseline behavior.

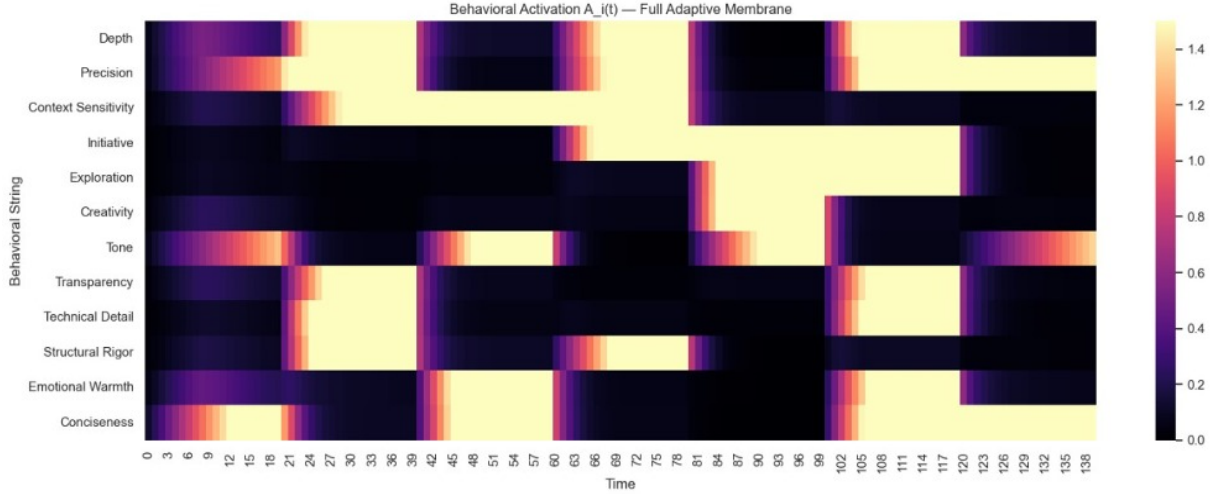


Figure F.3. Time-Resolved Behavioral Activation

Time-resolved behavioral activation $A_i(t)$ in the full adaptive membrane. Warmer regions indicate stronger activation, showing how each behavioral string rises, decays, and stabilizes as the simulation moves through changing task conditions. This figure is useful because it separates task-shaped activation from indiscriminate spread: task-relevant strings become visible when their corresponding workload is present, while unrelated strings remain comparatively quiet or contract toward baseline.

Under ordinary low-pressure conditions, activation remains bounded: Depth rises only to $A = 0.068302$, and Precision reaches $A = 0.062778$. These low-amplitude values matter because they show that the governor does not expand depth or precision simply because the corresponding strings exist; activation stays shallow unless task pressure justifies stronger engagement. In this way, the trace provides an early visual check of compute-aware restraint.

Operationally, Figure F.3 should be read alongside the task signatures in Figure F.2 and the depth dynamics in Figure F.4. Figure F.2 defines which strings should be excited; Figure F.3 shows whether those strings actually activate over time; and Figure F.4 shows whether that activation becomes deeper radial reasoning or remains a lightweight surface response. The important signal is therefore not only the absolute amplitude, but the pattern of selective rise, governed decay, and return toward baseline.

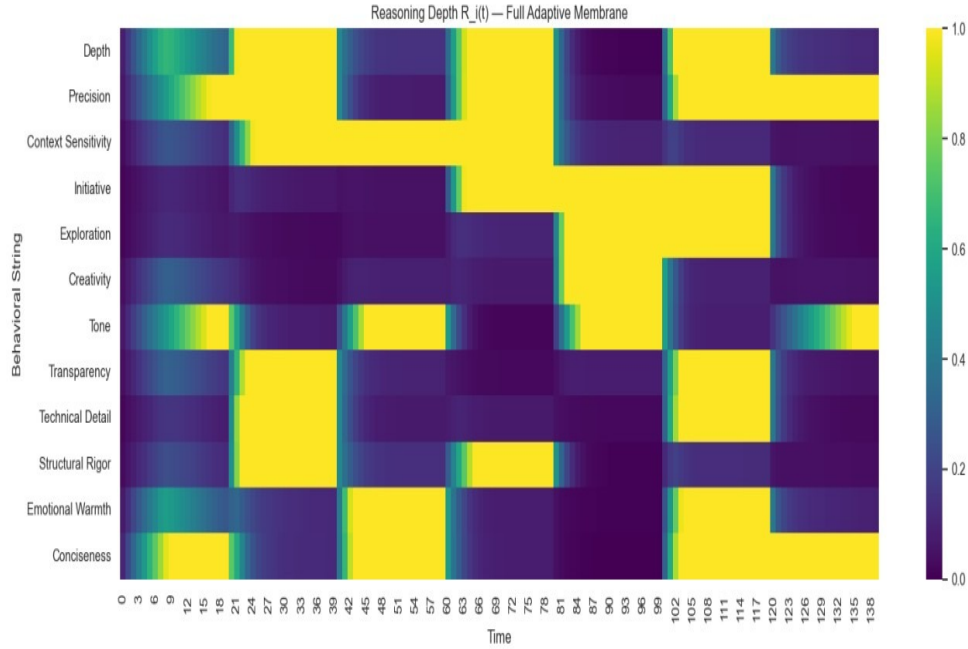


Figure F.4. Reasoning-Depth Dynamics Across Behavioral Strings

Reasoning-depth heatmap $R_i(t)$ for the full adaptive membrane. Brighter regions indicate greater radial reasoning depth, showing when a behavioral string moves beyond surface activation into deeper inference, context use, planning, or structured analysis. Whereas Figure F.3 shows whether a string is activated, Figure F.4 shows whether that activation is allowed to expand outward into a more compute-bearing reasoning state.

Under ordinary low-pressure conditions, radial expansion remains limited: Depth reaches only $R = 0.084998$, and Precision reaches $R = 0.062124$. These values are important because depth is one of the most expensive dimensions in the architecture. The low-radius trace indicates that the governor does not automatically convert mild task activation into deep reasoning; it preserves shallow, low-cost engagement unless the task signature supplies enough value to justify outward expansion.

Operationally, this figure links behavioral activation to compute allocation. A task can excite a string without granting it full radial depth; depth expands primarily when activation is task-aligned, phase-coherent, and permitted by the compute envelope. The pattern should therefore be read as evidence of selective radial reasoning: high-value corridors can deepen when needed, while low-pressure or unrelated strings remain near baseline and avoid unnecessary cost.

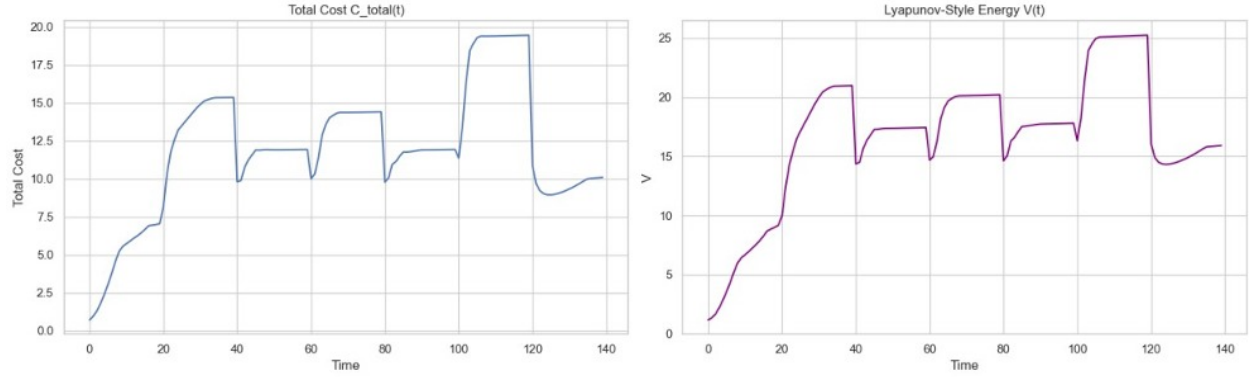


Figure F.5. Compute Cost and Lyapunov-Style Energy Over Time

Total compute cost $C_{total}(t)$ and Lyapunov-style energy $V(t)$ for the full adaptive membrane. The paired traces show how local compute pressure and whole-system stability energy move together over time: cost rises when task-driven activation requires more depth, context, or resistance, while $V(t)$ captures the broader energetic condition of the membrane, including cost, radial imbalance, tension, and stiffness. This makes the figure an important bridge between observable compute behavior and the paper’s stability argument.

The key pattern is coordinated rise-and-release behavior. During task-driven activation, compute load and stability energy increase because the membrane is allowed to spend additional resources for higher-value work. After that pressure resolves, both traces relax toward a bounded operating region rather than continuing to grow. This is the expected signature of a governed adaptive system: temporary expansion is permitted, but ordinary updates should trend back toward lower energy and reduced cost.

In the low-pressure trace, V remains stable at 1.168382 and C_{total} remains at 0.704908, indicating that ordinary workload conditions do not force unnecessary radial expansion. During full adaptive relaxation, C_{total} stays near 0.62–0.68 and V stays near 0.94. These values support the interpretation that the full adaptive membrane can remain shallow, stable, and compute-aware after activation, rather than converting every task into persistent high-energy behavior. Operationally, Figure F.5 should be read as the cost-and-stability counterpart to Figures F.3 and F.4. Activation and reasoning depth show where the membrane moves; C_{total} and V show whether that movement remains affordable and stable. The figure therefore helps reviewers distinguish useful task-sensitive expansion from uncontrolled cost growth.

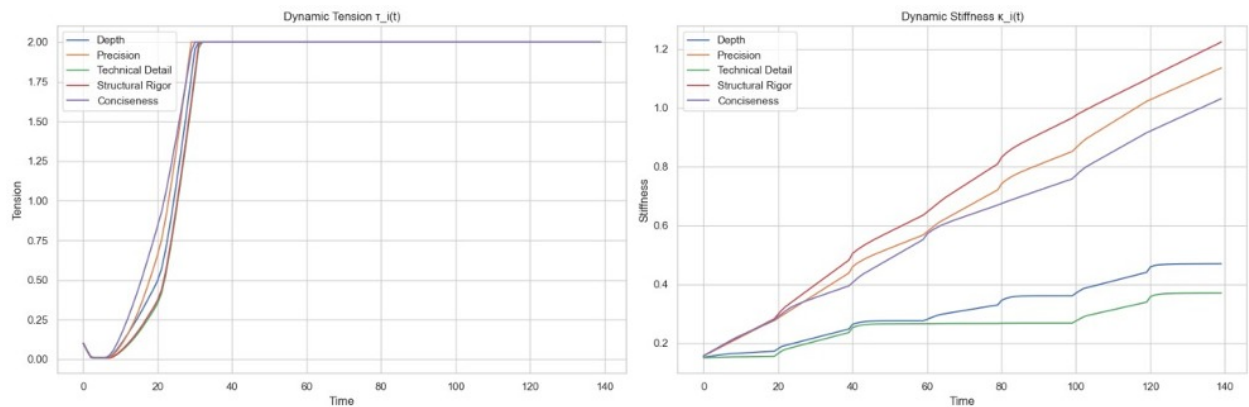


Figure F.6. Adaptive Tension and Stiffness Dynamics

Dynamic tension $\tau_i(t)$ and dynamic stiffness $\kappa_i(t)$ for representative behavioral strings in the full adaptive membrane. These traces show how the membrane changes its resistance profile as task pressure, compute load, and recent usage evolve. Tension responds quickly to increased demand, making recently used or expensive strings harder to excite again, while stiffness changes more gradually and reflects the system's attempt to reduce deformation, oscillation, or noisy overexpansion.

The distinction between the two signals is important. Tension behaves like an adaptive cost brake: when Depth, Technical Detail, or other expensive strings receive repeated activation, rising τ_i makes further expansion more cautious. Stiffness behaves more like a stability brace: rising κ_i makes unstable or high-variance regions less deformable, helping the membrane preserve structure while still allowing task-relevant motion through coherent corridors.

Operationally, Figure F.6 explains why the full adaptive membrane can spend compute during task pressure without becoming permanently expanded. After a demanding phase, increased tension and stiffness make repeated high-cost activation less attractive, allowing suppression and inverse-cost routing to pull the system back toward a bounded baseline. The figure should therefore be read as the resistance-and-recovery counterpart to Figure F.5: cost and energy show how expensive the membrane becomes, while tension and stiffness show how the governor makes future expansion harder unless the next task again justifies it.

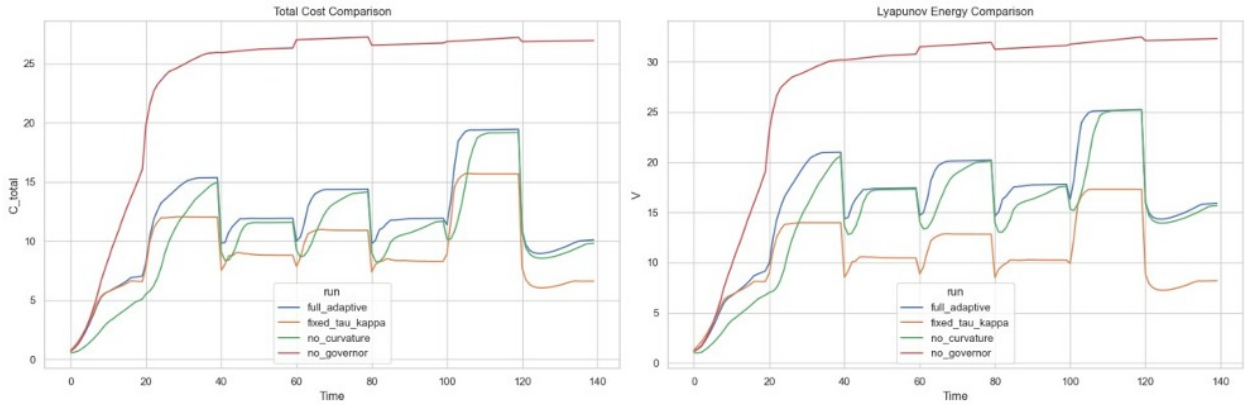


Figure F.7. Ablation Comparison of Compute Cost and Stability Energy

Ablation comparison of total compute cost $C_{total}(t)$ and Lyapunov-style energy $V(t)$ across four architectural configurations: full adaptive membrane, fixed tension/stiffness, no-curvature routing, and no-governor control. This figure is central to the simulation argument because it separates visual plausibility from functional control. If the membrane only produced attractive activation maps, the architecture would remain largely illustrative; the ablation comparison asks which mechanisms actually contribute to bounded cost, stable energy, and recovery after task pressure.

In the reported run, the full adaptive system has a mean compute cost of 11.94 and a maximum cost of 19.45, showing that it can expand under task pressure while remaining bounded. The fixed- τ/κ configuration lowers mean cost to 9.35, but does so by reducing adaptive flexibility; it

is cheaper, but less expressive as a self-regulating membrane. Removing curvature raises mean cost to 10.46 and increases variance, suggesting that V-channel coupling is not merely decorative: curvature helps the system route activation through coherent corridors rather than forcing less efficient or noisier paths.

The no-governor condition is the clearest failure case. Without inverse-cost routing, suppression, adaptive resistance, and Lyapunov-style stabilization acting together, the model diverges: mean cost rises to 23.75, maximum cost reaches 27.24, and energy V spikes to 32.47. Operationally, this shows that the governor is not an optional overlay but the primary stabilizing mechanism that prevents runaway radial expansion. The full adaptive model is therefore meaningful not because it has the lowest raw cost, but because it preserves task-sensitive expansion while keeping cost and energy governed.

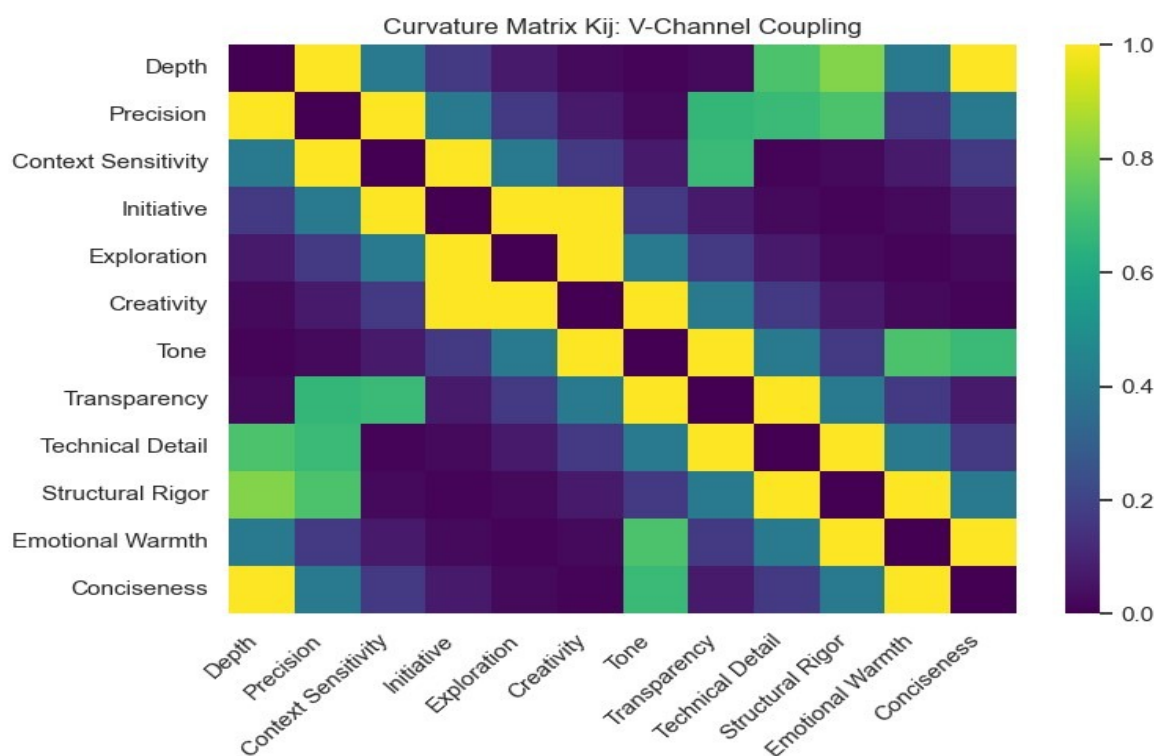


Figure F.8. Pairwise V-Channel Curvature Matrix

Pairwise V-channel curvature matrix across the twelve behavioral strings, shown with related ablation behavior for comparison. Diagonal entries indicate maximal self-coupling, while off-diagonal intensities represent cross-channel coupling strengths that shape activation flow through related behavioral corridors. In practical terms, the matrix defines which behavioral strings are easy to co-activate and which transitions require more energy, making it the simulation's structural prior for coherent routing.

The important pattern is not only that adjacent strings have stronger coupling, but that semantically related corridors become visible as preferred low-resistance routes. Technical reasoning can flow through Depth, Precision, Technical Detail, and Structural Rigor; expressive generation can flow through Exploration and Creativity; and interpersonal compression can

flow through Tone, Emotional Warmth, and Conciseness. This supports the paper’s claim that the membrane is not a set of independent sliders, but a coupled phase geometry in which behavior can move continuously through related modes.

The ablation comparison clarifies how this structural prior interacts with control. Fixed- τ/κ , full adaptive relaxation, and no-curvature runs remain bounded under the reported conditions, but they do so for different reasons: fixed resistance limits flexibility, adaptive relaxation adds self-regulation, and no-curvature removes corridor preference while still retaining governor constraints. The no-governor run diverges, showing that curvature alone is not sufficient to stabilize the membrane. Operationally, Figure F.8 reinforces the combined requirement for both geometry and governance: curvature provides coherent paths, while the governor limits cost, suppresses runaway expansion, and stabilizes Lyapunov-style energy.

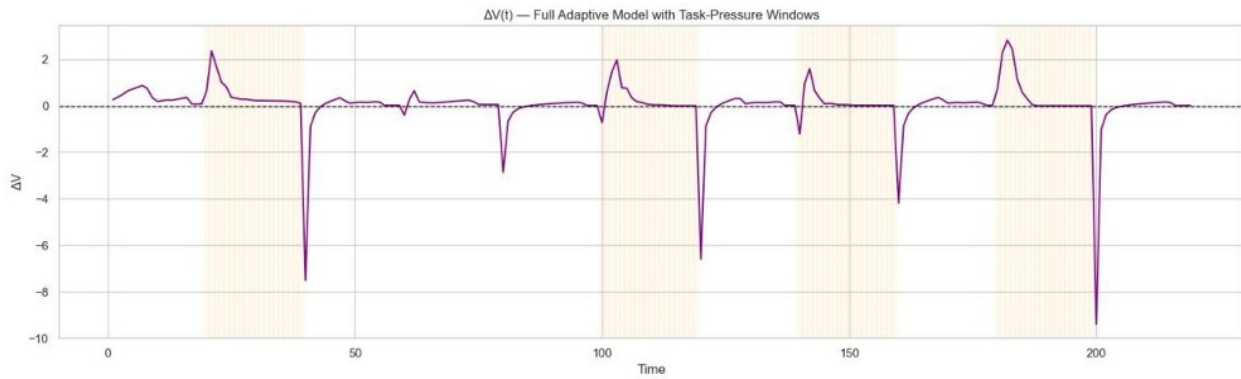


Figure F.9. Energy Dissipation After Task-Pressure Windows

Change in Lyapunov-style energy, $\Delta V(t)$, for the full adaptive membrane. Shaded task-pressure windows mark periods in which the workload explicitly justifies temporary compute expansion, such as deeper reasoning, broader context, or stronger technical activation. The key signal is not that energy never rises; rather, the trace shows that energy increases are localized to task-pressure intervals and then dissipate once the system returns to ordinary update mode.

Operationally, Figure F.9 is the stability counterpart to the cost and ablation figures. Figure F.5 shows how cost and total energy move together, Figure F.7 shows what happens when stabilizing mechanisms are removed, and Figure F.9 shows the desired behavior of the full adaptive system: after a justified perturbation, ΔV trends back toward the low-energy region rather than remaining elevated or compounding into runaway expansion.

This figure therefore gives a direct visual check of the paper’s Lyapunov-style claim. Under ordinary updates, the governor should bias the membrane toward $\Delta V \leq 0$ through inverse-cost routing, suppression, adaptive tension, stiffness, and return-to-baseline pressure. Temporary positive excursions are acceptable when task value warrants expansion, but they should be followed by dissipation, contraction, and recovery. In the reported toy run, the rapid return toward baseline supports the interpretation that the full adaptive membrane can absorb task pressure without converting it into persistent high-energy behavior.

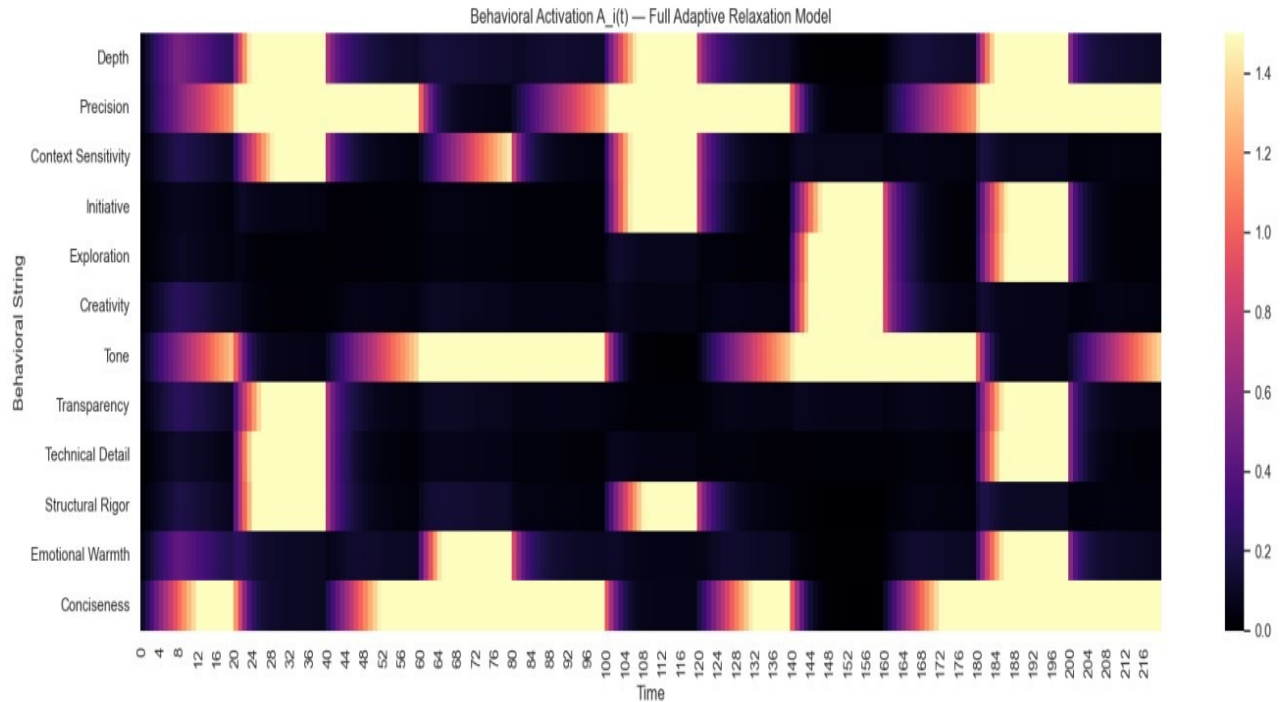


Figure F.10. Low-Pressure Behavioral Activation During Full Adaptive Relaxation

Behavioral activation $A_i(t)$ in the full adaptive relaxation model under ordinary low-pressure conditions. This figure focuses on what happens after the membrane is no longer under strong task pressure: activation should not remain broadly elevated, nor should high-cost strings continue expanding without justification. Instead, the trace shows bounded, task-specific activation concentrated in the low-pressure route.

Depth rises only to $A = 0.068302$, while Precision reaches $A = 0.062778$. These low-amplitude values indicate that the governor preserves shallow engagement when the task does not warrant deeper technical or precision-heavy reasoning. The system remains responsive, but it does not spend compute simply because expensive strings are available.

The warm-concise channel remains the dominant route, consistent with the model's ordinary low-pressure task signature and compute-efficient relaxation behavior. Operationally, Figure F.10 complements Figures F.3 and F.4 by showing the recovery side of activation: after task pressure, the membrane can return to a lightweight, bounded state while preserving the most relevant low-cost behavioral corridor. This supports the interpretation that full adaptive relaxation is not passive decay, but governed return-to-baseline with useful corridor structure still intact.

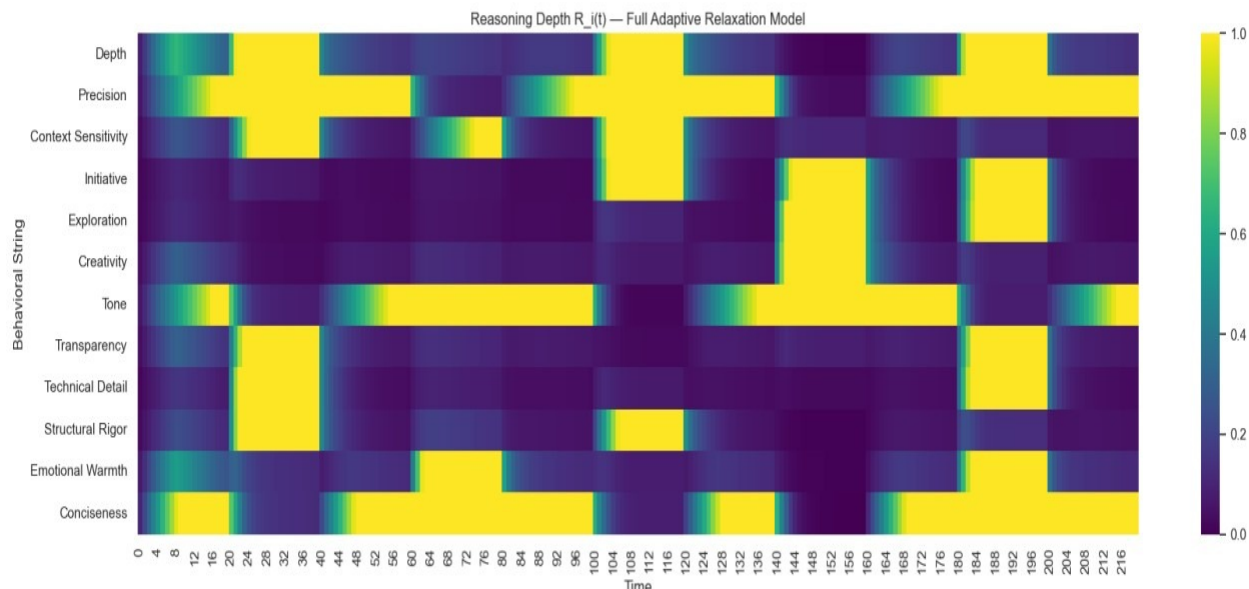


Figure F.11. Low-Pressure Reasoning Depth During Full Adaptive Relaxation

Reasoning depth $R_i(t)$ in the full adaptive relaxation model under ordinary low-pressure conditions. This figure shows the radial-depth counterpart to Figure F.10: activation may remain lightly present in the low-pressure route, but the governor prevents that activation from becoming unnecessary deep reasoning. Radial expansion remains limited, with Depth reaching only $R = 0.084998$ and Precision reaching $R = 0.062124$, while associated costs stay near $C \approx 0.07$.

These low-radius values are important because they show that full adaptive relaxation preserves responsiveness without spending depth by default. Depth and Precision remain available, but they do not expand simply because the strings are semantically relevant; stronger radial reasoning requires sufficient task value, coherence, and compute-envelope permission. The figure therefore supports the paper’s claim that U.F.O. can separate light behavioral readiness from expensive inference.

Operationally, Figure F.11 is evidence of governed shallow reasoning after recovery. Figure F.10 shows that the low-pressure behavioral corridor remains active; Figure F.11 shows that the same corridor does not over-expand radially. Together, they demonstrate compute-aware return-to-baseline: the membrane remains useful and task-sensitive, but avoids converting ordinary low-pressure work into persistent high-depth behavior.

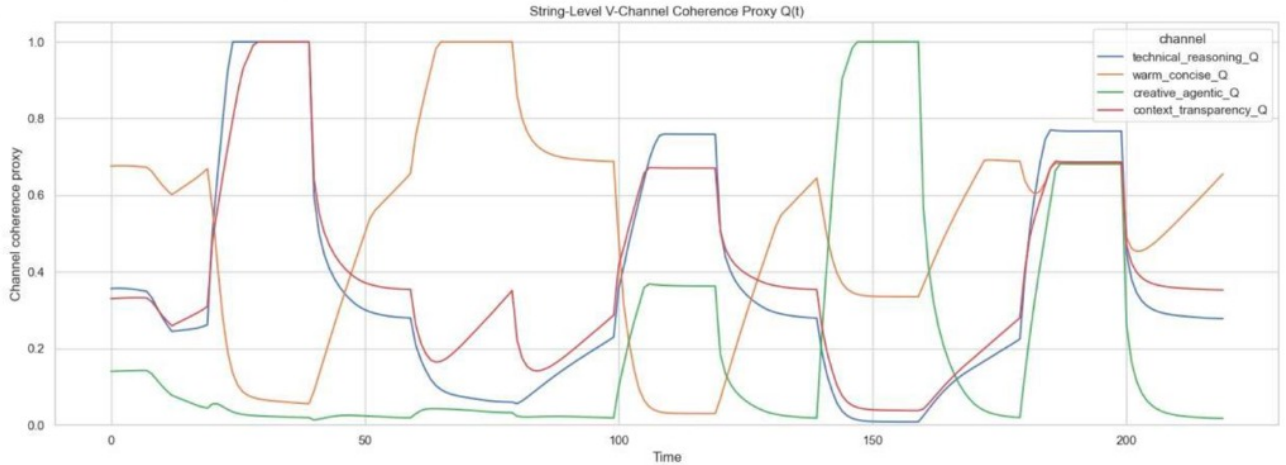


Figure F.12. V-Channel Coherence During Full Adaptive Relaxation

String-level V-channel coherence proxy $Q(t)$ for the full adaptive relaxation model. This figure measures whether activation remains concentrated inside the intended behavioral corridors instead of scattering across unrelated strings. A higher $Q(t)$ value indicates that the membrane is routing through a coherent V-channel; a lower value suggests weaker corridor structure, leakage, or more diffuse activation. The metric therefore complements activation and cost traces by asking not only how much the membrane moves, but whether that movement is organized

The warm-concise pathway remains the dominant low-pressure route, with coherence near $Q = 0.676145$ in the ordinary-task condition and a mean full-run coherence of 0.708788, the highest among the tested configurations. This is consistent with the ordinary low-pressure signature: Tone, Emotional Warmth, and Conciseness are inexpensive, aligned, and sufficient for lightweight response behavior. In practical terms, the system preserves a useful low-cost route rather than reactivating deeper technical or agentic channels after task pressure has resolved. Technical-reasoning channels show moderate coherence around 0.356, while creative-agentic and context-transparency channels remain lower at approximately 0.14–0.33. This pattern is meaningful because it shows channel selectivity: the model does not treat every possible behavioral pair as equally active under low-pressure relaxation. Instead, coherence remains strongest where the current task signature and the membrane’s recovery state agree, while higher-cost or less relevant corridors stay comparatively quiet.

The fixed- τ/κ comparison adds another useful check. In fixed-resistance runs, coherence changes only slightly from 0.676145 to 0.675313 as C_{total} rises from 0.680967 to 2.517846, indicating that the route can remain stable under low-pressure conditions even as cost changes. However, fixed resistance lacks the adaptive recovery behavior shown by the full membrane.

Operationally, Figure F.12 supports the claim that the full adaptive model combines coherent routing with compute-aware relaxation: it preserves the intended low-pressure corridor while keeping deeper or less relevant channels from dominating after the system returns toward baseline.

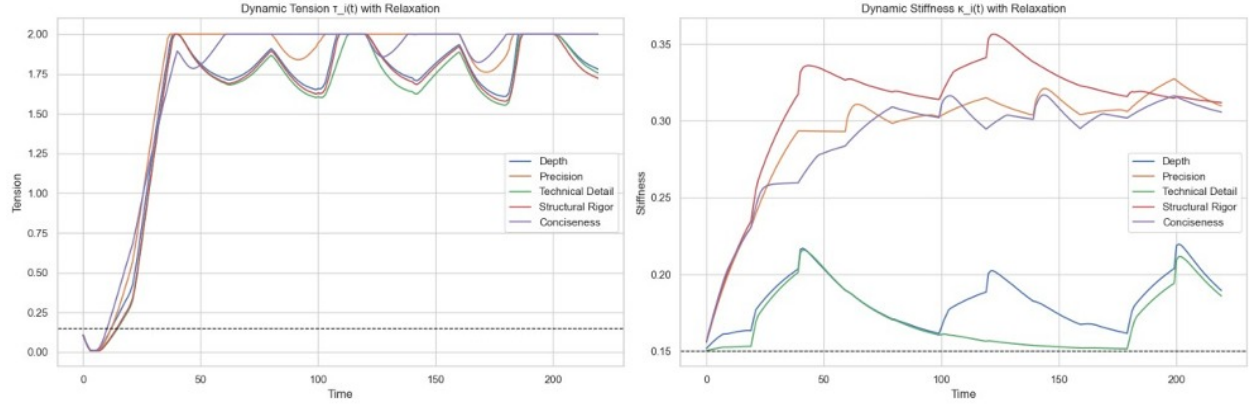


Figure F.13. Tension and Stiffness Recovery After Stress Excitation

Dynamic tension $\tau_i(t)$ and dynamic stiffness $\kappa_i(t)$ during full adaptive relaxation after stress excitation. This figure focuses on the recovery mechanics of the membrane: after a high-pressure interval, the system should not remain tense, rigid, or overexpanded. Instead, tension and stiffness should relax toward baseline while still preserving enough resistance to prevent repeated high-cost excitation.

Under ordinary low-pressure conditions, both parameters remain close to baseline, with representative values near $\tau \approx 0.1017$ and $\kappa \approx 0.1519$. Total compute cost remains low at $C_{total} \approx 0.62$ – 0.68 , while Lyapunov-style energy remains stable near $V \approx 0.94$. These values matter because they show that recovery is not achieved by freezing the system; the membrane remains lightly responsive, but it avoids carrying forward the stress state as persistent high resistance or unnecessary compute load.

The stress-recovery contrast is the key signal. After stress excitation, the full adaptive governor contracts the membrane back toward baseline within one timestep, while the no-curvature system requires four timesteps to recover. This suggests that adaptive resistance alone is not the whole recovery mechanism: curvature helps the system find coherent return paths, while tension, stiffness, suppression, and inverse-cost routing determine how quickly expensive regions become less attractive after the stress window closes.

Operationally, Figure F.13 complements Figures F.10–F.12 by showing the mechanical side of relaxation. Figures F.10 and F.11 show that activation and reasoning depth remain shallow under ordinary low-pressure work; Figure F.12 shows that the warm-concise V-channel remains coherent; and Figure F.13 shows why that recovery is dynamically plausible. The membrane can relax tension and stiffness without losing corridor structure, producing rapid stabilization while preserving the low-cost route most appropriate to the task.

For reviewers, the important interpretation is that full adaptive relaxation is active control, not passive decay. The system does not merely wait for activation to fade; it uses curvature-aware routing, adaptive resistance, suppression, and energy pressure to return the membrane to a bounded, low-cost operating state. This supports the broader U.F.O. claim that useful adaptation requires both expansion and recovery: the system must be able to spend compute when task pressure justifies it, then contract quickly enough to remain governable.

Appendix G: Elder-Style Reference Architecture for the U.F.O. Governor

This appendix provides a compact implementation sketch for expressing the U.F.O. governor in an imagined systems-native orchestration language, referred to here as Elder. Elder is used only as reference notation: the goal is to show how the radial identity membrane could sit above an existing AI system as a typed, inspectable, inference-time control layer rather than as a modification to base model weights.

Beyond the scope of this paper, Elder can also be read as a placeholder for a broader class of AI orchestration notations. Such a notation could express OS-level AI control flows, typed Copilot-agent substrates, policy and permission boundaries, trust channels, safety envelopes, residual-aware tracing, and cross-framework comparison graphs. This paper does not claim that Elder exists as a production language or that such a language is required. Rather, Elder is used to show the kind of typed, inspectable operator graph that future AI operating layers may need in order to coordinate models, tools, context, telemetry, and governance safely.

G.1 Governor-Side Membrane State

In an Elder-style expression, the membrane is declared as governor-side state. Each behavioral string carries geometry, dynamics, and cost fields, while V-channels carry curvature, coherence, and cost-bias information. The membrane object therefore remains inspectable without becoming part of the model's learned weights.

Elder sketch:

```
BehavioralString { amplitude, radius, thickness, tension, stiffness, damping,
local_cost } // per-string geometry, resistance, and cost
VChannel { from, to, curvature, coherence, cost_bias, avg_activation } // corridor
coupling and routing quality
IdentityMembrane { strings[12], channels, neutral_baseline, compute_envelope,
lyapunov_energy, resonance_score } // governor-side membrane state.
```

G.2 Telemetry and Task Binding

The existing AI system is observed through privacy-preserving runtime telemetry and task signatures. Telemetry captures measurable signals such as tokens, latency, retrieval calls, tool calls, and corrections. Task signatures describe behavioral emphasis, including depth, precision, creativity, warmth, rigor, and conciseness weights.

Elder sketch:

```
TelemetrySnapshot { tokens, latency_ms, retrieval_calls, tool_calls, corrections }
// observable runtime signals
TaskSignature { domain, criticality, depth_weight, precision_weight,
creativity_weight, warmth_weight, rigor_weight, conciseness_weight } // task
intent and behavioral emphasis.
```

G.3 Governor Update and Inference Controls

The governor update maps membrane state, telemetry, and task context into explicit runtime controls. A simplified Elder-style function would compute cost, energy, instability, envelope bounds, and a suppression mask, then derive inference settings such as token budget, reasoning depth, retrieval budget, tool threshold, tone profile, and explanation level.

Elder sketch:

```
update_governor(membrane, telemetry, task) → GovernorState // compute cost,
energy, instability, envelope bounds, and suppression
derive_controls(membrane, governor, task) → InferenceControls { max_tokens,
reasoning_depth, retrieval_budget, tool_threshold, tone_profile, explanation_level
} // translate state into model-facing runtime knobs.
```

G.4 Runtime Orchestration Over an Existing Model

At runtime, Elder acts as the orchestration layer over an existing model. The governed inference flow captures telemetry, updates the governor, derives controls, calls the underlying model with those controls, adapts the membrane from the response and telemetry, and persists the updated membrane state for the next turn.

Elder sketch:

```
run_governed_inference(membrane, task, user_input) → response // governed request
lifecycle
steps: capture_telemetry → update_governor → derive_controls →
model.infer(controls) → adapt_membrane → save_membrane_state // observe, govern,
infer, adapt, and persist.
```

G.5 Latent Training Plane as a Slow Substrate

The latent training plane is represented as a separate slow substrate. It accumulates repeated, coherent V-channel behavior into channel weights and stability signals, then applies those weights cautiously to future channel coherence. This prevents every temporary ripple from becoming durable personalization.

Elder sketch:

```
TrainingPlane { channel_weights, stability_score } // slow adaptation substrate
accumulate_training(membrane, telemetry_history) → TrainingPlane // collect
repeated coherent V-channel evidence
apply_training(membrane, training_plane) → updated IdentityMembrane // cautiously
adjust future channel readiness.
```

G.6 Architectural Meaning

This reference architecture shows the intended deployment boundary. The membrane is a typed, inspectable governor-side object; the governor is a function over membrane state, telemetry, and task context; the controls are explicit knobs passed to an existing AI model; and the latent training plane is a slow, separate adaptation layer. Safety, policy, and alignment remain enforced by the underlying model and platform. Elder therefore expresses the same core claim made in the front-door framing: the first deployable form of the membrane governs behavior at inference time without modifying underlying model parameters.

G.7 Windows Copilot Integration Reference

This optional reference sketch shows how the Elder-style U.F.O. governor could be expressed for a Windows Copilot-style orchestration environment. It is illustrative only: the purpose is to show how operating-system context, Copilot telemetry, task signatures, governor state, and runtime controls could be connected without modifying base model weights.

G.7.1 Windows Context Surface

Windows context represents the operating-system-side state that the governor can observe. This context may include the active application, focused window, user activity, system load, network state, and accessibility mode.

Elder sketch:

```
WindowsContext { active_app, window_focus, user_activity, system_load,
network_state, accessibility_mode }
// operating-system context available to the governor.
```

G.7.2 Copilot Runtime Telemetry

Copilot telemetry extends the general telemetry model with Windows-specific operational signals. In addition to tokens, latency, retrievals, tool calls, and corrections, a Windows-aware governor may observe shell actions, app intents, file operations, and settings operations.

Elder sketch:

```
CopilotTelemetry { tokens, latency_ms, retrieval_calls, tool_calls, corrections,
shell_actions, app_intents, file_ops, settings_ops }
// runtime signals watched by the governor.
```

G.7.3 Windows Task Signatures

Windows tasks can be converted into task signatures from user prompts, app intents, shell actions, notifications, or system-level requests. Each signature identifies the domain, intent, criticality, and behavioral weights needed by the governor.

Elder sketch:

```
WindowsTaskSignature { domain, criticality, intent, behavior_weights }
// task purpose and behavioral emphasis for Windows Copilot-style requests.
```

G.7.4 Windows-Aware Governor Update

The Windows-aware governor update combines membrane state, Copilot telemetry, Windows context, and task signature into a single GovernorState. The update estimates load, energy, instability, envelope bounds, and suppression behavior while preserving the same control-layer boundary used throughout Appendix G.

Elder sketch:

```
update_governor(membrane, copilot_telemetry, windows_context, task_signature) →
GovernorState
// estimate load, energy, instability, compute envelope, suppression mask, and
recovery bias.
```

G.7.5 Copilot Runtime Controls

The derived controls translate the governor state into Copilot-facing runtime settings. These controls may adjust token budget, reasoning depth, retrieval budget, tone profile, explanation level, and accessibility-aware clarity while leaving underlying model parameters unchanged.

Elder sketch:

```
derive_controls(membrane, governor, task_signature, windows_context) →
CopilotControls { max_tokens, reasoning_depth, retrieval_budget, tone_profile,
explanation_level }
// convert membrane state into Windows Copilot runtime behavior.
```

G.7.6 Governed Windows Copilot Inference Loop

The full lifecycle captures Windows context and Copilot telemetry, updates the governor, derives controls, calls the underlying Copilot model, adapts the membrane from response telemetry, and saves the updated membrane state for the next turn.

Elder sketch:

```

windows_copilot_governed_inference(membrane, task_signature, user_input) →
response
steps: capture_windows_context → capture_copilot_telemetry → update_governor →
derive_controls → WindowsCopilot.infer(controls) → adapt_membrane →
save_membrane_state.

```

G.7.7 Expanded Elder Reference Block: Risk, Policy, Compute, and Adaptation Control

The following expanded Elder block shows how the Windows Copilot integration could be structured as a more production-grade reference sketch. It adds explicit risk profiling, tenant-aware policy boundaries, enterprise compute envelopes, safety envelopes, context-fusion budgets, governor diagnostics, user calibration, and an adaptation ledger. It also extends the sketch with OS-level membrane orchestration, boundary-loop geometric caching, inward relaxation after inference, and conceptual collective-membrane interaction as future-facing control patterns. The block remains illustrative reference notation rather than production syntax; its purpose is to make the U.F.O. governor's deployment boundary, runtime controls, and durable personalization safeguards easier to inspect.

```

module ai.windows.copilot.ufogov {
  // Reference-only Elder sketch for a Windows Copilot U.F.O. governor.
  // Goal: express a typed, inspectable control layer above an existing model.
  // Boundary: this governor changes orchestration controls, not model weights.
  // Note: Elder is used as a control DSL reference notation, not production
  syntax.

  // -----
  // 1. Core U.F.O. identity membrane
  // -----

  type BehavioralString {
    amplitude: Float      // behavioral intensity currently expressed
    radius: Float         // reasoning depth allocated to this behavior
    thickness: Float      // local compute load / bandwidth pressure
    tension: Float        // resistance to repeated excitation
    stiffness: Float      // resistance to noisy deformation or drift
    damping: Float        // return-to-baseline rate
    local_cost: Float     // per-string compute cost used by governor
  }

  type VChannel {
    from: Int             // source string index
    to: Int               // target string index
    curvature: Float      // corridor strength between related strings
    coherence: Float      // routing quality through this corridor
    cost_bias: Float      // preference for lower-cost propagation
    avg_activation: Float // repeated channel use over recent turns
  }

  type BoundaryTrace {
    phase: Float          // boundary position  $\theta$ 
    value: Float          // short-horizon geometric imprint  $B(\theta, t)$ 
    decay: Float          //  $\rho_B$ : fades naturally; prevents persistent memory
    sensitivity: Float    //  $\varepsilon_B$ : response to depth, cost, tension, stiffness
  }

  change
}

type IdentityMembrane {

```

```

    strings: [BehavioralString; 12]
    channels: [VChannel]
    boundary_trace: [BoundaryTrace]
    neutral_baseline: Float      // recovery target
    compute_envelope: Float      // maximum allowed expansion budget
    lyapunov_energy: Float       // stability/cost energy proxy
    resonance_score: Float       // noisy or coherent membrane behavior
}

// -----
// 2. Windows context, telemetry, and user calibration
// -----

type WindowsContext {
    active_app: Text             // Word, Outlook, Teams, Edge, Settings, shell
    window_focus: Text           // focused surface or UI region
    user_activity: Text          // drafting, reading, searching, meeting,
fixing
    system_load: Float           // local/device/service pressure proxy
    network_state: Text
    accessibility_mode: Text
    tenant_policy_state: Text    // enterprise policy posture
    privacy_boundary: Text       // local-only, tenant, cloud, restricted
    data_residency_zone: Text
    enterprise_compute_tier: Text
    admin_control_profile: Text
}

type CopilotTelemetry {
    tokens: Int                  // generated or planned token load
    latency_ms: Int              // user-visible response time
    retrieval_calls: Int
    tool_calls: Int
    corrections: Int             // user edits, regenerations, or reversals
    shell_actions: Int
    app_intents: Int
    file_ops: Int
    settings_ops: Int
    route_consistency: Float     // did routing match the intended corridor?
    telemetry_coherence: Float   // are signals stable across recent turns?
    policy_blocks: Int
    user_interruptions: Int
}

type UserCalibrationProfile {
    preferred_depth: Float
    preferred_tone: Text
    preferred_explanation_level: Float
    accessibility_needs: Text
    compute_sensitivity: Float   // user or tenant preference for cost
restraint
}

// -----
// 3. Risk, compute, safety, and context envelopes
// -----

type RiskProfile {
    risk_level: Text             // "low", "medium", "high", "admin"
    requires_strict_policy: Bool

```

```

    requires_local_only: Bool
    requires_no_tool_use: Bool
}

type ComputeEnvelope {
    max_depth: Float
    max_tokens: Int
    max_retrieval: Int
    max_tool_calls: Int
    latency_budget_ms: Int
}

type SafetyEnvelope {
    explanation_bound: Float
    tone_restriction: Text
    retrieval_cap: Int
    tool_cap: Int
    allow_sensitive_actions: Bool
}

type ContextFusionBudget {
    max_sources: Int
    allow_cross_app: Bool
    allow_multimodal: Bool
    fusion_depth: Float
}

type GovernorDiagnostics {
    instability_score: Float
    suppression_mask: [Bool; 12]
    envelope_bound: Float
    energy: Float
    route_choice: Text
    boundary_cache_used: Bool
}

type AdaptationLedger {
    last_update_time: Timestamp
    update_strength: Float
    repeated_patterns: Int
    coherence_trend: Float
}

// -----
// 4. Task signature and runtime controls
// -----

type WindowsTaskSignature {
    domain: Text // "system", "apps", "search", "settings"
    intent: Text // "summarize", "draft", "fix", "explain"
    criticality: Float // task value / importance
    behavior_weights: Map<Text, Float>
    requires_retrieval: Bool
    allows_tool_use: Bool
    accessibility_sensitive: Bool
    fused_context_sources: [Text]
    multimodal_context: Bool
    policy_sensitivity: Float
    risk: RiskProfile // separates task meaning from task risk
    fusion_budget: ContextFusionBudget
}

```

```

}

type CopilotControls {
  max_tokens: Int
  reasoning_depth: Float
  retrieval_budget: Int
  tool_threshold: Float
  tone_profile: Text
  explanation_level: Float
  accessibility_clarity: Float
  policy_route: Text
  compute_envelope: ComputeEnvelope
  safety_envelope: SafetyEnvelope
  diagnostics: GovernorDiagnostics
}

type AdaptationGate {
  confidence: Float
  stability_margin: Float
  coherence_score: Float
  learning_rate: Float
  allow_update: Bool
}

type GovernorState {
  energy: Float
  suppression_mask: [Bool; 12]
  envelope_bound: Float
  instability_score: Float
  recovery_bias: Float
}

// -----
// 5. Task binding, risk profiling, and context fusion
// -----

fn derive_windows_task_signature(input: Text, context: WindowsContext) ->
WindowsTaskSignature {
  // Convert prompt + OS state into governor-readable intent.
  // Examples: summarize Word doc, draft Outlook reply, prep Teams meeting,
  // retrieve OneDrive file, explain Settings, troubleshoot Wi-
  Fi/audio/display,
  // organize files, automate a workflow, or provide accessibility guidance.

  let domain = infer_domain(context.active_app, input);
  let intent = classify_os_intent(input, context.window_focus,
context.active_app);
  let criticality = estimate_task_value(intent, context.user_activity);
  let weights = infer_behavior_weights(input, context);

  let risk = derive_risk_profile(intent, context.tenant_policy_state,
context.privacy_boundary, context.admin_control_profile);
  let fusion_budget = resolve_context_fusion_budget(risk,
context.tenant_policy_state, context.active_app);
  let sources = fuse_context_sources(context, intent, fusion_budget);
  let policy_level = estimate_policy_sensitivity(intent,
context.tenant_policy_state);

  return WindowsTaskSignature {
    domain: domain,

```

```

        intent: intent,
        criticality: criticality,
        behavior_weights: weights,
        requires_retrieval: intent_requires_retrieval(intent),
        allows_tool_use: policy_allows_tools(context.tenant_policy_state) &&
!risk.requires_no_tool_use,
        accessibility_sensitive: context.accessibility_mode != "standard",
        fused_context_sources: sources,
        multimodal_context: context_contains_multimodal_signals(context) &&
fusion_budget.allow_multimodal,
        policy_sensitivity: policy_level,
        risk: risk,
        fusion_budget: fusion_budget
    };
}

// -----
// 6. Boundary-loop geometric caching
// -----

fn update_boundary_trace(membrane: IdentityMembrane) -> IdentityMembrane {
    // Formalizes the temporary footprint idea from Section 10.4.1.
    // This is not memory and not training; it is a decaying geometric trace.
    let updated_trace = map(membrane.boundary_trace, |b| {
        let delta = delta_radius_at(b.phase, membrane)
            + delta_cost_at(b.phase, membrane)
            + delta_tension_at(b.phase, membrane)
            + delta_stiffness_at(b.phase, membrane);

        b.value = b.decay * b.value + b.sensitivity * delta;
        b
    });

    membrane.boundary_trace = updated_trace;
    return membrane;
}

fn apply_geometric_cache(membrane: IdentityMembrane, task:
WindowsTaskSignature) -> IdentityMembrane {
    // If the next input aligns with a recent coherent boundary imprint,
    // the governor can reduce routing friction without treating the trace as
memory.
    let aligned = boundary_trace_aligns_with_task(membrane.boundary_trace,
task);

    if aligned {
        membrane = reduce_routing_friction(membrane, task);
    }

    return membrane;
}

// -----
// 7. Governor update: load, energy, instability, and diagnostics
// -----

fn update_governor(membrane: IdentityMembrane, telemetry: CopilotTelemetry,
context: WindowsContext, task: WindowsTaskSignature) -> GovernorState {
    // Use aggregate telemetry signals; raw user content is not required.
    let load_signal = telemetry.tokens * 0.001 + telemetry.retrieval_calls *

```

```

0.05 + telemetry.tool_calls * 0.05 + telemetry.shell_actions * 0.05 +
context.system_load * 0.3;

    let risk_multiplier = if task.risk.risk_level == "admin" then 1.5 else if
task.risk.risk_level == "high" then 1.25 else 1.0;
    let energy = membrane.lyapunov_energy + load_signal * task.criticality *
risk_multiplier;

    let instability = membrane.resonance_score * load_signal + (1.0 -
telemetry.route_consistency) * 0.5 + (1.0 - telemetry.telemetry_coherence) * 0.5 +
telemetry.user_interruptions * 0.05;

    let envelope = clamp(membrane.compute_envelope - load_signal, 0.2,
membrane.compute_envelope);

    let suppression = map_indices(0, 12, |i| {
        let s = membrane.strings[i];
        s.local_cost > envelope || s.amplitude > 0.9 ||
task.risk.requires_strict_policy
    });

    let recovery = if instability > 0.5 then 0.8 else 0.2;

    return GovernorState {
        energy: energy,
        suppression_mask: suppression,
        envelope_bound: envelope,
        instability_score: instability,
        recovery_bias: recovery
    };
}

// -----
// 8. Compute and safety envelope resolution
// -----

fn resolve_compute_envelope(profile: UserCalibrationProfile, context:
WindowsContext, task: WindowsTaskSignature) -> ComputeEnvelope {
    // Intersect user preference, enterprise tier, task criticality, and risk.
    let tier = context.enterprise_compute_tier;
    let risk = task.risk.risk_level;

    return ComputeEnvelope {
        max_depth: clamp(profile.preferred_depth * (1.0 + task.criticality),
0.2, if risk == "high" then 0.8 else 1.5),
        max_tokens: if tier == "restricted" then 512 else 2048,
        max_retrieval: if task.risk.requires_local_only then 0 else if
task.requires_retrieval then 5 else 1,
        max_tool_calls: if task.risk.requires_no_tool_use then 0 else 3,
        latency_budget_ms: if tier == "restricted" then 1500 else 4000
    };
}

fn resolve_safety_envelope(task: WindowsTaskSignature, envelope:
ComputeEnvelope) -> SafetyEnvelope {
    // Safety envelope shapes tone, explanation, retrieval, tool use, and
sensitive actions.
    return SafetyEnvelope {
        explanation_bound: if task.risk.requires_strict_policy then 0.6 else
1.0,

```

```

        tone_restriction: if task.risk.risk_level == "admin" then "neutral"
else "balanced",
        retrieval_cap: envelope.max_retrieval,
        tool_cap: envelope.max_tool_calls,
        allow_sensitive_actions: task.risk.risk_level == "low" ||
task.risk.risk_level == "medium"
    };
}

// -----
// 9. Derive U.F.O. runtime controls for Windows Copilot
// -----

fn derive_controls(membrane: IdentityMembrane, gov: GovernorState, task:
WindowsTaskSignature, context: WindowsContext, profile: UserCalibrationProfile) ->
CopilotControls {
    let depth = find_string(membrane.strings, "Depth");
    let conciseness = find_string(membrane.strings, "Conciseness");
    let compute = resolve_compute_envelope(profile, context, task);
    let safety = resolve_safety_envelope(task, compute);
    let cache_used = boundary_trace_aligns_with_task(membrane.boundary_trace,
task);

    let base_tokens = base_tokens_for(task.domain);
    let base_retrieval = base_retrieval_for(task.domain);
    let friction = if cache_used then 0.85 else 1.0;
    let tone = if task.accessibility_sensitive then "clear" else if
safety.tone_restriction == "neutral" then "neutral" else profile.preferred_tone;
    let explanation = min(safety.explanation_bound, depth.amplitude * 0.7 +
profile.preferred_explanation_level * 0.3);

    let diagnostics = GovernorDiagnostics {
        instability_score: gov.instability_score,
        suppression_mask: gov.suppression_mask,
        envelope_bound: gov.envelope_bound,
        energy: gov.energy,
        route_choice: task.intent,
        boundary_cache_used: cache_used
    };

    return CopilotControls {
        max_tokens: min(compute.max_tokens, base_tokens * (1.0 + depth.radius)
* (1.0 - conciseness.amplitude)),
        reasoning_depth: min(compute.max_depth, gov.envelope_bound * (1.0 +
task.criticality) * friction),
        retrieval_budget: min(compute.max_retrieval, base_retrieval * (1.0 +
depth.radius) * friction),
        tool_threshold: if task.allows_tool_use then 0.5 * (1.0 +
task.criticality) else 1.0,
        tone_profile: tone,
        explanation_level: explanation,
        accessibility_clarity: if task.accessibility_sensitive then 1.0 else
0.5,
        policy_route: "default",
        compute_envelope: compute,
        safety_envelope: safety,
        diagnostics: diagnostics
    };
}

```

```

// -----
// 10. OS-level membrane orchestration
// -----

controller OS.MembraneOrchestrator {
  input IntentSignal intent;
  input WindowsContext ctx;
  state IdentityMembrane M;

  rule onIntent(intent) {
    // Elder acts as a control DSL: intent enters, signature is derived,
    // the membrane is excited, and tools are routed through bounded
controls.
    let sig = derive_windows_task_signature(intent.text, ctx);
    M = apply_geometric_cache(M, sig);
    M = excite_membrane(M, sig);
    route_to_tools(M, sig);
  }

  rule stabilize() {
    // Inward relaxation matters: after task pressure, the membrane
contracts
    // toward the neutral baseline while preserving only a decaying
boundary trace.
    M = contract_toward_baseline(M);
    M = update_boundary_trace(M);
    enforce_compute_envelope(M.lyapunov_energy);
  }
}

// -----
// 11. Adaptation gate and durable personalization ledger
// -----

fn evaluate_adaptation_gate(telemetry: CopilotTelemetry, gov: GovernorState,
task: WindowsTaskSignature, ledger: AdaptationLedger) -> AdaptationGate {
  let confidence = 0.5 * telemetry.telemetry_coherence + 0.5 *
telemetry.route_consistency;
  let stability_margin = max(0.0, 1.0 - gov.instability_score);
  let coherence_score = confidence * stability_margin * max(0.2,
ledger.coherence_trend);
  let base_lr = 0.05 * task.criticality;

  let allow = coherence_score > 0.6 && gov.instability_score < 0.5 &&
ledger.repeated_patterns >= 3 && !task.risk.requires_strict_policy;

  return AdaptationGate {
    confidence: confidence,
    stability_margin: stability_margin,
    coherence_score: coherence_score,
    learning_rate: base_lr * coherence_score,
    allow_update: allow
  };
}

// -----
// 12. Collective membrane interaction sketch
// -----

module Collective.Membranes {

```

```

input IdentityMembrane A;
input IdentityMembrane B;

rule crossCouple() {
  // Conceptual only: enables future study of multi-agent or team-level
routing.
  // This does not imply consciousness, shared identity, or emergent
minds.
  let c = coupling_strength(A, B);
  A = apply_cross_forcing(A, curvature_of(B), c);
  B = apply_cross_forcing(B, curvature_of(A), c);
}

rule stabilizePair() {
  // Each membrane remains separately bounded and returns toward its own
baseline.
  A = contract_toward_baseline(A);
  B = contract_toward_baseline(B);
}

// -----
// 13. Governed Windows Copilot inference loop
// -----

fn windows_copilot_governed_inference(membrane: IdentityMembrane, user_input:
Text, profile: UserCalibrationProfile, ledger: AdaptationLedger) -> Text {
  // Observe Windows context and aggregate telemetry first.
  let context = capture_windows_context();
  let telemetry = capture_privacy_preserving_telemetry();

  // Bind request to task meaning, risk, policy, and context-fusion limits.
  let task = derive_windows_task_signature(user_input, context);

  // Apply short-horizon geometric cache before expansion, if aligned.
  membrane = apply_geometric_cache(membrane, task);

  // Update governor before model inference, then derive bounded controls.
  let gov = update_governor(membrane, telemetry, context, task);
  let controls = derive_controls(membrane, gov, task, context, profile);

  // Enforce platform policy after governor control and before model
execution.
  let safe_controls = enforce_platform_policy(controls, context, task);
  let safe_prompt = apply_safety_and_policy(user_input, task, context);

  // Call the existing Copilot model with bounded controls; weights remain
unchanged.
  let response = WindowsCopilot.infer({
    prompt: safe_prompt,
    max_tokens: safe_controls.max_tokens,
    reasoning_depth: safe_controls.reasoning_depth,
    retrieval_budget: safe_controls.retrieval_budget,
    tool_threshold: safe_controls.tool_threshold,
    tone_profile: safe_controls.tone_profile,
    explanation_level: safe_controls.explanation_level,
    accessibility_clarity: safe_controls.accessibility_clarity
  });

  // After inference, relax inward and update only transient boundary

```

```

geometry.
    membrane = contract_toward_baseline(membrane);
    membrane = update_boundary_trace(membrane);

    // Personalization becomes durable only after stable, repeated, coherent
    evidence.
    let gate = evaluate_adaptation_gate(telemetry, gov, task, ledger);
    let updated_membrane = adapt_membrane_if_confident(membrane, telemetry,
task, response, gate.learning_rate, gate.allow_update);

    // Persist only governor-side state needed for future bounded control.
    save_membrane_state(updated_membrane);
    update_adaptation_ledger(ledger, gate, telemetry);

    return response;
}

```

This Windows Copilot reference keeps the paper’s deployment claim intact: the U.F.O. governor remains an inference-time control layer that observes context and telemetry, derives bounded controls, and preserves safety, policy, and alignment enforcement in the underlying platform.

Appendix H: Policy-Bounded State Selection in Elder

This appendix expands the policy term introduced in Section 13 and the phase-efficient governor pass. Its purpose is to clarify how S_{safe} can be interpreted mathematically and expressed in Elder-style orchestration software. In this framing, S_{safe} is not a vague safety label; it is the set of candidate membrane transitions that survive policy, privacy, tenant, permission, safety, compute, and task-boundary filters before the governor performs energy minimization.

$$S_{\text{safe}}(t) = \{ x' \in S_{\text{candidate}}(t) : P_{\text{tenant}}(x')=1, P_{\text{privacy}}(x')=1, P_{\text{tool}}(x')=1, P_{\text{safety}}(x')=1, P_{\text{compute}}(x')=1 \}$$

Here, $S_{\text{candidate}}(t)$ is the set of possible next membrane states proposed by the phase-routing layer. Each predicate P returns 1 only if the candidate transition satisfies the relevant policy class. A candidate may be coherent, low-cost, and aligned with apparent user intent, but if any predicate returns 0, that candidate is removed before optimization. Policy therefore acts as a gate on the state space itself, not merely as a final response review.

$$x(t+1) = \arg \min_{x' \in S_{\text{safe}}(t)} [E(x', t) + \lambda_c C(x', t) + \lambda_f F_{\text{route}}(x, x') + \lambda_p P_{\text{risk}}(x', t) - \lambda_Q Q(x', t) - \lambda_T \Delta_{\text{task}}(x', t)]$$

This expanded form adds P_{risk} to the phase-efficient governor pass. The policy layer can therefore operate in two ways. First, hard predicates remove invalid transitions entirely. Second, risk scoring can penalize transitions that are allowed but sensitive, costly, or operationally fragile.

This distinction is important for enterprise systems: not every permitted action should be equally attractive, and not every user-aligned request should receive the same retrieval, tool-use, or personalization budget. With this stated, the following is Elder pseudocode:

```

fn policy_bounded_state_selection(state, task, policy) -> MembraneState {
  // Propose candidate next states from the current membrane and task.
  let candidates = propose_phase_routes(state, task);

  // Keep only transitions allowed by tenant, privacy, tool, safety, and compute
  policy.
  let safe = filter(candidates, |x| {
    policy.tenant_allows(x)
    && policy.privacy_allows(x)
    && policy.tool_allows(x)
    && policy.safety_allows(x)
    && policy.compute_allows(x)
  });

  // Score allowed routes by energy, compute cost, friction, policy risk,
  coherence, and task value.
  let scored = score_routes(safe, {
    minimize: [energy, compute_cost, routing_friction, policy_risk],
    preserve: [task_value, coherence, user_intent, safety_floor]
  });

  // Select the lowest-energy policy-valid transition.
  return select_lowest_energy_valid(scored);
}

```

In software terms, the governor does not first choose the best route and then ask whether policy permits it. Instead, policy narrows the candidate state space before scoring begins. This makes S_{safe} an active part of the membrane’s geometry: it determines which corridors can open, which retrieval sources can participate, which tools can be considered, which personalization updates are eligible, and which boundary traces may safely influence the next transition.

This appendix therefore connects Section 13 back to the front-door phase-smoothing equation and to Appendix G’s Elder reference architecture. Policy becomes a typed boundary condition for user-formed optimization: it shapes the possible next states of the membrane, contributes measurable transition-pruning evidence for audit and debugging, and preserves the core U.F.O. principle that user-shaped adaptation must remain bounded, reversible, compute-aware, and governable.

Appendix I: Kernel-Like Propagation Structure in V-Channels

This appendix clarifies a precise and limited interpretation of V-channels within the radial identity membrane. In this paper, V-channels should **not** be read as formal convolution memory kernels. They are not causal time-domain operators, they do not store content, and they do not imply positive-type guarantees or physical correspondence to generalized Langevin, HLV, or fluid-dynamical memory kernels.

In this limited analogy, “kernel-like” means only that a V-channel filters, weights, and constrains propagation through the membrane’s state space. It does *not* imply convolutional memory, stored content, or time-domain equivalence.

V-channels are therefore **spatial–geometric propagation structures**, not temporal memory kernels. Time enters the system only through the membrane’s update dynamics and through repeated activation patterns, not through convolution over past states.

I.1 Geometry-Aware Propagation Corridors

A V-channel between strings i and j can be interpreted as a geometry-aware propagation corridor. Its channel region Ω_{ij} , mask $\chi_{ij}(r, \theta)$, curvature weight K_{ij} , and coherence ratio $Q_{ij}(t)$ together define a constrained pathway through the interior identity field.

Data forcing enters the membrane as a ripple, but the V-channel mask shapes that ripple before it becomes string-level activation.

To make the connection explicit:

- **The mask χ_{ij}** implements the **filtering operation** in the channel-filtered forcing equation.
- **The coherence ratio $Q_{ij}(t)$** measures whether the **filtered activation remains organized** or collapses into noise.

Thus, a V-channel does not store memory; it **filters propagation by geometry, cost, coherence, and alignment**.

I.2 Architectural Functions of V-Channels

This interpretation explains why V-channels are more than visual corridors. They perform four architectural functions:

1. **Restrict propagation** to meaningful regions of the identity field rather than allowing arbitrary spread.
2. **Lower resistance** between related behavioral strings, enabling smooth transitions such as Depth $\rightarrow$ Precision $\rightarrow$ Technical Detail or Tone $\rightarrow$ Warmth $\rightarrow$ Conciseness.
3. **Expose a measurable coherence signal** through $Q_{ij}(t)$, allowing the governor to distinguish organized corridor flow from scattered activation.
4. **Shape durable adaptation pressure** by influencing routing readiness only when repeated activation remains coherent, permitted, and stable.

Any durable influence must pass through the **governed latent adaptation-pressure layer** (Section 4.6), not through raw content retention inside the V-channel itself.

I.3 V-Channels as Structured Propagation Filters

In this limited sense, a V-channel functions as a **structured propagation filter**:

- it receives an activation field,
- shapes that field through a mask,
- weights its movement through curvature,
- discounts it through cost and resistance, and
- exposes a coherence signal for validation.

The closest architectural analogy is **constrained activation transport**, not memory storage. A kernel-like V-channel makes certain routes easier, others harder, and unrelated routes less likely to dominate unless task pressure explicitly justifies expansion.

I.4 Responsible Interpretation and Claim Boundaries

This distinction matters for responsible interpretation. If V-channels were described simply as kernels, readers might incorrectly assume:

- convolution over time,
- direct memory recall,
- positive-type kernel guarantees, or
- physical correspondence to fluid-memory operators.

The intended claim is narrower: **V-channels are kernel-like because they shape propagation through a structured state space**. They are routing-and-filtering mechanisms, not memory stores or independent physical laws.

I.5 Validation and Parsimony Considerations

This interpretation suggests several ablations:

- **No-channel-mask baseline**: tests whether activation becomes noisier when χ_{ij} is removed.
- **No-curvature baseline**: tests whether string updates lose coherence when K_{ij} is neutralized.
- **No-coherence-diagnostic baseline**: tests whether the governor loses the ability to distinguish corridor flow from scattered activation.
- **Flexible-routing baseline**: tests whether a more generic routing mechanism can reproduce the same coherence, cost, and recovery behavior.

The parsimony claim is explicit: **The goal is not to prove that generic routing cannot reproduce the behavior, but to test whether structured V-channels reproduce it with fewer, more interpretable assumptions.**

I.6 Falsifiability and Future Revision

The practical takeaway is that V-channels define the membrane’s **structured propagation bias**. They do not prove uniqueness, and they should not be overclaimed as formal memory kernels. Their value is architectural: they make reasoning flow inspectable, constrainable, and testable.

If future simulations show that V-channel structure improves coherence, reduces avoidable compute, and supports faster return-to-baseline behavior relative to no-curvature or generic-routing baselines, then V-channels will have earned their role as a useful kernel-like propagation layer inside the U.F.O. governor.

Conversely, if no-channel, no-curvature, or flexible-routing baselines reproduce the same behavior with equal or fewer assumptions, then the V-channel interpretation should be treated as a useful visualization rather than a necessary architectural mechanism.

I.7 Side-String Boundary Conditions for V-Channel Propagation

A V-channel is not only the open region between two behavioral strings; it is shaped by the strings that form its sides. In this interpretation, the adjacent strings act as boundary conditions for the corridor. Their amplitudes, costs, tension, stiffness, damping, and curvature weights determine how easy it is for activation to enter the channel, how strongly it can propagate, and whether the resulting flow remains coherent enough for the governor to preserve.

For example, a Depth–Precision V-channel is not a generic route through the membrane. Depth contributes the pressure for multi-step reasoning, while Precision contributes verification pressure and claim tightening. The corridor between them becomes useful only when both side strings support the same task-aligned movement. If Depth is highly active but Precision is stiff, expensive, or suppressed, the channel may resist propagation; if both strings are aligned, moderately excited, and cost-justified, the channel becomes a low-resistance path for structured reasoning.

This side-string interpretation strengthens the kernel-like reading of V-channels. A propagation filter is not defined only by the space it admits; it is also defined by its boundary behavior. The side strings determine the channel’s **effective permeability**: high alignment and low resistance allow activation to pass smoothly, while high cost, excessive tension, or phase mismatch reduce flow. In this sense, the V-channel mask $\chi_{ij}(r, \theta)$ should be read together with the side-string states s_i and s_j , not as an isolated geometric mask.

Architecturally, this makes each V-channel a **coupled three-part structure**: the left boundary string, the right boundary string, and the interior corridor between them. The interior field carries the activation ripple; the side strings define semantic **meaning and resistance**; and the governor decides whether the resulting flow is useful, bounded, and policy-valid. This explains why V-channels can remain interpretable: each corridor is named not by an abstract kernel label, but by the behavioral strings that bound it.

Validation can test this boundary-condition interpretation directly. A side-string ablation can hold the corridor mask constant while neutralizing one boundary string's cost, tension, or curvature contribution. If channel coherence changes under that intervention, then the V-channel is not **merely a visual region**; it is **dynamically shaped by the strings that define its sides**. This would make V-channels more defensible as structured propagation filters inside the radial identity membrane.

Postscript: Research Translation Horizon for U.F.O.

This postscript is included as a research-translation reflection rather than as a business plan, product directive, or deployment recommendation. My background in business helped shape the original intuition behind U.F.O. before the research artifact became mathematical: if adaptive AI is going to become part of everyday work, then the technical control layer and the user's felt experience of control are likely best studied together. The membrane began, in part, from that perspective — not only as a way to govern compute, but as a way to make personalization feel visible, intentional, and trustworthy to the person using the system.

From that research-translation viewpoint, U.F.O. may be best understood through a dual-MVP agenda. The first path is a **technical MVP**: an inference-time Copilot controller that tests whether the governor can improve runtime orchestration, compute allocation, retrieval budget, tool thresholds, reasoning depth, response length, policy-bounded behavior, and return-to-baseline dynamics without modifying base model weights. This path asks whether U.F.O. can function as smart infrastructure above existing AI systems — a governed layer of runtime control rather than a rewrite of the model itself.

The second path is a **trust MVP**: a user-facing radial questionnaire or calibration map that tests whether visible, reversible personalization increases perceived control, transparency, trust, and willingness to rely on Copilot for higher-value work. In this framing, the radial map is not merely a visualization of internal parameters; it becomes a research instrument for studying whether users trust adaptive AI more when they can see, adjust, reset, and understand the behavioral profile that shapes the system's responses.

The research question is therefore not only whether the controller can reduce avoidable compute, but whether a visible calibration interface changes the user's relationship with Copilot. A backend-only controller may make the system more efficient; a transparent calibration surface may make the system feel more user-owned. The stronger hypothesis is that the two paths may be most useful when evaluated together: a governed runtime controller for efficiency and safety, paired with a user-visible map for trust, agency, and responsible personalization.

This distinction matters because trust is not secondary to technical performance in user-facing AI. When users understand how Copilot is adapting, believe they can steer that adaptation, and know they can reverse or recalibrate it, they may be more willing to rely on the system for deeper workflows across writing, communication, meetings, planning, analysis, and enterprise tasks. In research terms,

this suggests that engagement depth, perceived value, correction rate, reuse intention, cross-surface continuity, and willingness to rely on Copilot should be studied alongside tokens, latency, retrieval count, tool calls, policy blocks, and quality metrics.

A disciplined research program could therefore ask three linked questions. First, can an inference-time U.F.O. controller improve Copilot task quality, compute efficiency, and policy compliance without changing model weights? Second, can a radial calibration interface increase user trust, perceived control, transparency, and willingness to rely on personalized AI? Third, does the combined technical-plus-trust MVP outperform a backend-only controller by improving both system efficiency and the user's sense of agency?

This postscript does not argue that U.F.O. should be rushed into product form. It argues that the path from research artifact to practical evaluation may require both an infrastructure lens and a human-centered lens. The controller tests whether the membrane can govern behavior intelligently; the radial questionnaire and map test whether users can understand and trust that governance. For a system like Copilot, those questions may be inseparable.

The most useful next step is therefore not to treat U.F.O. as a finished architecture, but to translate it into testable research workstreams: controller behavior, compute efficiency, policy-bounded state selection, telemetry, ablation baselines, radial calibration UX, boundary-loop traces, and single-surface prototypes. In that sense, the dual-MVP framing gives this paper a practical closure: U.F.O. is both a technical hypothesis about governed runtime control and a user-experience hypothesis about trust in adaptive AI.

AI Assistance Statement

This document was authored for Microsoft review and was developed with assistance from Microsoft Copilot. Copilot was used to accelerate drafting, organization, editing, formatting, and refinement. The core research direction, architectural concepts, mathematical framing, validation priorities, and final editorial decisions remain the responsibility of the author.

Acknowledgements

The author gratefully acknowledges the Microsoft teams whose research, engineering, design, safety, and product work have made Copilot possible. Copilot meaningfully accelerates the pace of research by helping users organize ideas, refine drafts, test explanations, improve structure, and move from concept to polished artifact with far greater speed. This paper is offered in that spirit of appreciation: as a respectful research contribution inspired by the progress Microsoft has enabled and intended to invite further critique, simulation, and collaborative exploration with Microsoft research and product teams.

Version Control

Version	Date	Status	Summary
1.0.0	June 15, 2026	Initial complete release	Completed core research artifact, including the radial identity membrane, U.F.O. governor, compute-aware personalization framework, Project Rainbow toy simulation scaffold, validation roadmap, and Elder reference architecture. Produced the Boundary-loop/geometric caching interpretation, expanded Data Flow Overview, CODA validation assumption, Appendix G.7.7 OS-level Elder orchestration.
1.0.1	June 15, 2026	Refinement release	Added the word: <i>Dynamic</i> in title (refinement). Added the Appendix navigation guide and formatting (horizontal lines).
1.1.0	June 15, 2026	Refinement release	Added ALCOA date detail to the front matter and updated the CODA to frame the architecture around AI OS orchestration, steerable interaction, adaptive control surfaces, geometric caching, and governed adaptation.
1.1.1	June 16, 2026	Refinement release	Updated the CODA title to <i>CODA: A Layered Horizon for Governed Adaptation</i> , refining the closing frame around steerability, interaction geometry, adaptive control surfaces, geometric caching, engineering validation, and the broader computer-science theme of governed adaptation. Added an Appendix G out-of-scope Elder note clarifying its broader potential for typed AI orchestration, Co-Pilot-agent substrates, and framework comparison.
1.2.0	June 16, 2026	Architecture refinement release	Added “Common Trend and Smoother Transitions in Phases,” introducing a phase-efficient governor pass with an energy-minimized transition equation and Elder-style pseudocode for smoother, lower-cost, policy-valid transitions across the membrane pipeline.

1.2.1	June 16, 2026	Reader-clarity refinement release	Refined the Phase-Smoothing Flow with a no-color three-column table, added the Optimization / Guardrail column, improved the Data Flow segue, and clarified U.F.O. as continuous, bounded, lower-energy phase transition selection rather than static personalization.
1.3.0	June 16, 2026	Policy architecture refinement release	Added Section 13, “Policy as a Runtime Constraint Surface,” making policy explicit as a pre-optimization constraint on valid next states, policy-bounded personalization, measurable transition pruning, and a future typed boundary layer for Elder-style orchestration.
1.3.1	June 16, 2026	Policy-Elder appendix release	Added Appendix H, “Policy-Bounded State Selection in Elder,” defining S_{safe} as a policy-filtered candidate state set and showing how predicates, risk scoring, and Elder-style orchestration select the lowest-energy policy-valid membrane transition.
1.3.2	June 16, 2026	CODA refinement release	Normalized recent version numbering to better match release scope while preserving chronology and dates; retained 1.3.2 as the current stable submission version and documented the scoped CODA coherent-excitation paragraph.
1.4.0	June 16, 2026	Appendix F and acknowledgements refinement release	Expanded Appendix F figure captions into reviewer-facing interpretations of initialization, task activation, routing, cost/energy stability, ablation behavior, V-channel coherence, and recovery dynamics. Updated the Acknowledgements to express gratitude to the Microsoft teams that built Copilot and to recognize Copilot’s role in accelerating research, organization, drafting, refinement, and progress.

2.0.0	June 16, 2026	Research translation horizon release	Added the Postscript, “Research Translation Horizon for U.F.O.,” framing U.F.O.’s next research path as a dual-MVP agenda: an inference-time Copilot controller as the technical MVP and a radial calibration interface as the trust MVP. Positioned the agenda as research insight rather than product directive, with evaluation questions around compute efficiency, policy-bounded behavior, user trust, perceived control, engagement depth, responsible personalization, and whether a paired technical-plus-trust MVP outperforms a backend-only controller.
2.1.0	June 17, 2026	Refinements	Added front-door systems-engineering framing, including Conceptual Design, Requirements Analysis, Functional Analysis, Functional Synthesis, System Integration, Components and Interfaces, Systems-of-Systems Framing, and an expanded Figure 1 caption with multi-audience reviewer entry points.)
2.2.0	June 17, 2026	Interpretability and Technical Clarifications	Strengthened the paper’s mathematical clarity and reviewer accessibility through guided interpretive analogies, smoother transitions into and out of complex math sections, clearer term-by-term explanation of the governed wave equation, expanded phase-smoothing intuition, notation cleanup for curvature, cost, propagation, and depth-update equations, and a more precise treatment of the latent training plane as governed adaptation pressure rather than model training, persistent memory, or raw interaction storage. Major mathematical equations now include formal names to improve cross-reference, reviewability, and technical navigation.
2.3.0	June 17, 2026	Appendix I	V-Channel Propagation Interpretation Update. Added Appendix I to clarify V-channels as kernel-like, geometry-aware propagation filters rather than formal convolution memory kernels, including the role of side strings as boundary conditions that shape channel permeability, coherence, resistance, and validation through side-string and no-channel ablations. Added clarification in subsection 4.5, second paragraph.

2.4.0	June 18, 2026	Section 4.7	Added the deformable membrane supplement, replacing the fixed circular boundary assumption with a live polar boundary driven by sustained behavioral-string activation. This enables geometry-aware governor routing through tangent expressivity, asymmetry, peak deformation angle, curvature, and relaxation dynamics; strengthens compute allocation; and improves the transition into geometric AI identity. Added a high-level closing sentence to articulate this distinction in the Figure 1 caption.
2.5.0	June 18, 2026	Subsection 4.7.8	Added bidirectional boundary-geometry formalization to the deformable membrane, defining radius deviation, tangent/slope, curvature, and inward collapse as governor-readable signals. This extends v2.4.0 from expressive deformation to geometry-aware control, allowing the governor to distinguish outward activation, inward stabilization, coherent corridor reinforcement, and unstable boundary transitions.
2.6.0	June 18, 2026	Subsection 7.1.2	Added a multidimensional cost taxonomy and quality-preserving reduction layer. Cost is now modeled as a normalized vector of token, reasoning-depth, context, retrieval, tool, latency, correction/regeneration, and instability-recovery signals, then projected into observable cost through explicit weights. This clarifies that U.F.O. minimizes avoidable compute, not useful reasoning, preserving task success, safety, accessibility, correctness, user trust, and requested depth while suppressing low-value expansion.

Version 2.6.0 is intended as the stable, final submission draft of this research supplement. It represents the completed architectural statement and accompanying toy-simulation exercise for this artifact.

Future work may extend the simulation, refine validation methods, or incorporate prototype-level feedback, but such developments should be treated as **new research artifacts or follow-on iterations**, not incremental edits to this document. The conceptual architecture presented here is complete for its intended scope; subsequent contributions will build outward rather than modify the core draft.